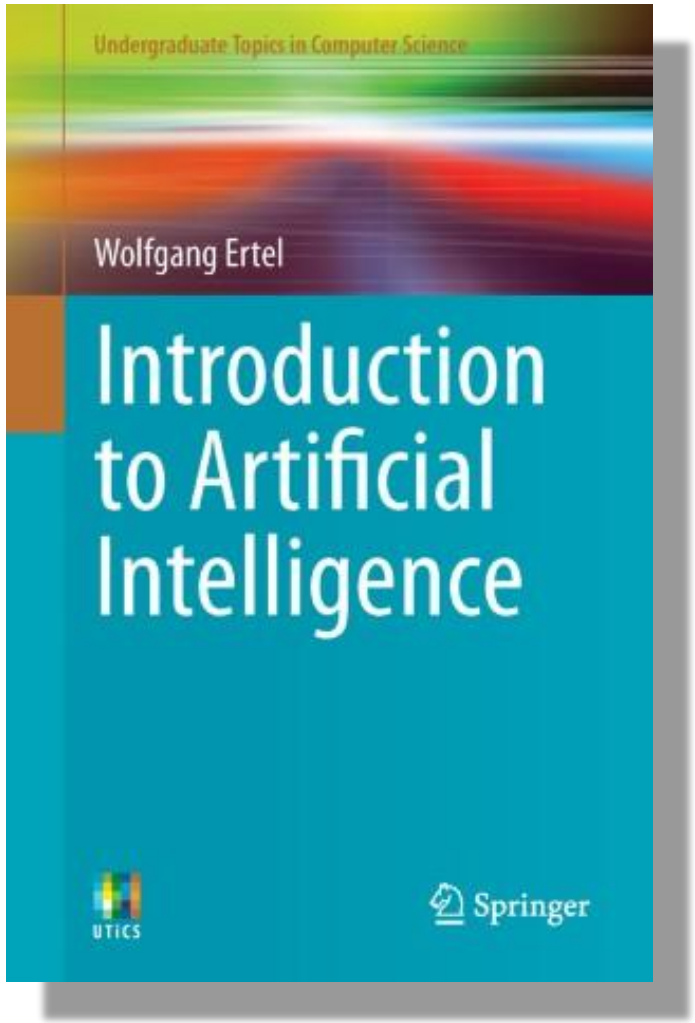
AI Fundamentals

Chapter 2- Machine Learning & Data Mining



hogeschool VIVES

VIVES University of Applied Sciences
Bachelor in electronics-ICT

AI Fundamentals

Chapter 2: Machine Learning & Data Mining

- We would like to thank the following people for their contribution:
  - Stefaan Haspeslagh, VIVES
  - Yves Sagaert, VIVES
  - Andy Louwyck, VIVES
  - Prof. Danny De Schreye, KU Leuven

AI Fundamentals

Chapter 2: Machine Learning & Data Mining

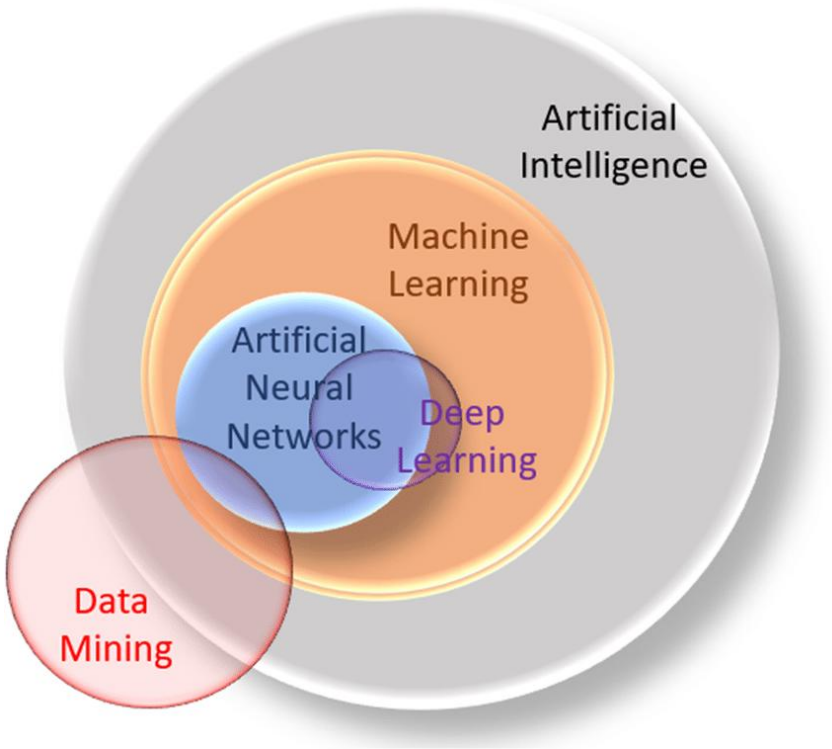
Contents

AI Fundamentals

Chapter 2: Machine Learning & Data Mining

1. Introduction – The difference between AI and Machine Learning

- **Artificial Intelligence** (AI) is a concept of creating intelligent machines that simulate human behaviour whereas **Machine learning** is a subset of Artificial Intelligence that allows machines to learn from data without being programmed.



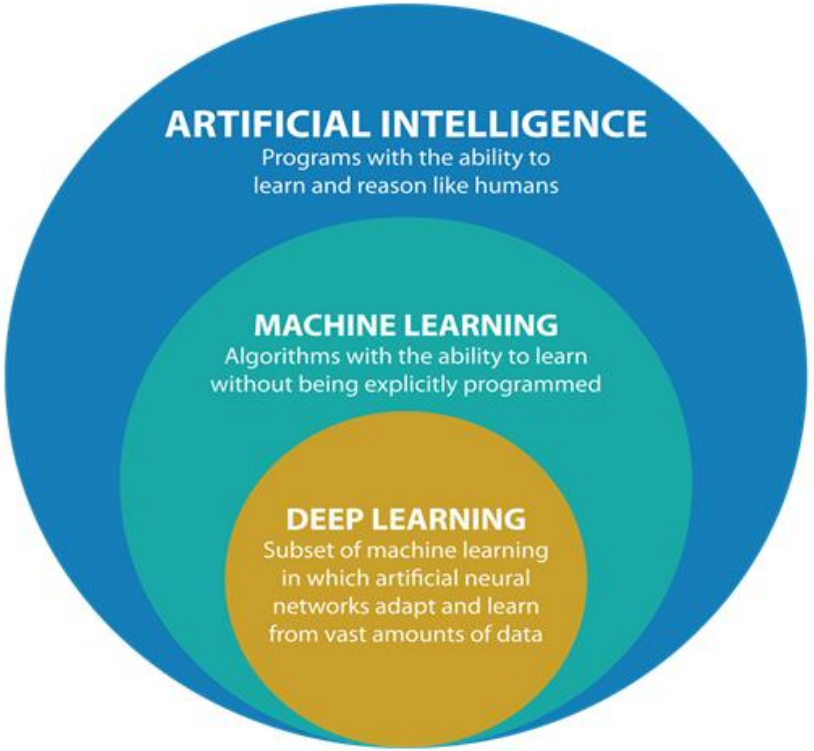
URL: Relation between AI and Machine Learning

AI Fundamentals

Chapter 2: Machine Learning & Data Mining

1. Introduction – The difference between AI and Machine Learning

- **Artificial Intelligence** (AI) is a concept of creating intelligent machines that simulate human behaviour whereas **Machine learning** is a subset of Artificial Intelligence that allows machines to learn from data without being programmed.



URL: Relation between AI, Machine Learning & Deep Learning

Chapter 2: Machine Learning & Data Mining

1. Introduction - What is Machine Learning?

- **Machine learning** is an application of **artificial intelligence** (AI) that involves **algorithms** and **data** that *automatically analyse* and *make decision by itself* without human intervention.
- It describes how computers perform tasks on their own by previous **experiences**.
- Therefore we can say in machine language that artificial intelligence is generated on the basis of **experience**.

Chapter 2: Machine Learning & Data Mining

1. Introduction – Normal computer software versus Machine Learning

- The *difference* between **normal computer software** and **machine learning** is that a human developer hasn't given codes that instructs the system how to react to a situation.

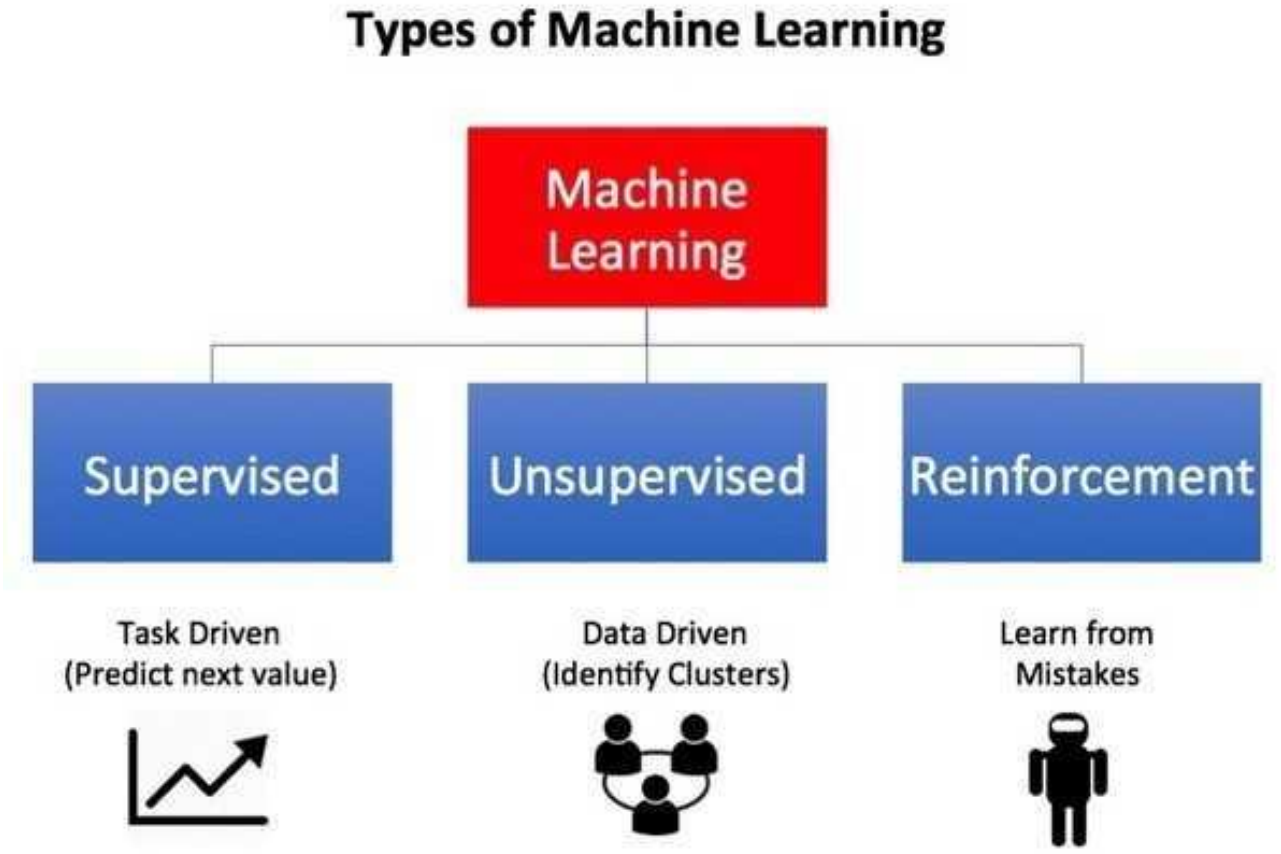
- Instead the system is being **trained** by a large number of **data**.

AI Fundamentals

Chapter 2: Machine Learning & Data Mining

1. Introduction – Types of Machine Learning

- There are three types of machine learning:
  - **Supervised learning**
  - **Unsupervised learning**
  - **Reinforcement learning**

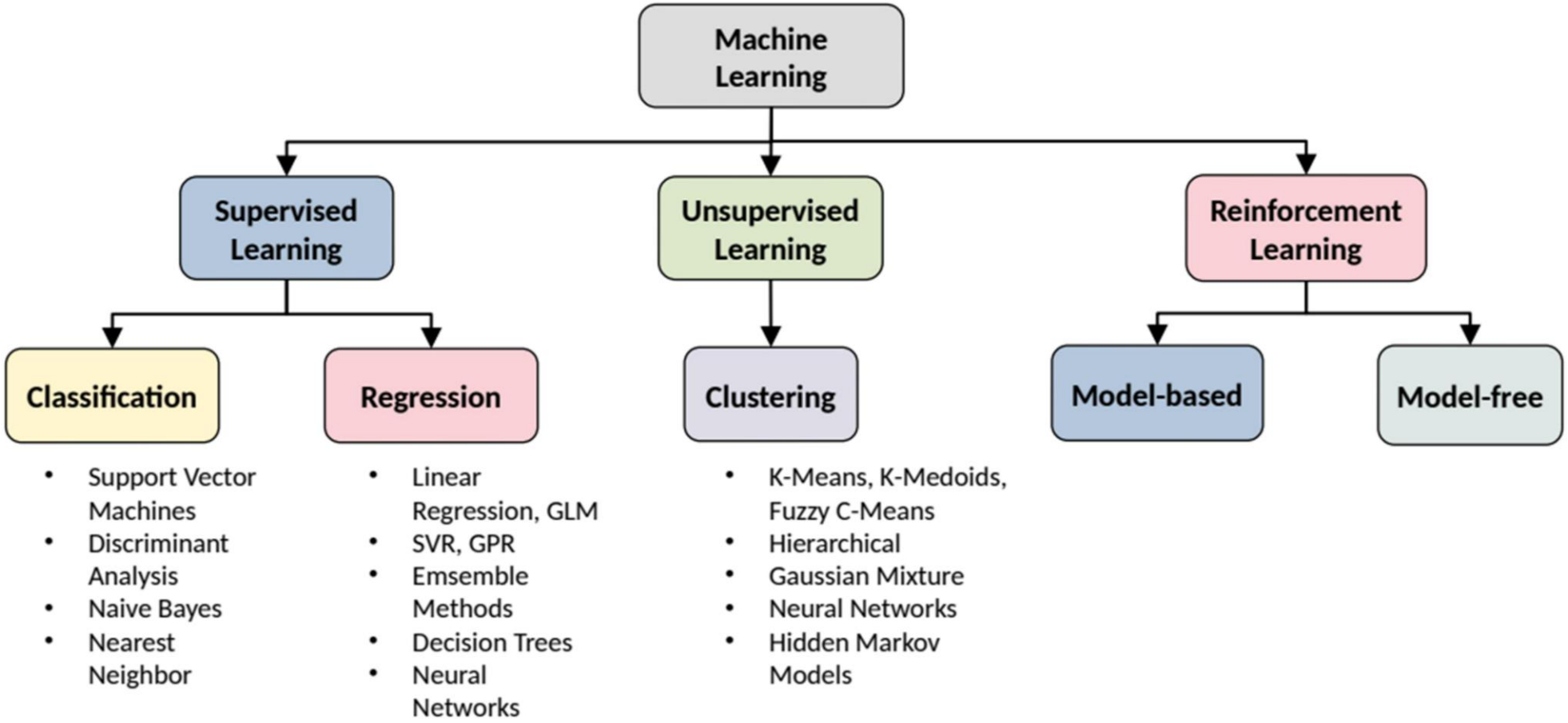


URL: Supervised, Unsupervised & Reinforcement Learning

AI Fundamentals

Chapter 2: Machine Learning & Data Mining

1. Introduction – Overview of Machine Learning Technologies

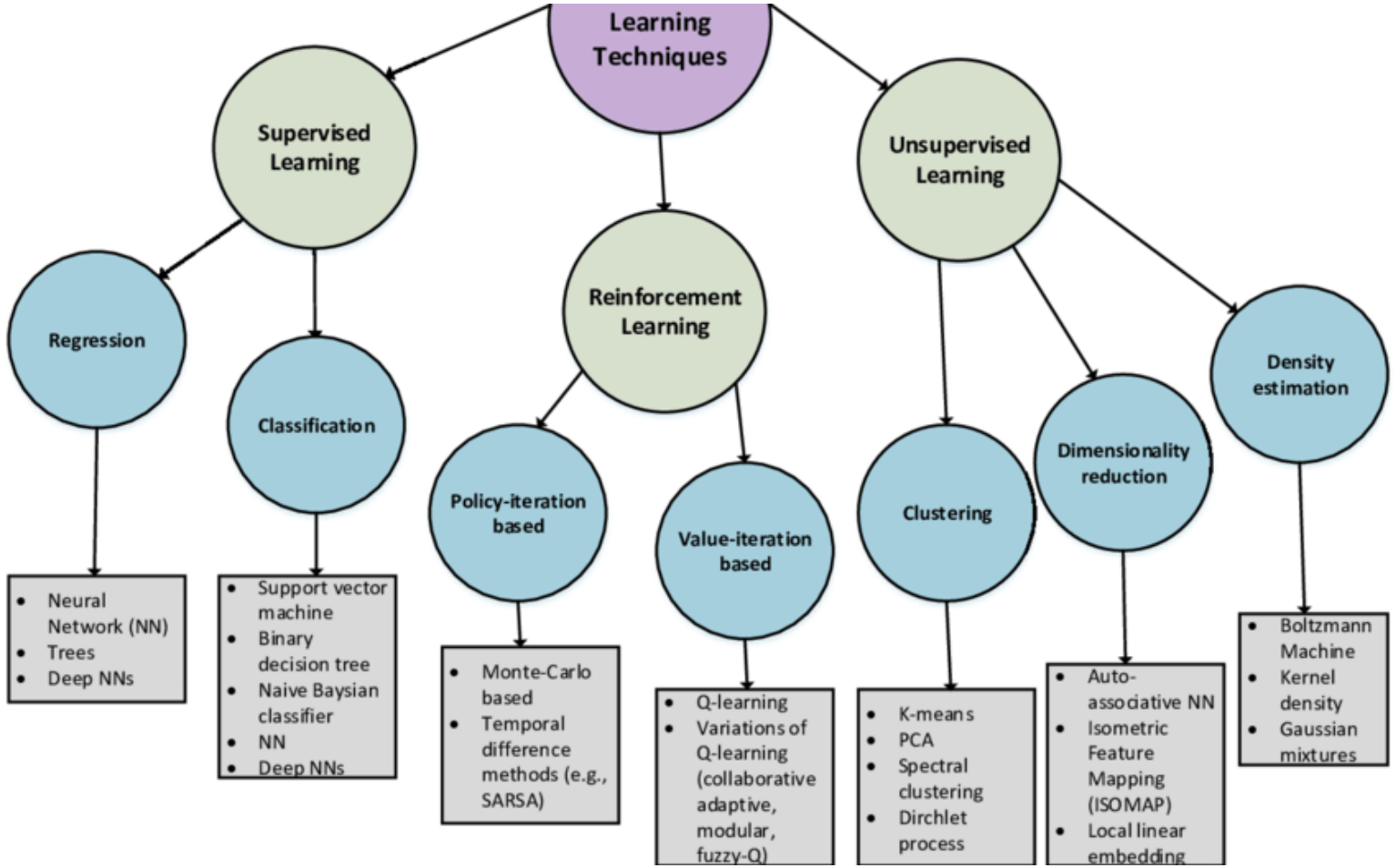


URL: Overview of Machine Learning Technologies

AI Fundamentals

Chapter 2: Machine Learning & Data Mining

1. Introduction – Overview of Machine Learning technologies

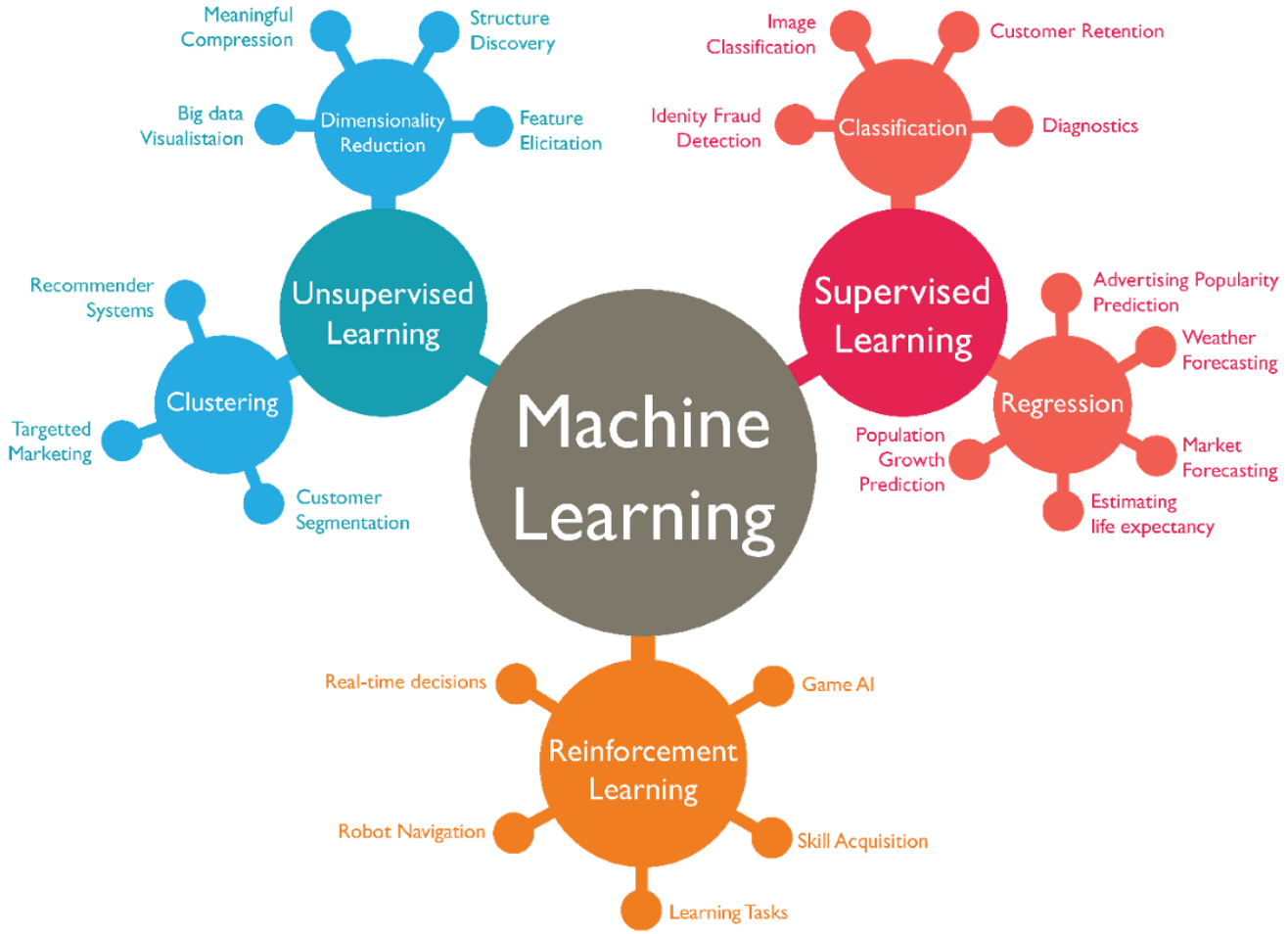


URL: Overview of Machine Learning Technologies

AI Fundamentals

Chapter 2: Machine Learning & Data Mining

1. Introduction – Overview of Machine Learning technologies



URL: Overview of Machine Learning Technologies

Machine learning tasks

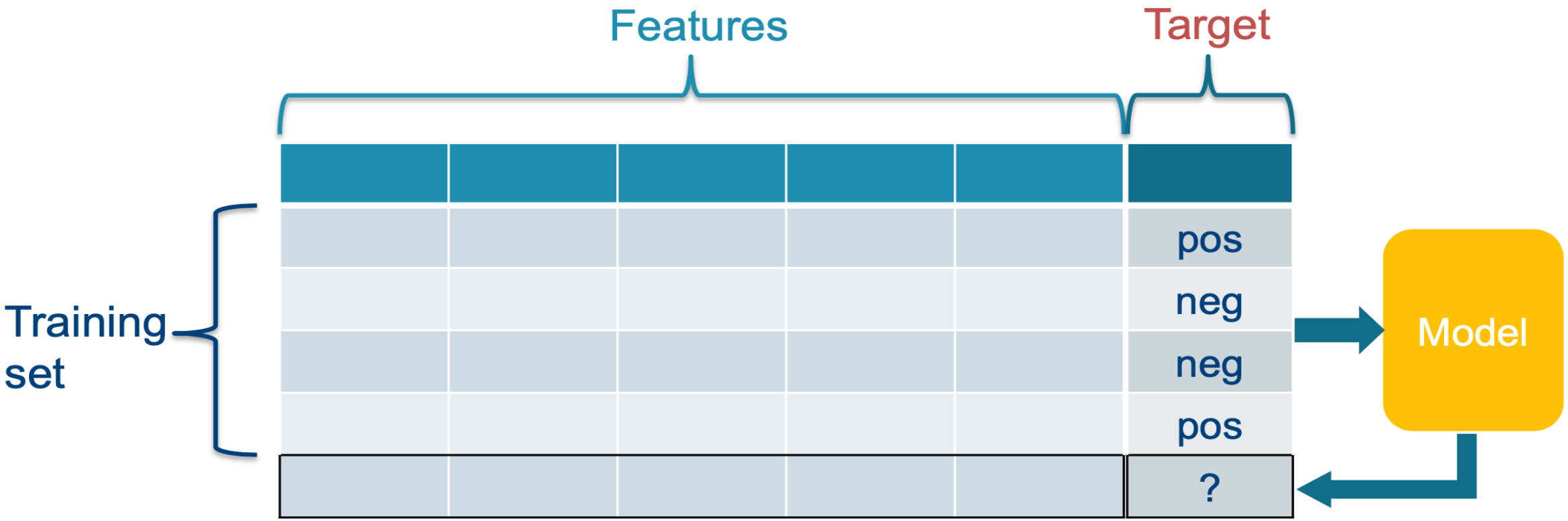
- **Some examples of Machine Learning tasks**:
  - Classification
  - Regression
  - Clustering
  - Association rule mining
  - Anomaly detection
  - …
- **Supervised Learning?**

Prediction → **supervised learning** → learning from {input, output} pairs

Chapter 2: Machine Learning & Data Mining

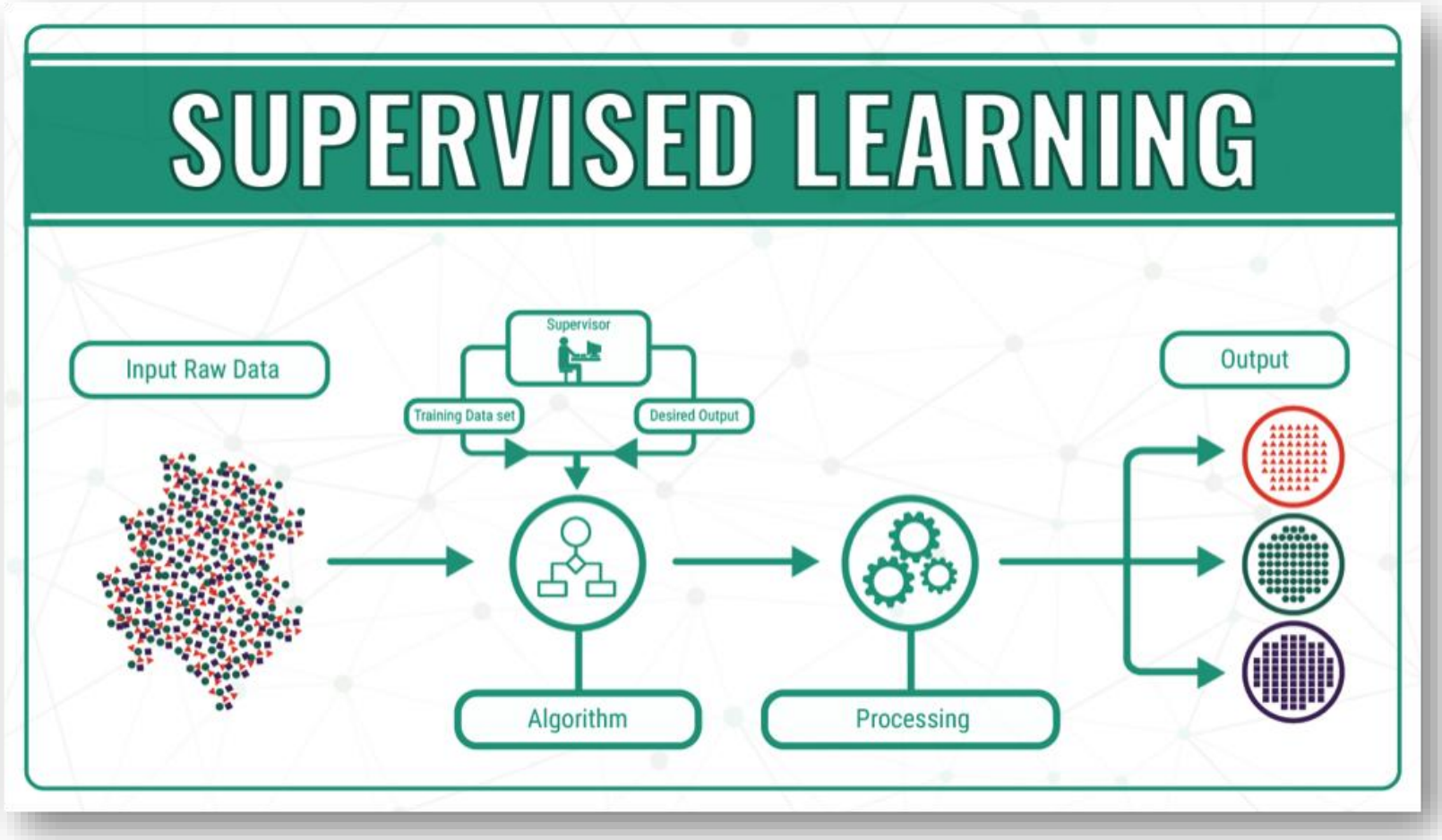
Machine Learning: supervised learning

- **Task**: learn a **model** to **predict** a property (target) for new data instances, based om **training set** of data instances for which the property is observed



Chapter 2: Machine Learning & Data Mining

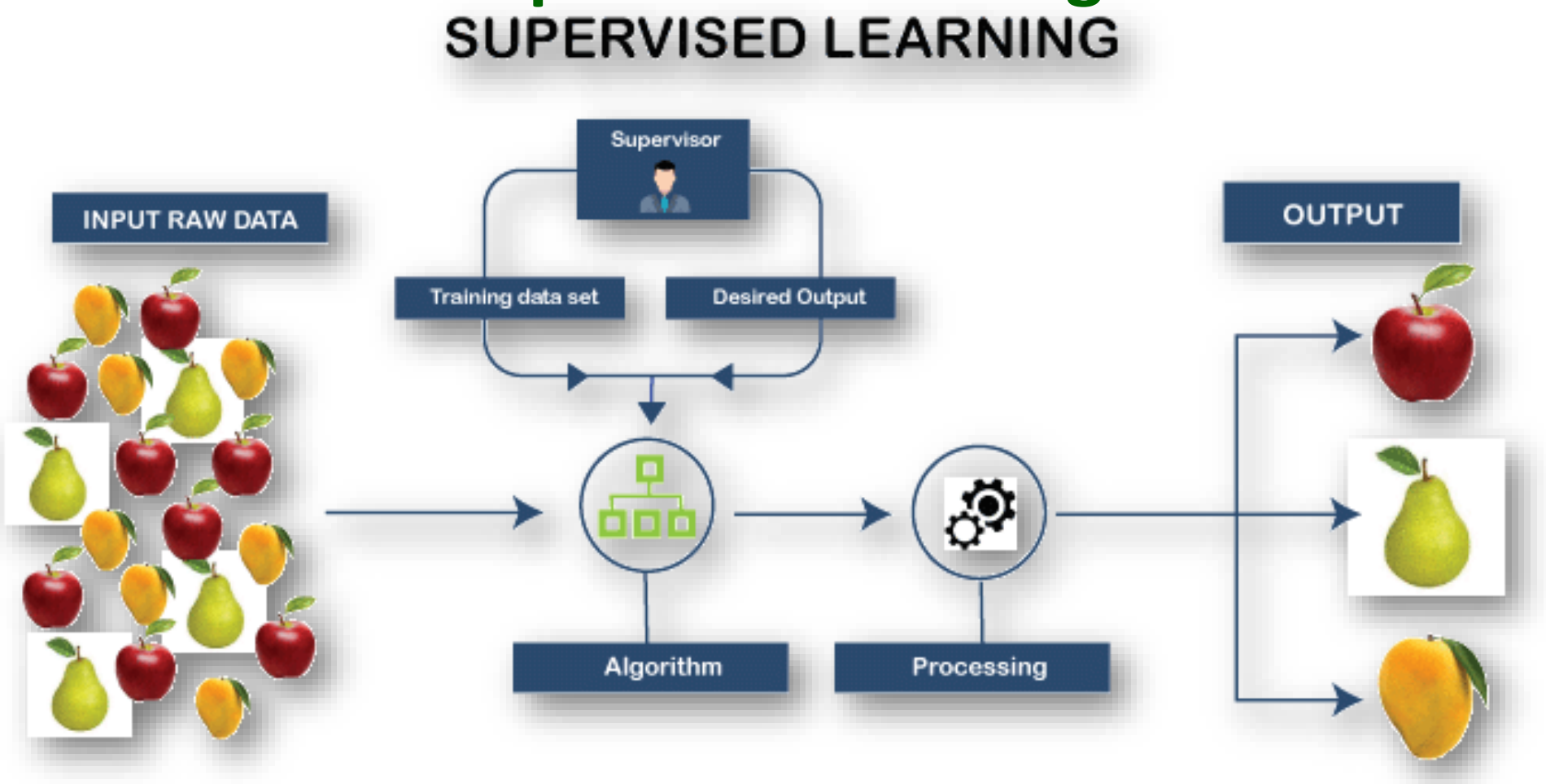
Supervised Learning



URL: Supervised Learning

Chapter 2: Machine Learning & Data Mining

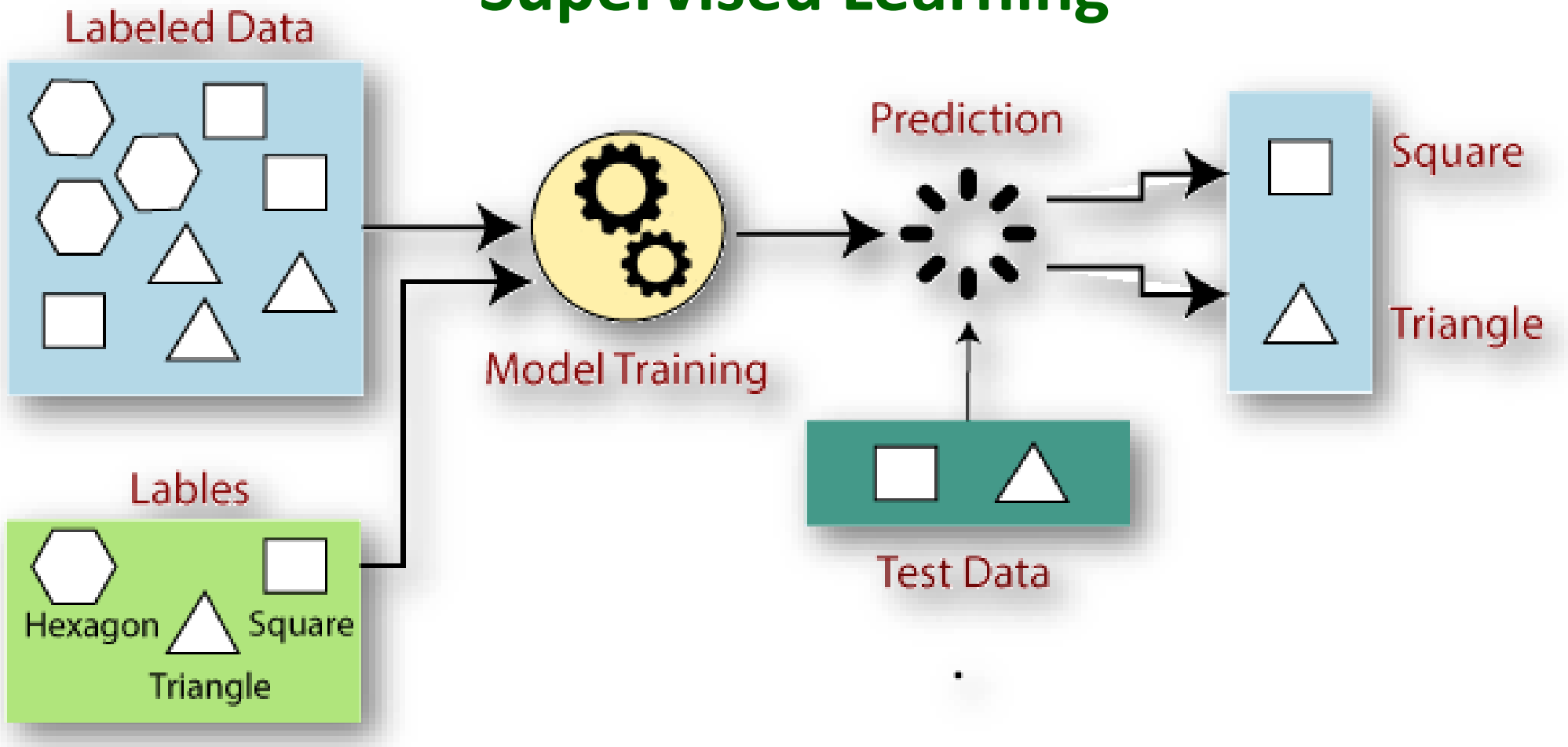
Supervised Learning



URL: Supervised Learning

Chapter 2: Machine Learning & Data Mining

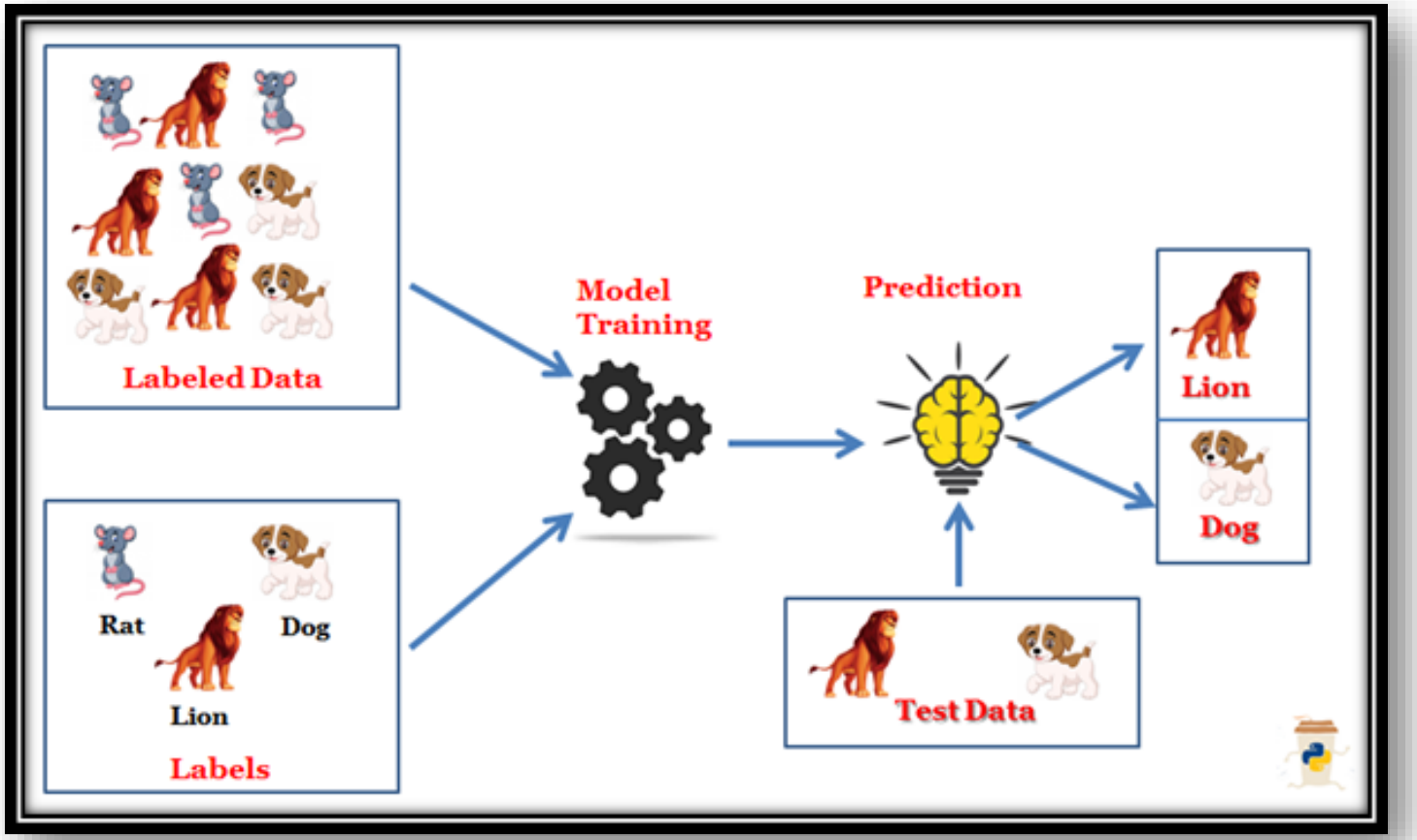
Supervised Learning



URL: Supervised Learning

Chapter 2: Machine Learning & Data Mining

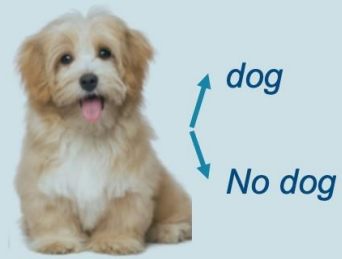
Supervised Learning



URL: Supervised Learning

Chapter 2: Machine Learning & Data Mining

Machine Learning: supervised learning tasks



Classification
(binary / multi-class)

Regression

Time-to-event prediction

Learning to rank

Link (interaction)
prediction

Multi-output prediction

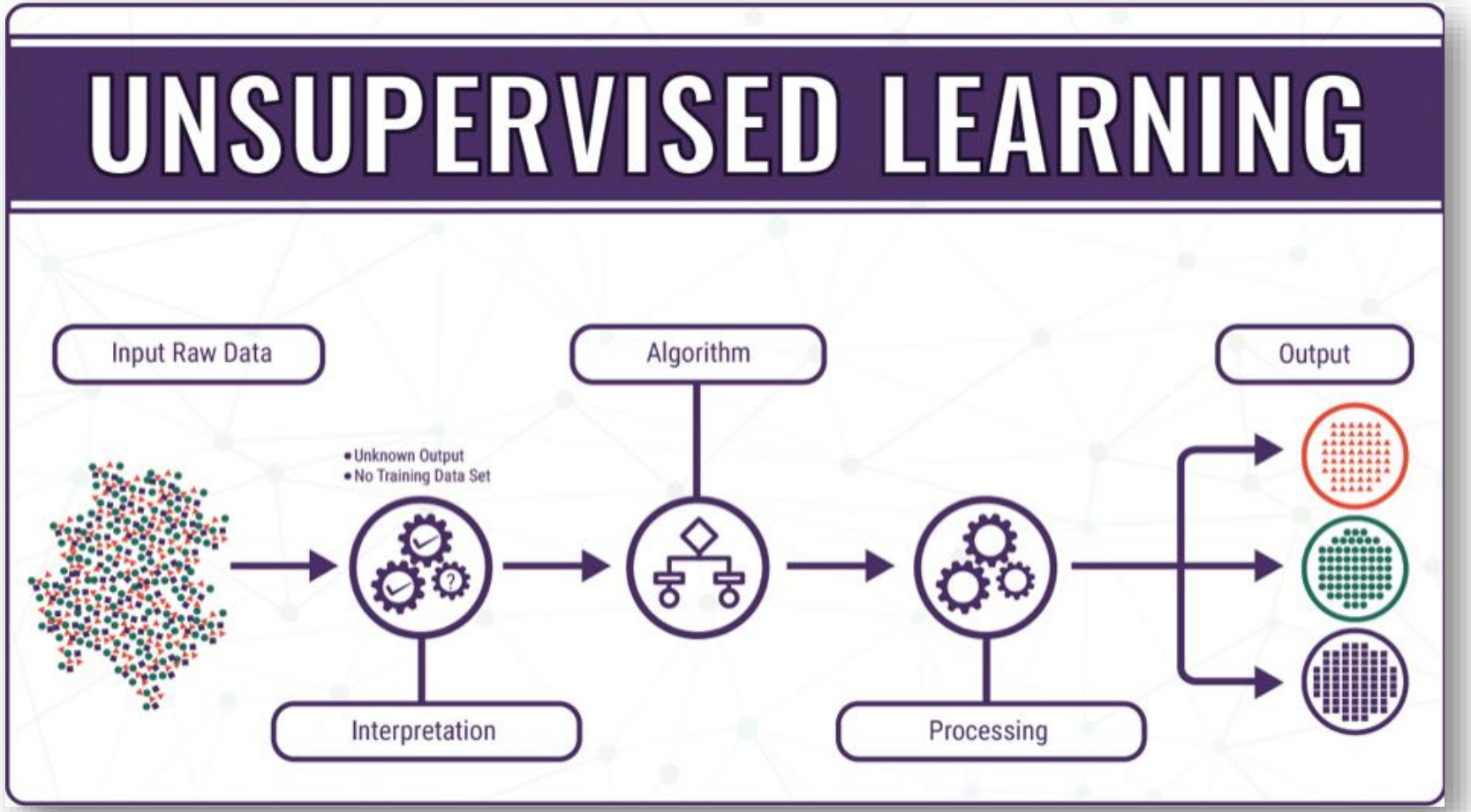
Machine learning tasks

- **Some examples of Machine Learning tasks**:
  - Classification
  - Regression
  - Clustering
  - Association rule mining
  - Anomaly detection
  - …
- **Unsupervised Learning?**

Prediction ➔ **unsupervised learning** ➔ learning from *unlabeled* input data

Chapter 2: Machine Learning & Data Mining

Unsupervised Learning

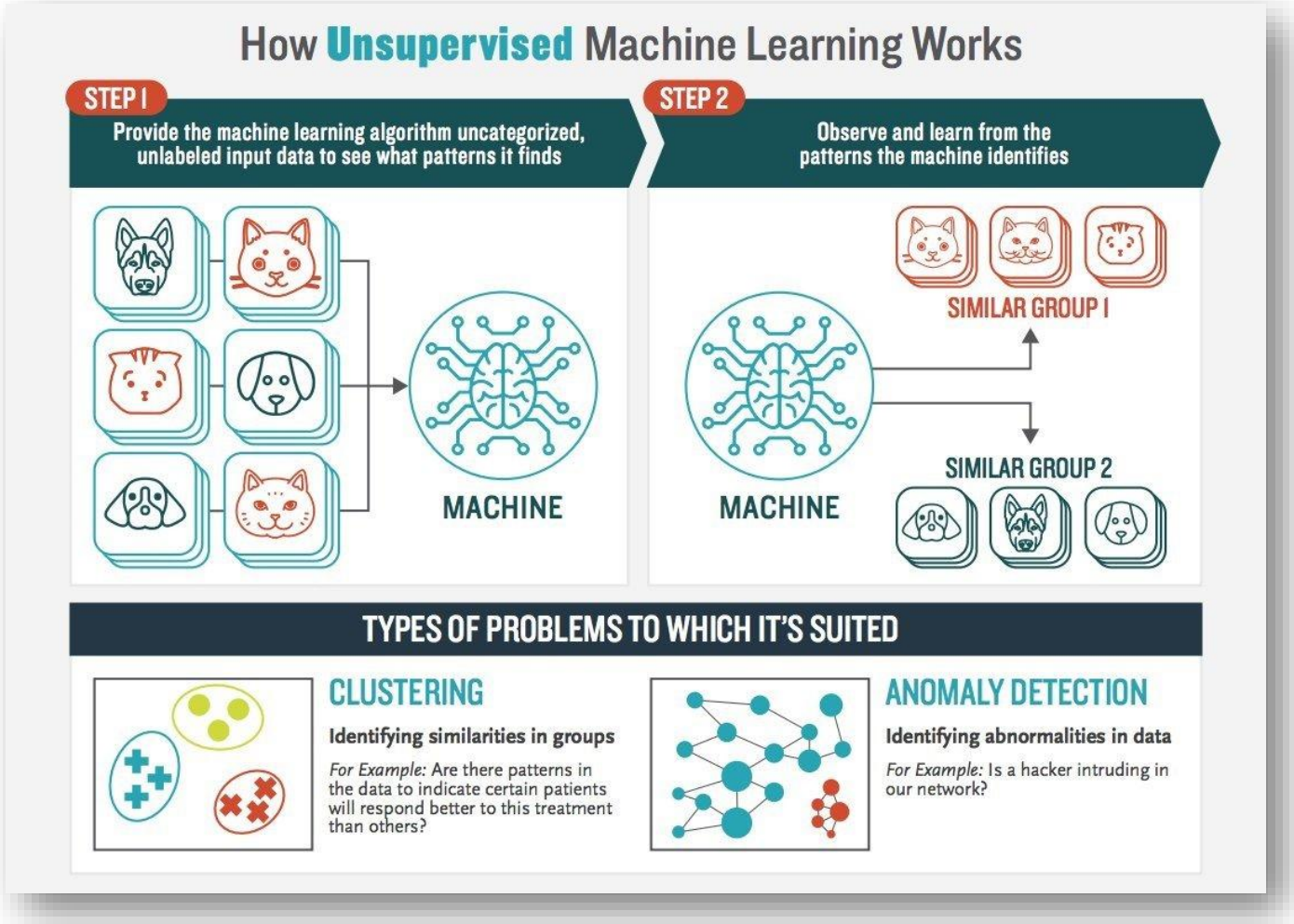


URL: Unsupervised Learning

AI Fundamentals

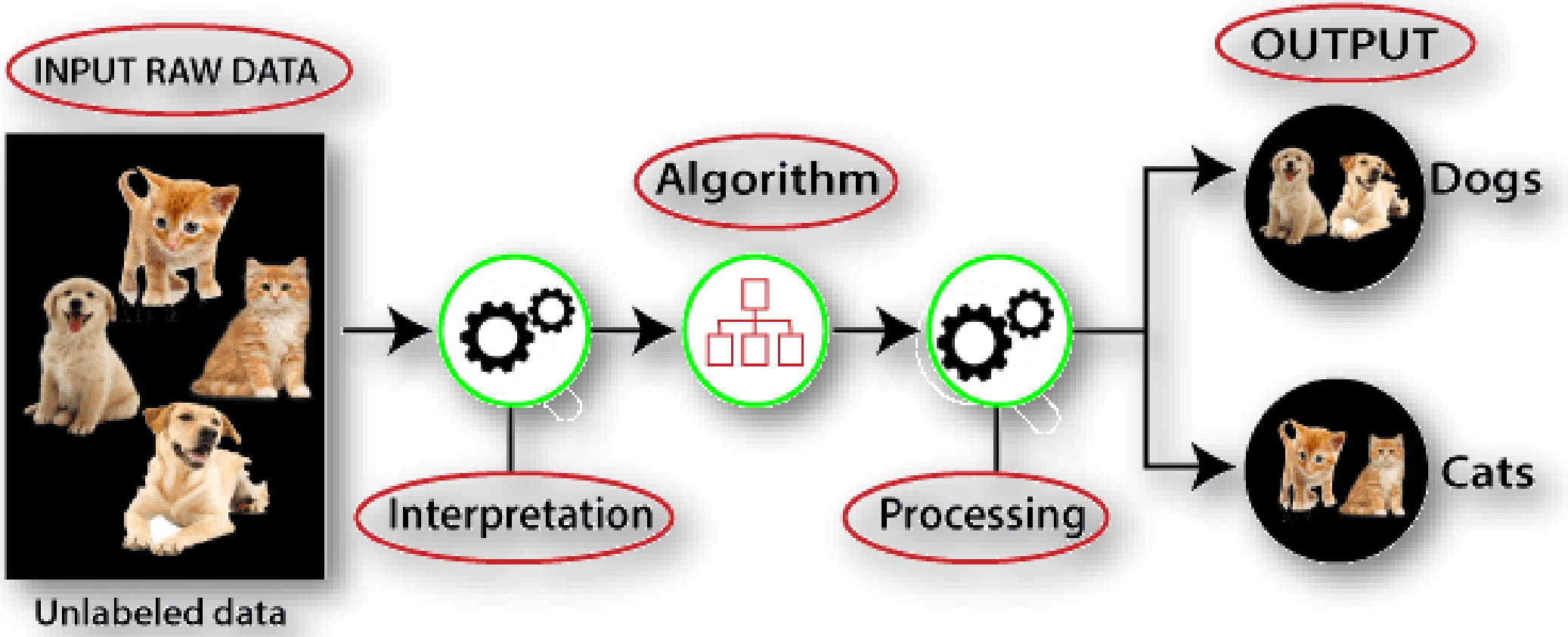
Chapter 2: Machine Learning & Data Mining

Unsupervised Learning



URL: Unsupervised Learning

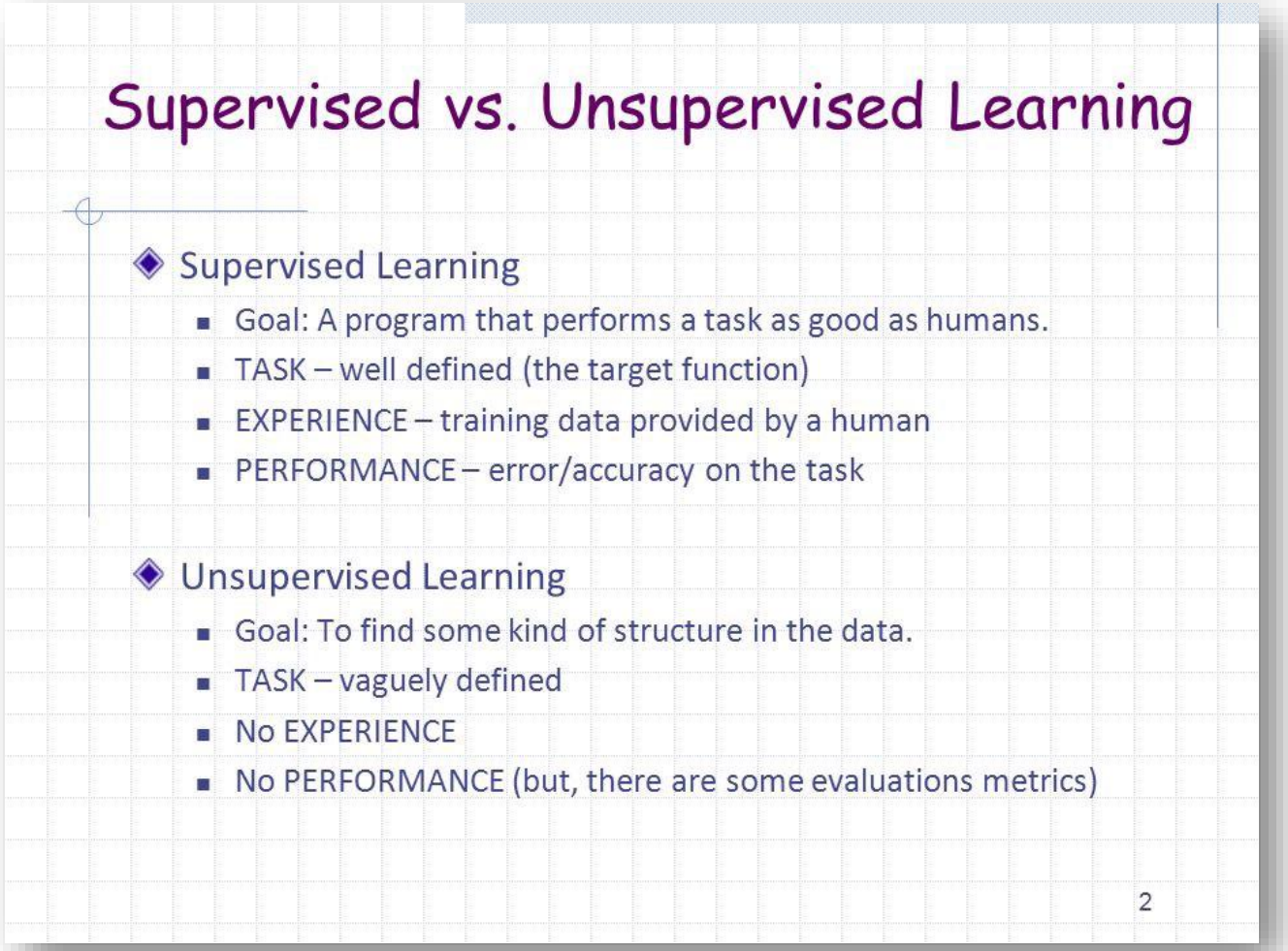
Unsupervised Learning



URL: [Unsupervised Learning]

Chapter 2: Machine Learning & Data Mining

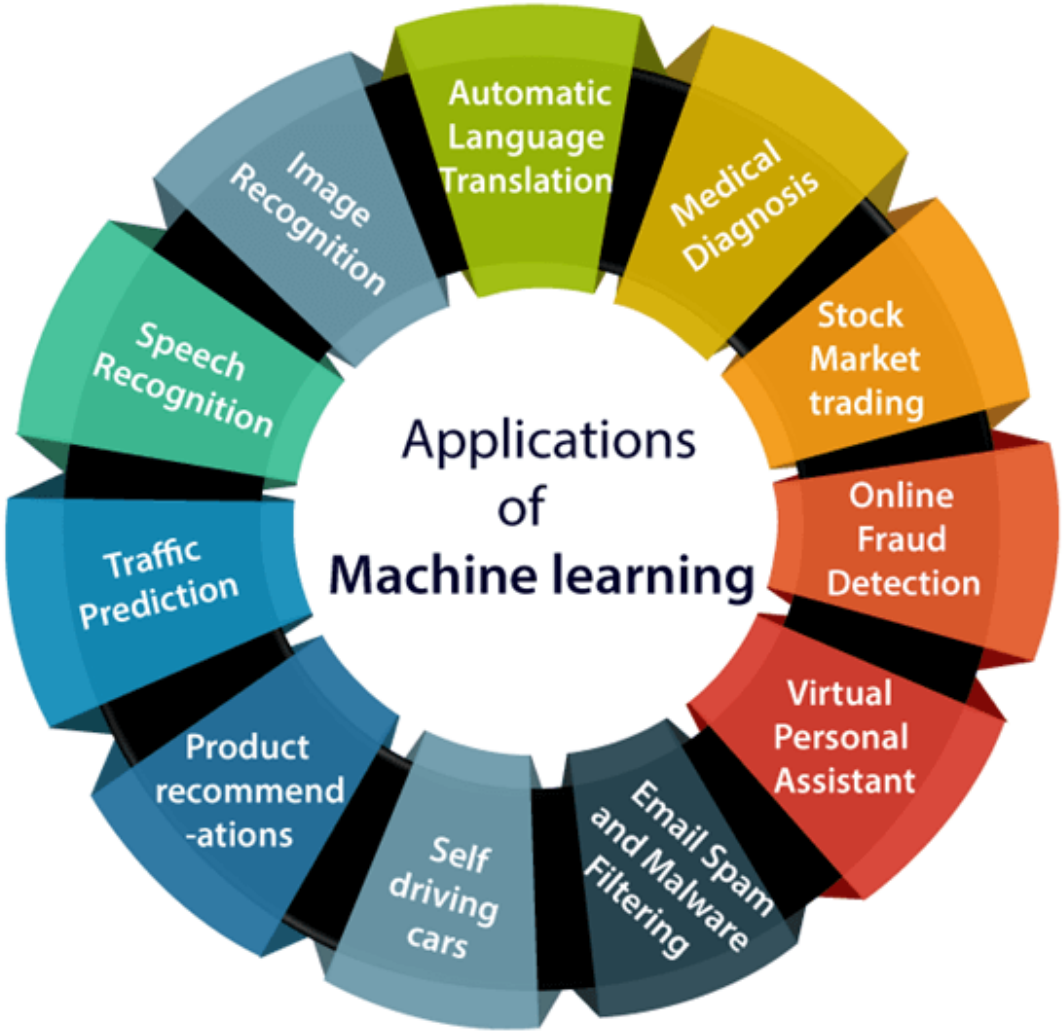
Supervised versus Unsupervised Learning



Chapter 2: Machine Learning & Data Mining

1. Introduction – Some Machine Learning applications

- Automatic Language Translation
- Medical Diagnosis
- Stock Market trading
- Online Fraud Detection
- Virtual Personal Assistant
- Email Spam and Malware Filtering
- Self driving cars
- Product recommendations
- Traffic prediction
- Speech recognition
- Image recognition



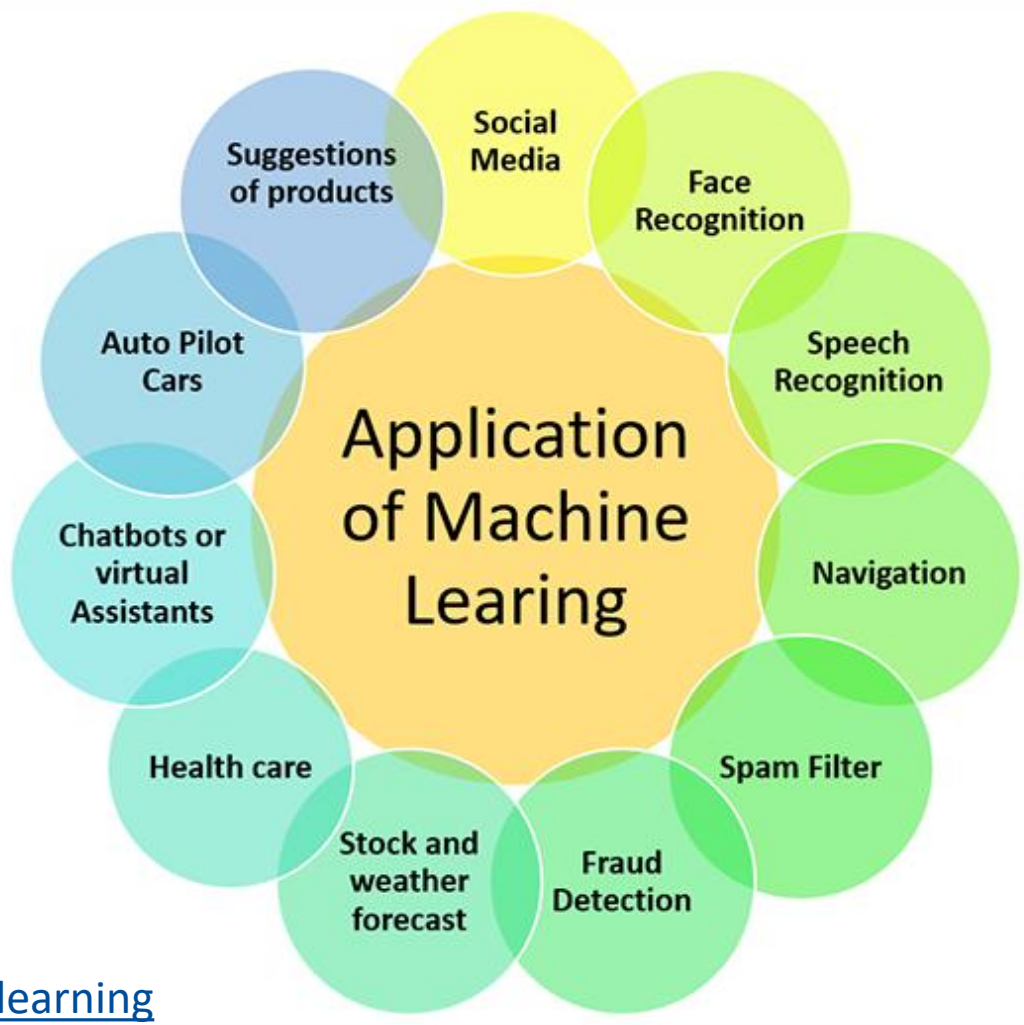
URL: Applications of Machine learning

AI Fundamentals

Chapter 2: Machine Learning & Data Mining

1. Introduction – Some Machine Learning applications

- Social Media
- Face Recognition
- Speech Recognition
- Navigation
- Spam Filter
- Fraud Detection
- Stock and Weather forecast
- Healthcare
- Chatbots or Virtual Assistants
- Auto Pilot Cars
- Suggestions of products
- Automatic Machine Translation
- Automatic Game Playing



URL: Applications of Machine learning

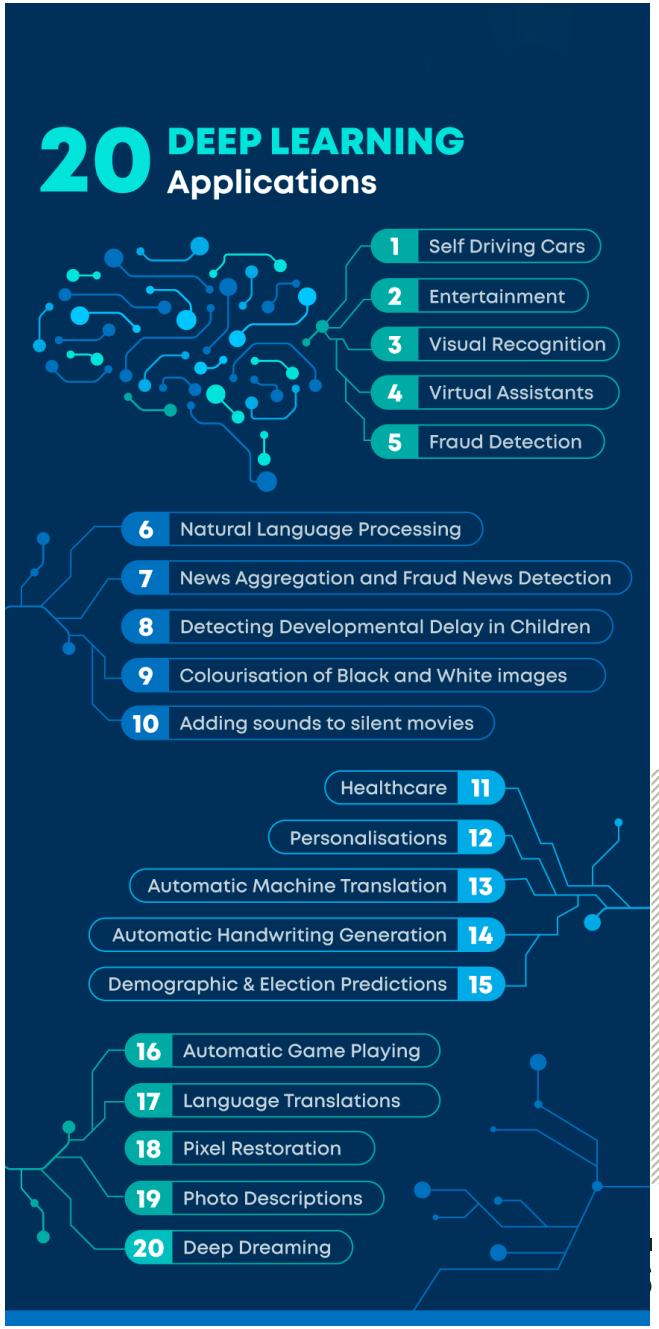
AI Fundamentals

Chapter 2: Machine Learning & Data Mining

1. Introduction – Some Machine Learning applications

- Deep Learning applications:
  - Self Driving Cars
  - Entertainment
  - Visual Recognition
  - Virtual (Personal) Assistants
  - Fraud Detection
  - Natural Language Processing
  - News Aggregation and Fraud News Detection
  - Healthcare
  - Personalisations
  - Automatic Machine Translation
  - Automatic Game Playing

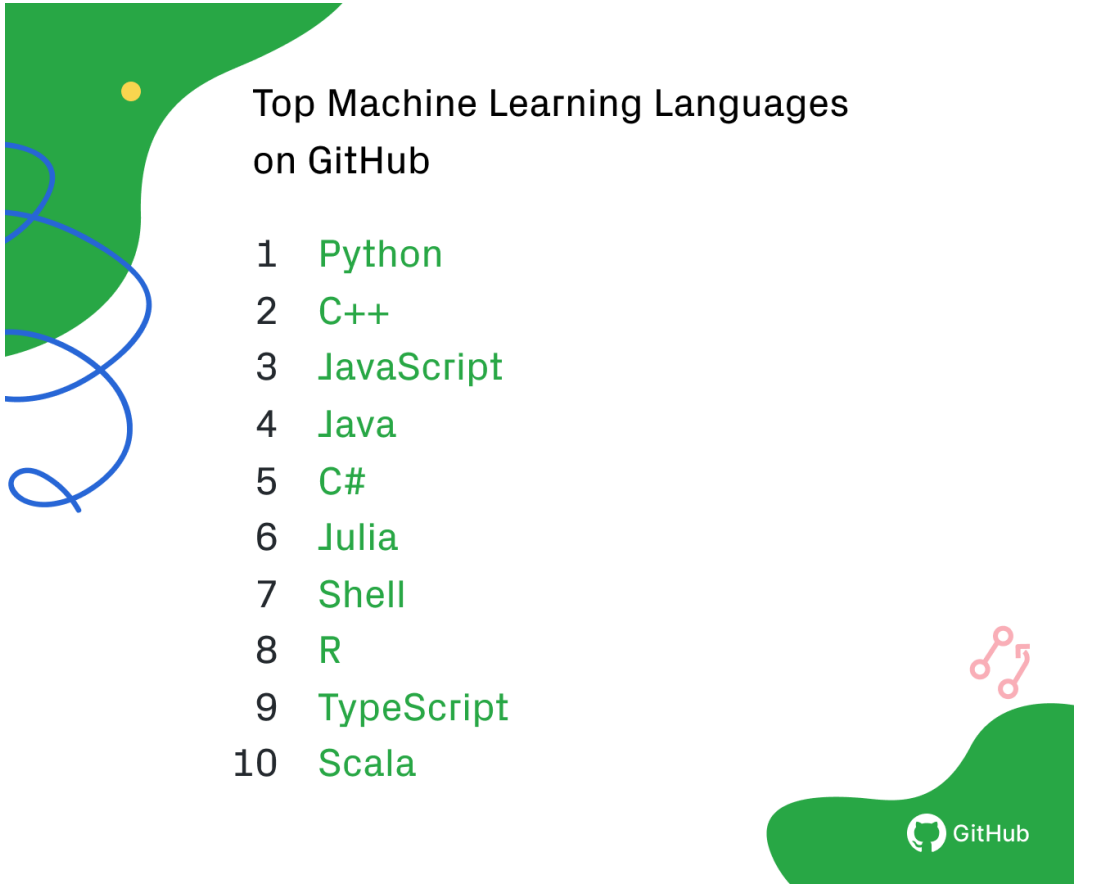
URL: Top 20 Applications of Deep Learning in 2022



Chapter 2: Machine Learning & Data Mining

1. Introduction – Programming Languages for Machine Learning

- Python
- C++
- JavaScript
- Java
- C#
- Julia
- Shell
- R
- TypeScript
- Scala



URL: Programming languages for Machine learning

Chapter 2: Machine Learning & Data Mining

1. Introduction – Advantages/Disadvantages of Machine Learning

- **Advantages**:
  - Fast, Accurate, Efficient
  - Automation of most applications
  - Wide range of real life applications
  - Enhanced cyber security and spam detection
  - No human Intervention is needed
  - Handling multi dimensional data

- **Disadvantages**:
  - It is very difficult to identify and rectify the errors
  - Data Acquisition
  - Interpretation of results requires more time and space

AI Fundamentals

Chapter 2: Machine Learning & Data Mining

1. Introduction – Advantages/Disadvantages of Machine Learning

TYPE	NAME	DESCRIPTION	ADVANTAGES	DISADVANTAGES
Linear	Linear regression	**The "best fit" line** through all data points. Predictions are numerical.	Easy to understand -- you clearly see what the biggest drivers of the model are.	X **Sometimes too simple to capture complex relationships between variables.** X **Does poorly with correlated features.**
	Logistic regression	The adaptation of **linear regression** to problems of classification (e.g., yes/no questions, groups, etc.)	Also easy to understand.	X **Sometimes too simple to capture complex relationships between variables.** X **Does poorly with correlated features.**

URL: Advantages/Disadvantages of Machine learning

AI Fundamentals

Chapter 2: Machine Learning & Data Mining

1. Introduction – Advantages/Disadvantages of Machine Learning

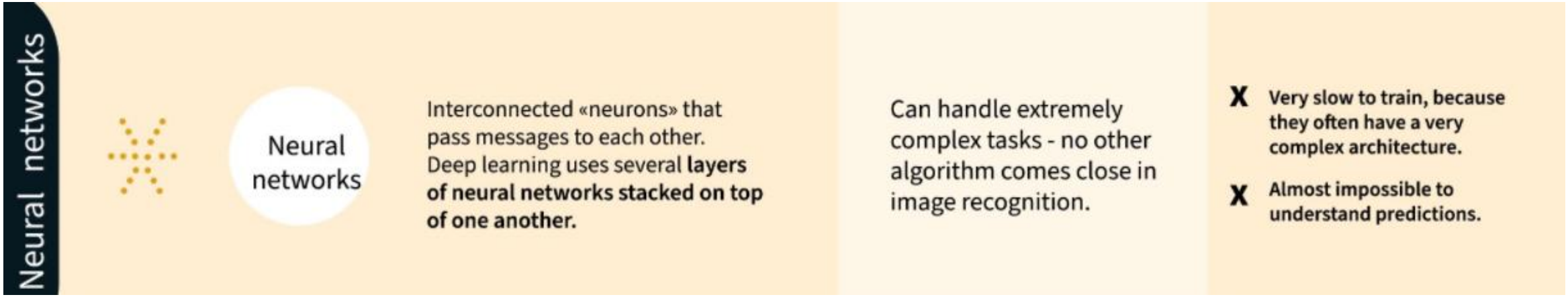
Tree-based	 Decision tree	**A series of yes/no rules based on the features,** forming a tree, to match all possible outcomes of a decision.	Easy to understand.	X Not often used on its own for prediction because it's also often too simple and not powerful enough for complex data.
	 Random Forest	Takes advantage of many decision trees, with rules created from subsamples of features. Each tree is weaker than a full decision tree, but **by combining them we get better overall performance.**	A sort of "wisdom of the crowd". Tends to result in very high quality models. Fast to train.	X Models can get very large. X Not easy to understand predictions.
	 Gradient Boosting	Uses even weaker decision trees, that are increasingly **focused on "hard" examples.**	High-performing.	X A small change in the feature set or training set can create radical changes in the model. X Not easy to understand predictions.

URL: [Advantages/Disadvantages of Machine learning]

AI Fundamentals

Chapter 2: Machine Learning & Data Mining

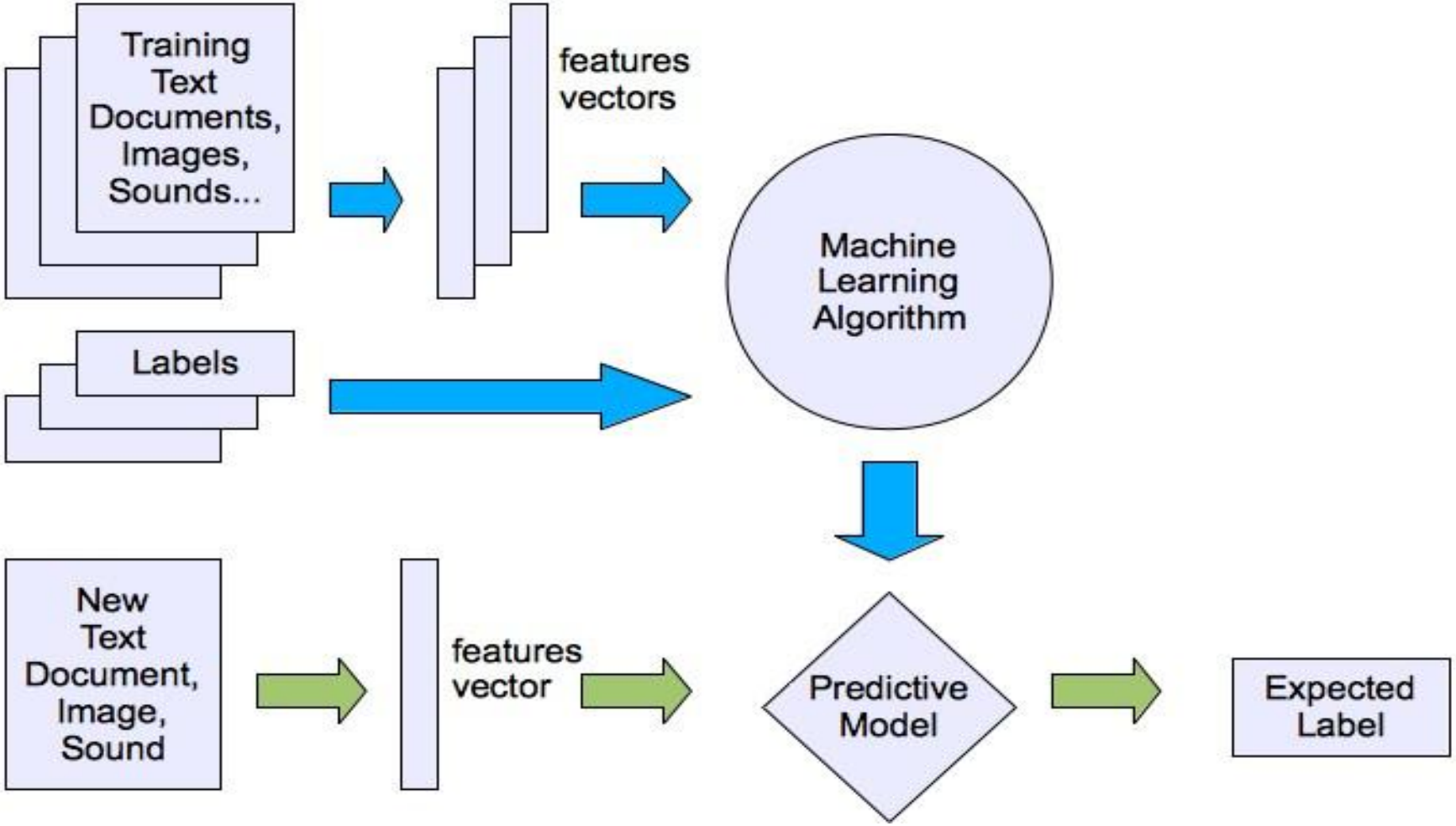
1. Introduction – Advantages/Disadvantages of Machine Learning



URL: Advantages/Disadvantages of Machine learning

Chapter 2: Machine Learning & Data Mining

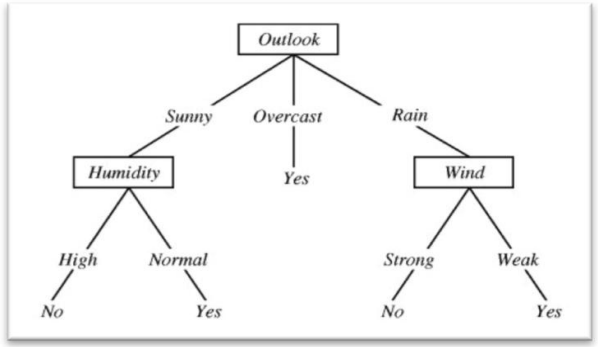
Machine learning: how to learn?

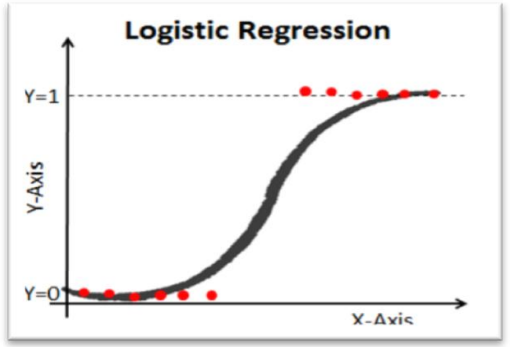


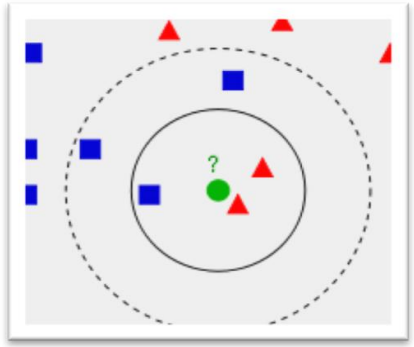
Chapter 2: Machine Learning & Data Mining

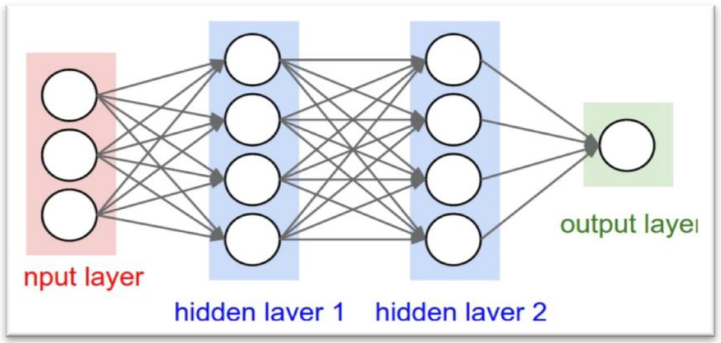
Machine learning: supervised learning

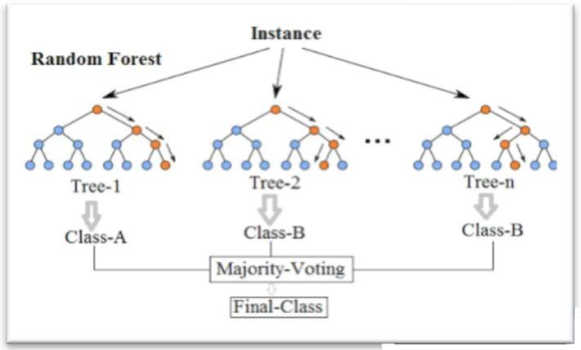
- There exist plenty of languages to express a model
- Each language has its bias and degree of interpretability
- No free lunch: There is no model (classifier) that works best for every problem





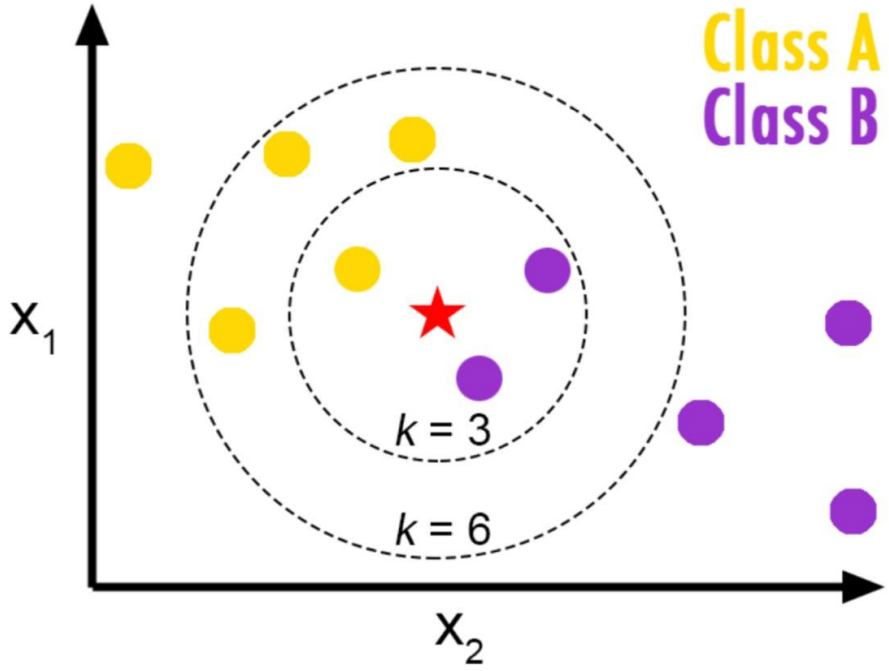






Machine learning models: lazy learning

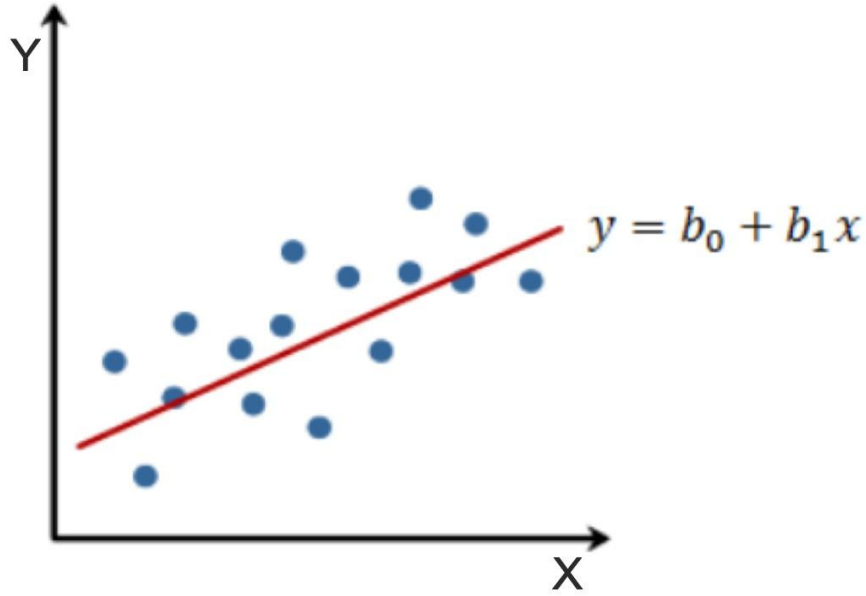
- k-Nearest Neighbors Method
- **Main task**: find suitable distance function



URL: Lazy learning

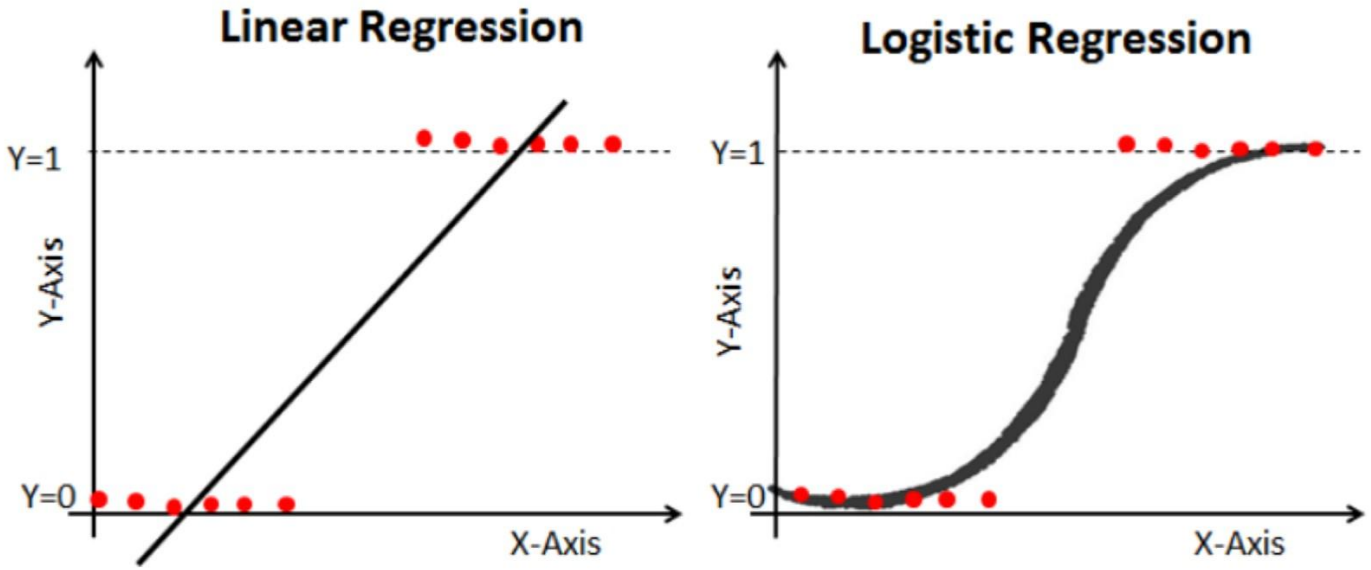
Machine learning models: regression

- **Regression for numeric targets**
  - **Main task**: estimate parameters b_0 and b_1, such that prediction and ground truth are as close as possible



Machine learning: regression (linear/logistic)

- **Regression** for binary targets



$$y = b_0 + b_1 x$$

$$p = \frac{1}{1 + e^{-(b_0 + b_1 x)}}$$

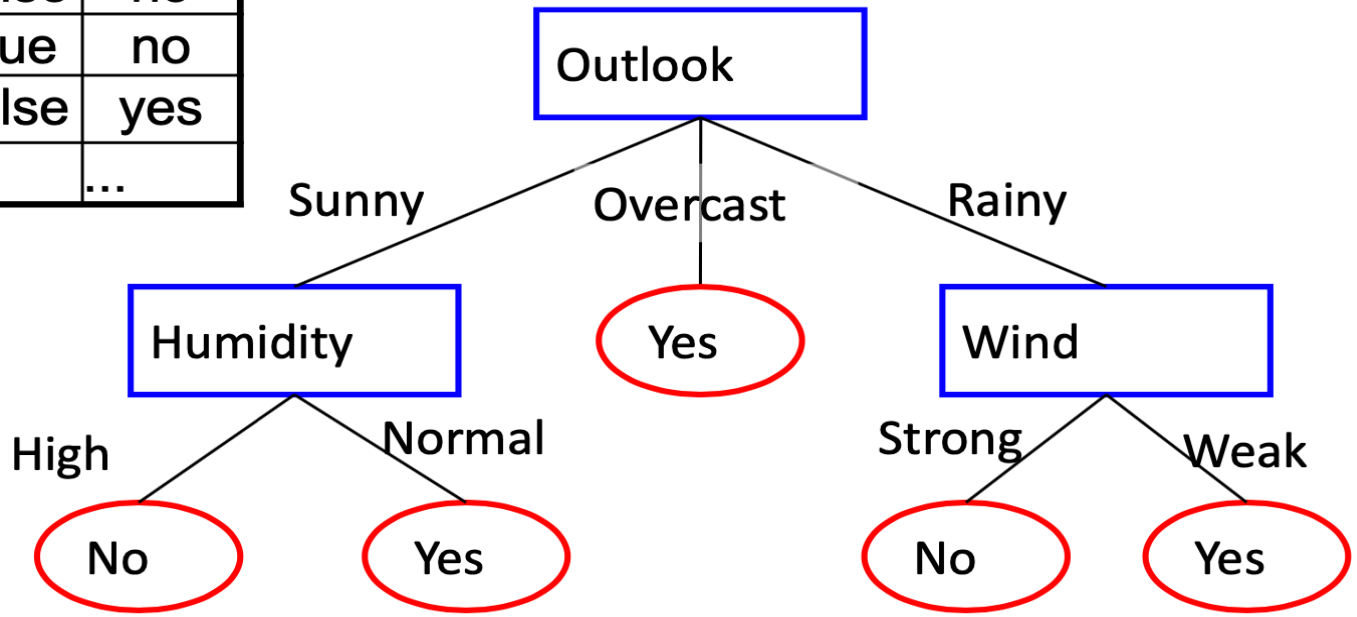
Machine learning models: decision trees

- **Decision tree** and **decision rule learners**:
  - Very popular data mining methods
  - High interpretability of models (e.g.medicine)
  - Efficient learning and prediction procedures

Machine learning models: decision trees

- Play tennis or not? (depending on weather conditions)

Outlook	Temp.	Hum.	Wind	Play?
Sunny	85	85	False	no
Sunny	80	90	True	no
Overcast	83	86	False	yes
...	...	...	...	...



- **Leaf nodes** versus **internal nodes**

AI Fundamentals

Chapter 2: Machine Learning & Data Mining

Contents

Chapter 2: Machine Learning & Data Mining

2. What is Learning?

- If we define AI as done in Elaine Rich's book (E. Rich. *Artificial Intelligence*, McGraw-Hill, 1983):

Artificial Intelligence is the study of how to make computers do things at which, at the moment, people are better.

- and we consider that the computer's learning ability is especially inferior to that of humans, then it follows that **research into learning mechanisms** and the **development of machine learning algorithms** is one of the most important branches of AI.

- The structure of the intelligent behavior can become so complex that it is very difficult or even impossible to program close to optimally, even with modern high-level languages such as **PROLOG** and **Python**.

- Machine learning algorithms are even used today to program robots in a way similar to how humans learn.

Chapter 2: Machine Learning & Data Mining

2. What is Learning?

- Learning vocabulary of a foreign language, or technical terms, or even memorizing a poem can be difficult for many people.
- For computers, however, these tasks are quite simple because they are little more than saving text in a file.
- *The memorization is uninteresting in AI.*
- In contrast, the acquisition of **mathematical skills** is usually not done by memorization.
- For addition of natural numbers this is not at all possible, because for each of the terms in the sum $x + y$ there infinitely many values. For each combination of the two values x and y, the triple $(x, y, x + y)$ would have to be stored, which is **impossible**.
- This poses the questions: *how do we learn mathematics?*
- The teacher explains the process and the students practice it on examples until they no longer make mistakes on new examples.
- This process is know as ***generalization***.

Chapter 2: Machine Learning & Data Mining

2. What is Learning? – Supervised learning

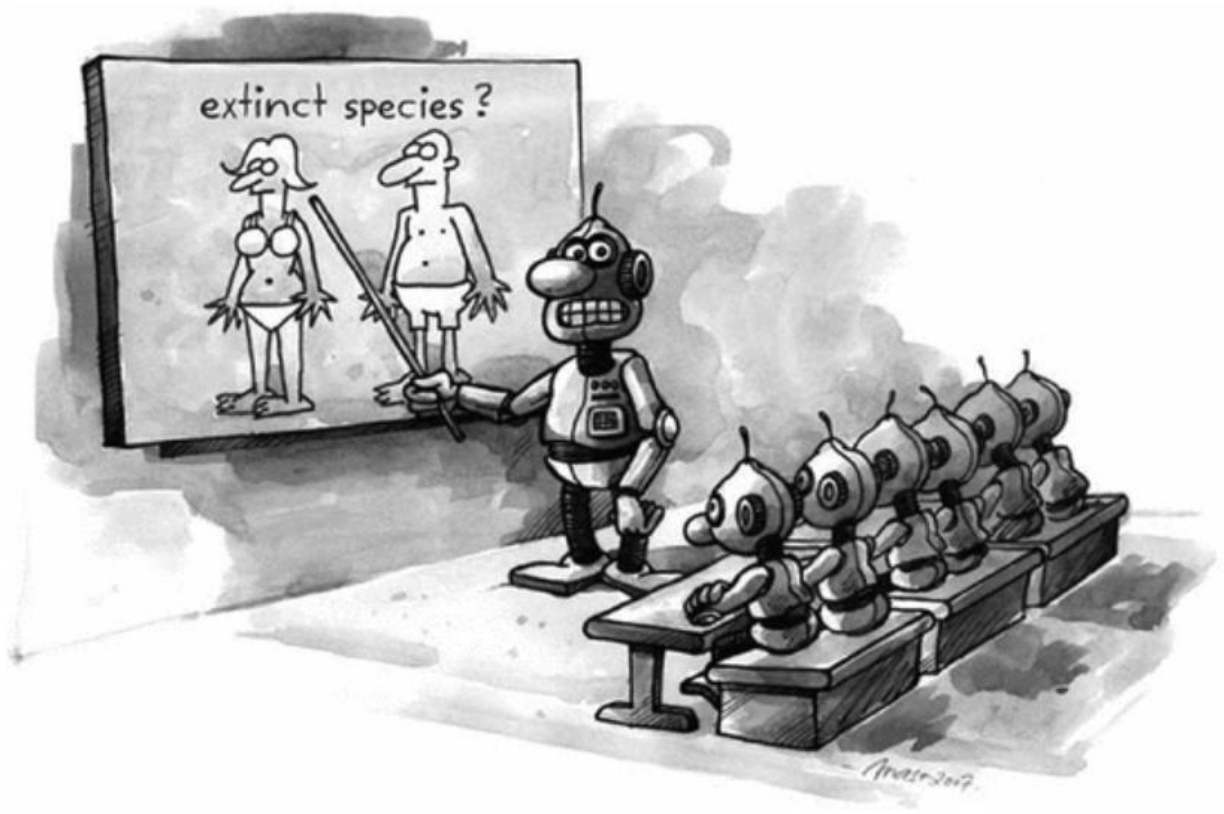


Figure 2.1: Supervised learning

Chapter 2: Machine Learning & Data Mining

2. What is Learning? – example 1

- A fruit farmer wants to automatically divide harvested apples into the merchandise **classes** A and B.
- The sorting device is equipped with sensors to measure two *features*, **size** and **color**, and then decide which of the two classes the apple belongs to.
- This is a typical **classification task**. Systems which are capable of dividing feature vectors into a finite number of classes are called **classifiers**.

- To configure the machine, apples are hand-picked by a specialist, that is, they are *classified*.
- Then the two measurements are entered together with their class label in a table (see **Table 2.1**).

Size [cm]	8	8	6	3	…
Color	0.1	0.3	0.9	0.8	…
Merchandise class	B	A	A	B	…

Table 2.1: Training data for the apple sorting agent

Chapter 2: Machine Learning & Data Mining

2. What is Learning? – example 1

- The **size** is given in the form of diameter in centimeters and the **color** by a numeric value between 0 (for **green**) and 1 (for **red**). A visualization of the data is listed as points in a scatterplot diagram in the right (see **Figure 2.2**).

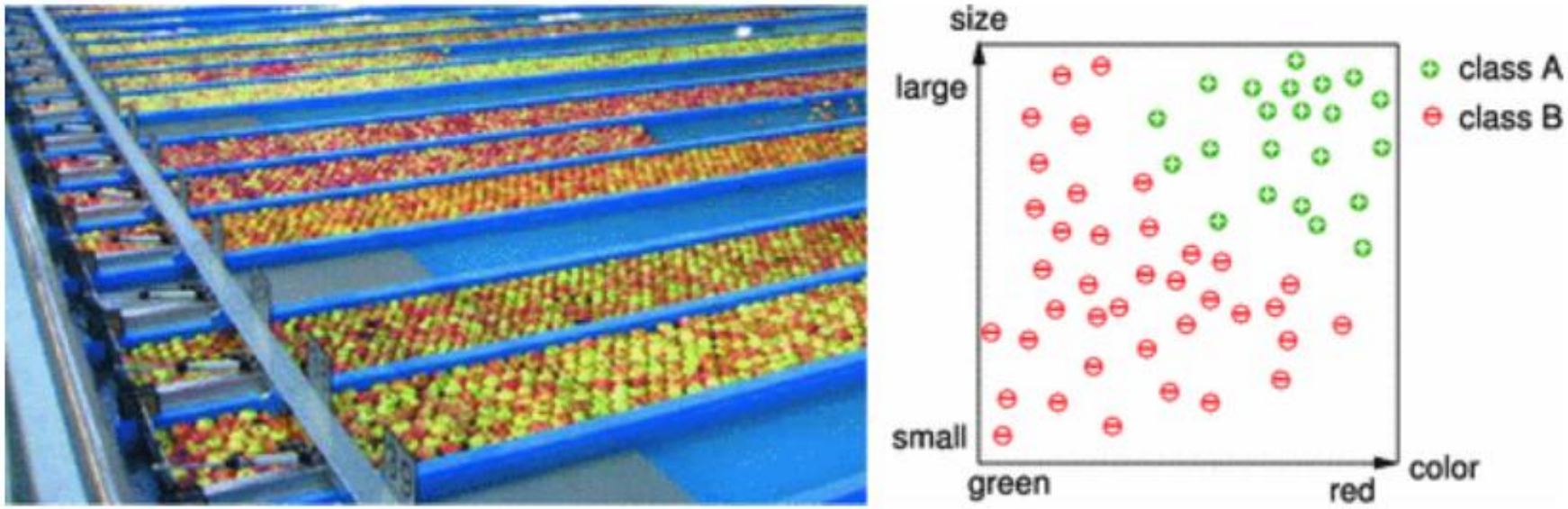


Figure 2.2: BayWa company apple sorting equipment in Kressbronn and some apples classified into merchandise classes A and B in feature space

Chapter 2: Machine Learning & Data Mining

2. What is Learning? – example 1

- The task in machine learning consists of generating a **function** from the collected, classified data which calculates the class value (A or B) for a new apple from the two features **size** and **color**.
- In **Figure 2.3** such a function is shown by the dividing line drawn through the diagram.
- All apples with a feature vector to the bottom left of the line are put into **class B**, and all others into **class A**.

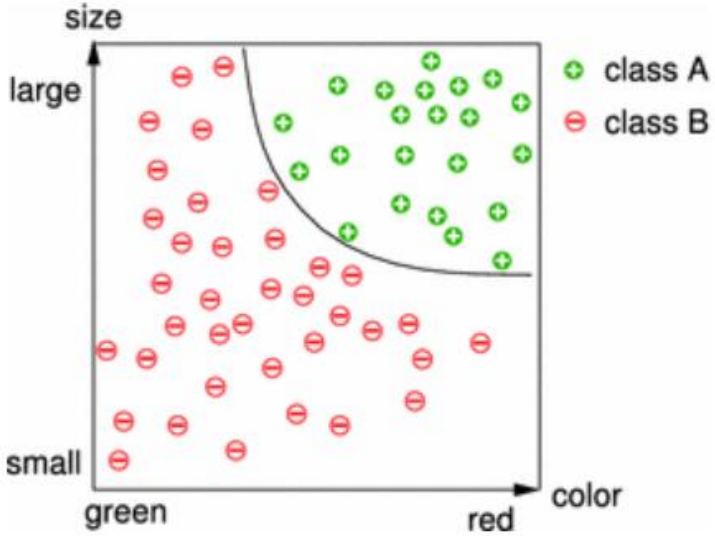


Figure 2.3: The curve drawn in into the diagram divides the classes and can then be applied to arbitrary new apples

Chapter 2: Machine Learning & Data Mining

2. What is Learning? – example 1

- In this example it is still very simple to find such a dividing line for the two classes.
- It is clearly more difficult, and above all much less visualizable task, when the objects to be classified are described by not just two, but **many features**. In practice 30 or more features are usually used.
- A “good” division means that the percentage of falsely classified objects is as small as possible.
- A **classifier** maps a **feature vector** to a **class value**.
- The desired mapping is also called **target function**. If the target function does not map onto a finite domain, then it is not a *classification*, but rather an *approximation* problem.

Chapter 2: Machine Learning & Data Mining

2. What is Learning? – The Learning Agent

- We can formally describe a **learning agent** as a function which maps a **feature vector** to a **discrete class value** or in general to a **real number**.
- This function is *not programmed*, rather it comes into existence or changes itself during the **learning phase**, influenced by the **training data**.
- In **Figure 2.4** such an agent is presented in the apple sorting example.
- During learning, the agent is fed with the already classified data from **Table 2.1**. Thereafter the agent constitutes as good a mapping as possible from the **feature vector** to the **function value** (e.g. merchandise class).

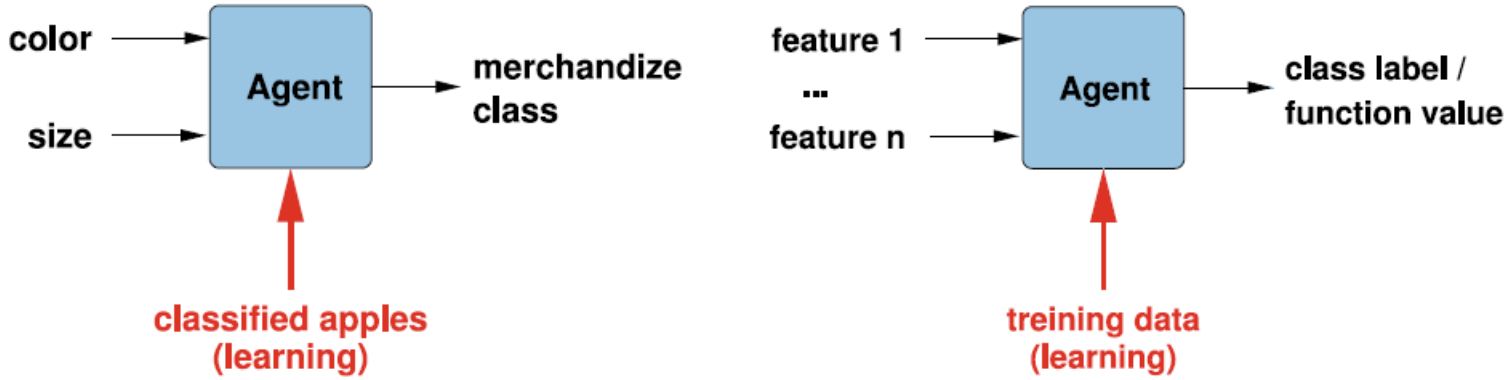


Figure 2.4: Functional structure of a learning agent for apple sorting (left) and in general (right)

Chapter 2: Machine Learning & Data Mining

2. What is Learning? – The Learning Agent

- We can now attempt to approach a definition of the term “machine learning”.

- **Tom Mitchell** (T. Mitchell. *Machine Learning*. McGraw Hill, 1997) gives this definition:

Machine Learning is the study of computer algorithms that improve automatically through experience.

Definition 2.1

*An agent is a **learning agent** if it improves its performance (measured by a suitable criterion) on new, unknown data over time (after it has seen many training examples).*

- It is important to test the **generalization** capability of the learning algorithm on unknown data, the **test data**. Otherwise every system that just saved the training data would appear to perform optimally just by calling up the saved data.

Chapter 2: Machine Learning & Data Mining

2. What is Learning? – The Learning Agent

- A **learning agent** is characterized by the following terms:
  - **Task**: the task of the learning algorithm is to learn a **mapping**. This could for example be the mapping from an apple's size and color to its merchandise class, but also the mapping from a patient's 15 symptoms to the decision of whether or not to remove the appendix.
  - **Variable agent** (more precisely a class of agents): here we have to decide which learning algorithm will be worked with. If this has been chosen, thus the class of all learnable functions is determined.
  - **Training data** (experience): the training data (sample) contain the knowledge which the learning algorithm is supposed to extract and learn. With the choice of training data one must ensure that it is a representative sample for the task to be learned.
  - **Test data**: important for evaluating whether the trained agent can generalize well from the training data to new data.
  - **Performance measure**: for the apple sorting device, the number of correctly classified apples. We need it to test the quality of an agent. Knowing the performance measure is usually much easier than knowing the agent's function.

AI Fundamentals

Chapter 2: Machine Learning & Data Mining

Contents

Chapter 2: Machine Learning & Data Mining

3. What is Data Mining?

Figure 2.5: Data Mining?

Chapter 2: Machine Learning & Data Mining

3. What is Data Mining?

- **Data Mining** is the task of a learning machine to extract **knowledge** from **training data**.
- Often the developer wants the learning machine to take the extracted knowledge readable for humans as well. It is still better if the developer can even alter the knowledge.
- The process of induction of **decision trees** is an example of this type of method.
- Similar challenges come from **electronic business** and knowledge management.
- A classic problem presents itself here: from the actions of the visitors to this web portal, the owner of an Internet business would like to create a relationship between the characteristics of a customer and the class of products which are interesting to that customer. Then a seller will be able to place **customer-specific advertisements**.
- In many areas of **advertisement** and **marketing**, as well in **customer relationship management** (CRM), **data mining techniques** are coming into use.
- Whenever large amounts of data are available, one can attempt to use these data for the analysis of *customer preferences* in order to show *customer-specific advertisements*.

Chapter 2: Machine Learning & Data Mining

3. What is Data Mining?

*The process of acquiring **knowledge** from **data**, as well as its **representation** and **application**, is called **data mining**. The methods used are usually taken from **statistics** or **machine learning** and should be applicable to very large amounts of data at reasonable cost.*

- In the context of acquiring information, for example on the Internet or in an intranet, **text mining** plays an increasingly important role.
- Typical tasks include finding similar text in a search engine or the **classification of texts**, which for example is applied in **spam filters** for email.
- A relatively new challenge for data mining is the **extraction** of **structural**, **static**, and **dynamic information** from **graph structures** such as *social networks*, *traffic networks*, or *Internet traffic*.
- Because the two described tasks of **machine learning** and **data mining** are *formally very similar*, the basic methods used in both areas are for the most part identical.
- Because of the huge commercial impact of data mining techniques, there are now many sophisticated optimizations and a whole line of powerful data mining systems, which offer a large palette of convenient tools for the extraction of knowledge from data.

AI Fundamentals

Chapter 2: Machine Learning & Data Mining

Contents

Chapter 2: Machine Learning & Data Mining

4. Data Analysis

- **Statistics** provide a number of ways to describe data with simple **parameters**.
- From these we choose a few which are especially important for the analysis of training data and test these on a subset of **LEXMED** data.
- The **medical expert system** **LEXMED** was developed at the Ravensburg-Weingarten University of Applied Sciences by *Manfred Schramm*, *Walter Rampf* and *Wolfgang Ertel*, in cooperation with the Weingarten 14-Nothelfer Hospital.
- The acronym **LEXMED** stands for ***l**earning **ex**pert system for **me**dical **d**iagnosis*.
- In this example dataset, the symptoms x_1, …, x_{15} of N=473 patients are described in **Table 2.2**, as well as the class label, that is the diagnosis (**appendicitis** positive/negative) are listed.

Var. num.	Description	Values
1	Age	Continuous
2	Sex (1 = male, 2 = female)	1, 2
3	Pain quadrant 1	0, 1
4	Pain quadrant 2	0, 1
5	Pain quadrant 3	0, 1
6	Pain quadrant 4	0, 1
7	Local muscular guarding	0, 1
8	Generalized muscular guarding	0, 1
9	Rebound tenderness	0, 1
10	Pain on tapping	0, 1
11	Pain during rectal examination	0, 1
12	Axial temperature	Continuous
13	Rectal temperature	Continuous
14	Leukocytes	Continuous
15	Diabetes mellitus	0, 1
16	Appendicitis	0, 1

Table 2.2: Description of variables x_1, …., x_{16}

Chapter 2: Machine Learning & Data Mining

4. Data Analysis

- In this example dataset, the symptoms x_1, …, x_{16} of N=473 patients are described in **Table 2.2**, as well as the class label, that is the diagnosis (**appendicitis** positive/negative) are listed.

- **Patient** number **one**, for example, is described by the vector:
  - $x^1 = (26,1,0,0,1,0,1,0,1,1,0,37.9,38.8,23100,0,1)$
- and **patient** number **two** by
  - $x^2 = (17,2,0,0,1,0,1,0,1,1,0,36.9,37.4,8100,0,0)$
- Patient two has the leukocyte value $x_{14}^2 = 8100$

Var. num.	Description	Values
1	Age	Continuous
2	Sex (1 = male, 2 = female)	1, 2
3	Pain quadrant 1	0, 1
4	Pain quadrant 2	0, 1
5	Pain quadrant 3	0, 1
6	Pain quadrant 4	0, 1
7	Local muscular guarding	0, 1
8	Generalized muscular guarding	0, 1
9	Rebound tenderness	0, 1
10	Pain on tapping	0, 1
11	Pain during rectal examination	0, 1
12	Axial temperature	Continuous
13	Rectal temperature	Continuous
14	Leukocytes	Continuous
15	Diabetes mellitus	0, 1
16	Appendicitis	0, 1

Table 2.2: Description of variables x_1, …., x_{16}

Chapter 2: Machine Learning & Data Mining

4. Data Analysis

- For each variable x_i, its **average** $\overline{x_i}$ is defined as

$$\overline{x_i} := \frac{1}{N}\sum_{p=1}^{N} x_i^p$$

- and the **standard deviation** s_i as a measure of its *average deviation* from the **average value** as

$$s_i := \sqrt{\frac{1}{N-1}\sum_{p=1}^{N}\left(x_i^p - \overline{x_i}\right)^2}$$

- The question of whether two variables x_i and x_j are **statistically dependent** (*correlated*) is important for the analysis of multidimensional data.
- For example, the **covariance** $\sigma_{ij} = \frac{1}{N-1}\sum_{p=1}^{N}\left(x_i^p - \overline{x_i}\right)\left(x_j^p - \overline{x_j}\right)$ gives information about this.
- In this sum, the summand returns a positive entry for the *p-th data vector* exactly when the deviations of the i-th and j-th components from the average both have the **same sign**.
- If they have **different signs**, then the entry is negative. Therefore the **covariance** $\sigma_{12,13}$ of the two different *fever values* shoud be quite clearly **positive**.
- **URL**: Covariantie

4. Data Analysis

- However, the **covariance** also depends on the absolute value of the variables, which makes comparison of the values difficult.

- To be able to compare the degree of dependence in the case of multiple variables, we therefore define the **correlation coefficient**.

$$K_{ij} = \frac{\sigma_{ij}}{s_i . s_j}$$

- for the two values x_i and x_j, which is nothing but a **normalized covariance**.

- The **matrix K** of all **correlation coefficients** contains values between -1 and +1, is **symmetric**, and all of its *diagonal elements* have the value 1.

- **URL**: correlatie coëfficiënt

Chapter 2: Machine Learning & Data Mining

4. Data Analysis

- The **matrix K** of all **correlation coefficients** contains values between -1 and +1, is **symmetric**, and all of its *diagonal elements* have the value 1.
- The correlation matrix for all the 16 variables is given in **Table 2.3**.

1.	−0.009	0.14	0.037	−0.096	0.12	0.018	0.051	−0.034	−0.041	0.034	0.037	0.05	−0.037	0.37	0.012
−0.009	1.	−0.0074	−0.019	−0.06	0.063	−0.17	0.0084	−0.17	−0.14	−0.13	−0.017	−0.034	−0.14	0.045	−0.2
0.14	−0.0074	1.	0.55	−0.091	0.24	0.13	0.24	0.045	0.18	0.028	0.02	0.045	0.03	0.11	0.045
0.037	−0.019	0.55	1.	−0.24	0.33	0.051	0.25	0.074	0.19	0.087	0.11	0.12	0.11	0.14	−0.0091
−0.096	−0.06	−0.091	−0.24	1.	0.059	0.14	0.034	0.14	0.049	0.057	0.064	0.058	0.11	0.017	0.14
0.12	0.063	0.24	0.33	0.059	1.	0.071	0.19	0.086	0.15	0.048	0.11	0.12	0.063	0.21	0.053
0.018	−0.17	0.13	0.051	0.14	0.071	1.	0.16	0.4	0.28	0.2	0.24	0.36	0.29	−0.0001	0.33
0.051	0.0084	0.24	0.25	0.034	0.19	0.16	1.	0.17	0.23	0.24	0.19	0.24	0.27	0.083	0.084
−0.034	−0.17	0.045	0.074	0.14	0.086	0.4	0.17	1.	0.53	0.25	0.19	0.27	0.27	0.026	0.38
−0.041	−0.14	0.18	0.19	0.049	0.15	0.28	0.23	0.53	1.	0.24	0.15	0.19	0.23	0.02	0.32
0.034	−0.13	0.028	0.087	0.057	0.048	0.2	0.24	0.25	0.24	1.	0.17	0.17	0.22	0.098	0.17
0.037	−0.017	0.02	0.11	0.064	0.11	0.24	0.19	0.19	0.15	0.17	1.	0.72	0.26	0.035	0.15
0.05	−0.034	0.045	0.12	0.058	0.12	0.36	0.24	0.27	0.19	0.17	0.72	1.	0.38	0.044	0.21
−0.037	−0.14	0.03	0.11	0.11	0.063	0.29	0.27	0.27	0.23	0.22	0.26	0.38	1.	0.051	0.44
0.37	0.045	0.11	0.14	0.017	0.21	−0.0001	0.083	0.026	0.02	0.098	0.035	0.044	0.051	1.	−0.0055
0.012	−0.2	0.045	−0.0091	0.14	0.053	0.33	0.084	0.38	0.32	0.17	0.15	0.21	0.44	−0.0055	1.

***Table 2.3**: Correlation matrix for the 16 appendicitis variables measured in 473 cases*

Chapter 2: Machine Learning & Data Mining

4. Data Analysis

- This matrix becomes somewhat more readable when we represent it as a **density plot**.
- Instead of the numerical values, the matrix elements in **Figure 2.6** are filled with *gray values*.
- In the **right** diagram, the **absolute values** are shown. Thus we can very quickly see which variables display a *weak* or *strong dependence*.

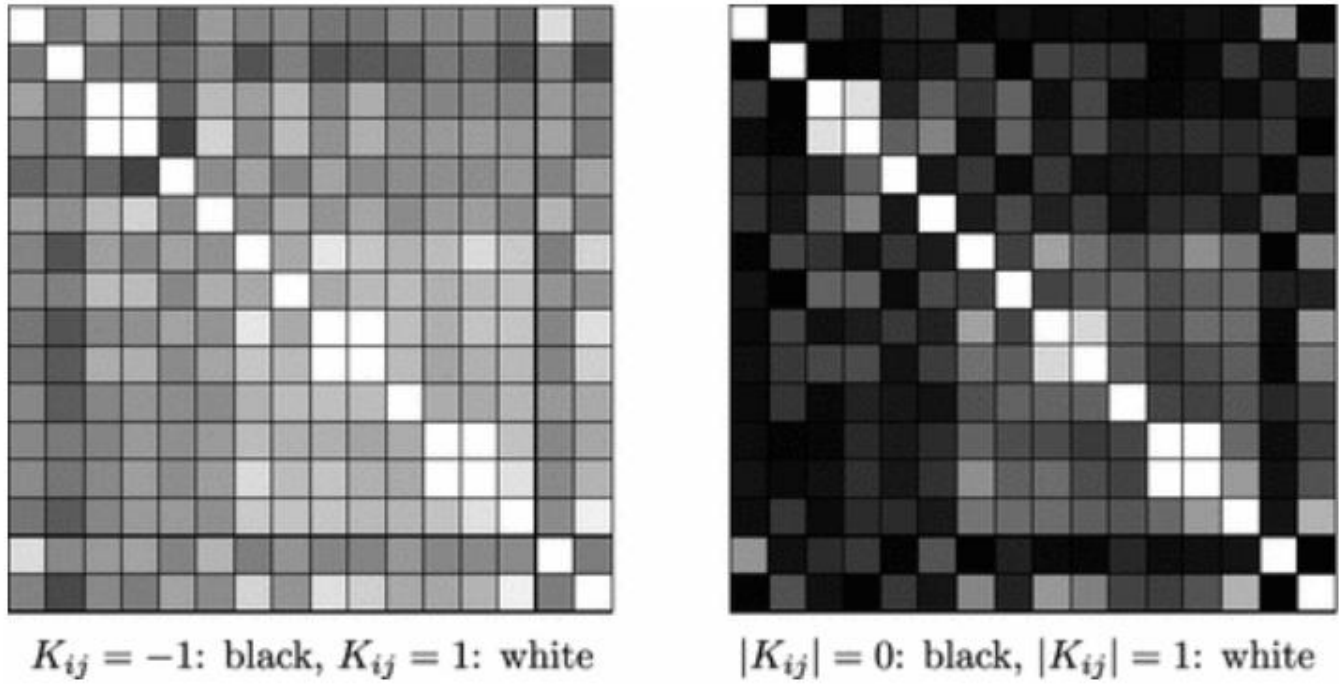


Figure 2.6: The correlation matrix as a frequency graph

Chapter 2: Machine Learning & Data Mining

4. Data Analysis

- In the **left** diagram, **dark** stands for *negative* and **light** for *positive*.
- In the **right** image, the absolute values were listed.
- Here **black** means $|K_{ij}| \approx 0$ $(uncorrelated)$ and **white** $|K_{ij}| \approx 1$ $(strongly\ correlated)$.

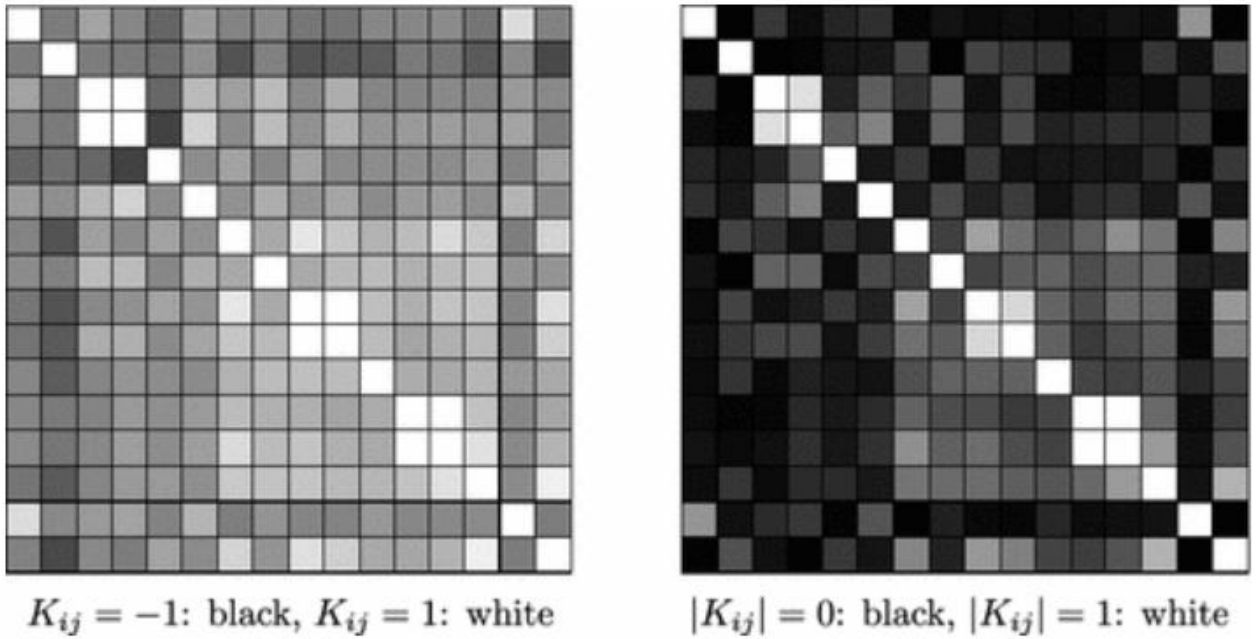


Figure 2.6: The correlation matrix as a frequency graph

4. Data Analysis

- We can see, for example, that the variables 7, 9, 10 and 14 show the strongest correlation with the class variable *appendicitis* (variable 16) and therefore are more important for the diagnosis than the other variables.
- We also see, however, that the variables 9 and 10 (rebound tenderness, pain on tapping) are strongly correlated. This could mean that one of these two values is potentially sufficient for the diagnosis.

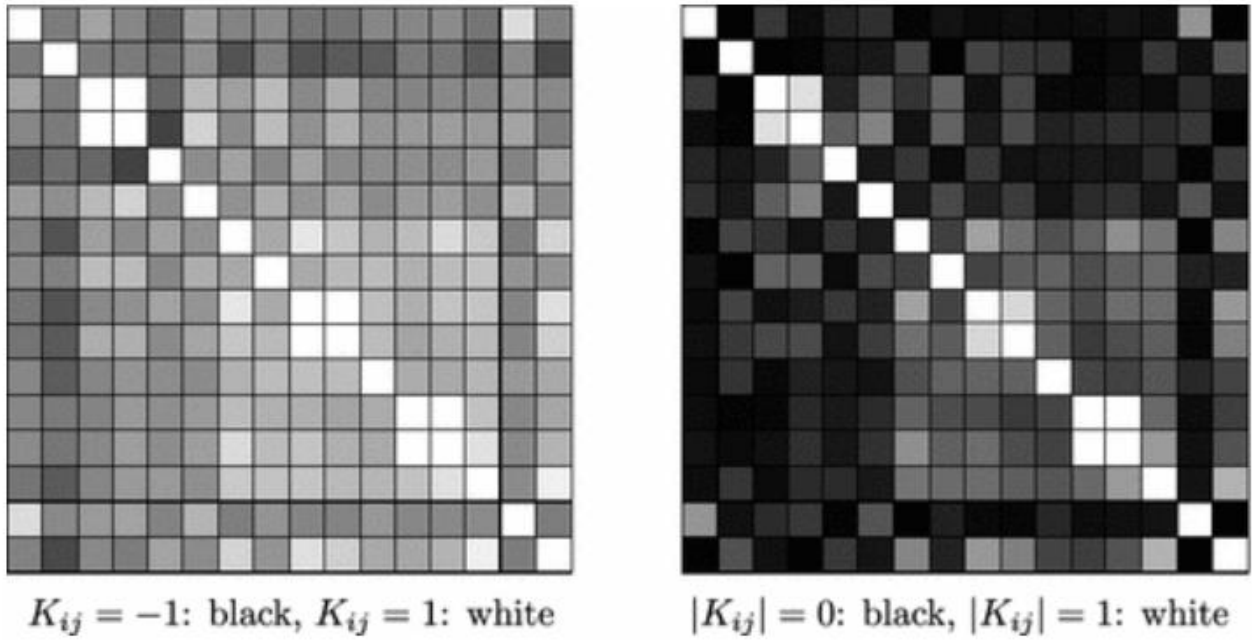


Figure 2.6: The correlation matrix as a frequency graph

AI Fundamentals

Chapter 2: Machine Learning & Data Mining

Contents

Chapter 2: Machine Learning & Data Mining

5. The Perceptron, a Linear Classifier

- In the *apple sorting classification example*, a curved dividing line is drawn between the two classes in **Figure 2.3**.
- A simpler case is shown in **Figure 2.7**. Here the two-dimensional training examples can be separated by a straight line. We call such a set of training data **linearly separable**.

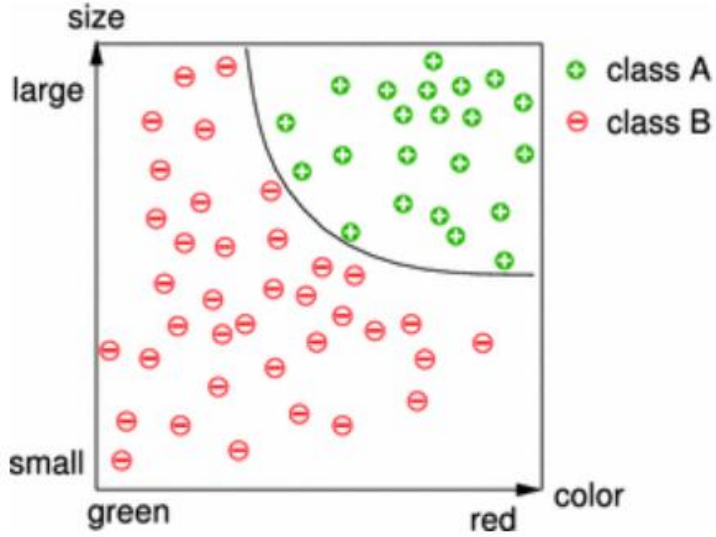


Figure 2.3: The curve drawn in into the diagram divides the classes and can then be applied to arbitrary new apples

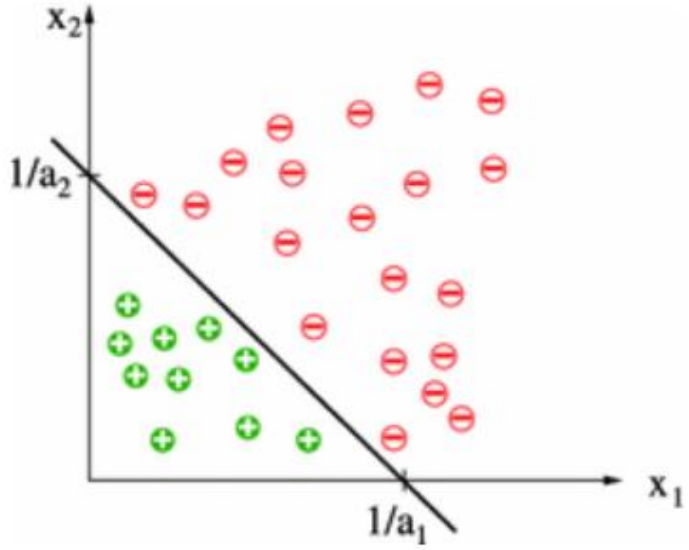


Figure 2.7: A linearly separable two dimensional data sets

Chapter 2: Machine Learning & Data Mining

5. The Perceptron, a Linear Classifier

- The equation for the dividing straight line is $\boldsymbol{a_1 x_1 + a_2 x_2 = 1}$
- Because every (n-1)-dimensional hyperplane in $\mathbb{R}^n$ can be described by an equation $\sum_{i=1}^{n} a_i x_i = \theta$ it makes sense to define **linear separability** as follows.
- **URL**: Hyperplane

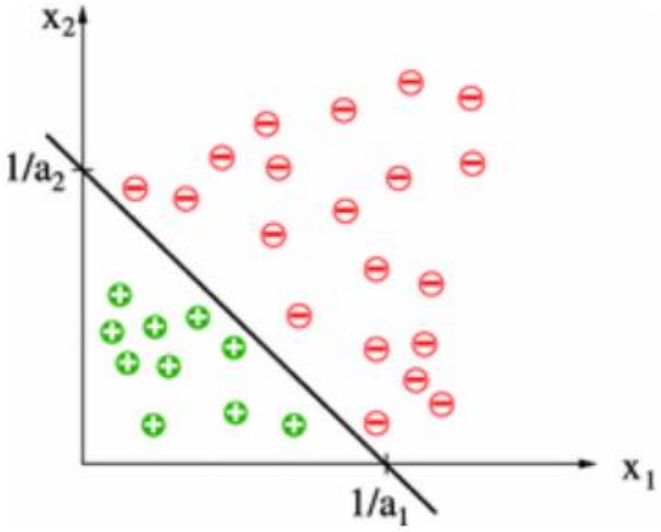


Figure 2.7: A linearly separable two dimensional data sets

Definition 2.2

*Two sets $M_1 \subset \mathbb{R}^n$ and $M_2 \subset \mathbb{R}^n$ are called **linear separable** if real numbers $a_1, \ldots, a_n, \theta$ exist with $\sum_{i=1}^{n} a_i x_i > \theta$ for all $x \in M_1$ and $\sum_{i=1}^{N} a_i x_i \leq \theta$ for all $x \in M_2$.*

*The value θ is denoted the **threshold**.*

Chapter 2: Machine Learning & Data Mining

5. The Perceptron, a Linear Classifier

- In **Figure 2.8** we see that the **AND** function is linearly separable, but the **XOR** function is not. The **green** points in the plane are **true**, the **red dots** are **false**.
- For **AND**, for example, the line $-x_1 + \frac{3}{2}$ separates **true** and **false** interpretations of the formula $x_1 \wedge x_2$. The equation of a line with two points (x_1, y_1) and (x_2, y_2) is

$$y - y_1 = \frac{y_2 - y_1}{x_2 - x_1}(x - x_1)$$ **URL**: rechte door 2 punten

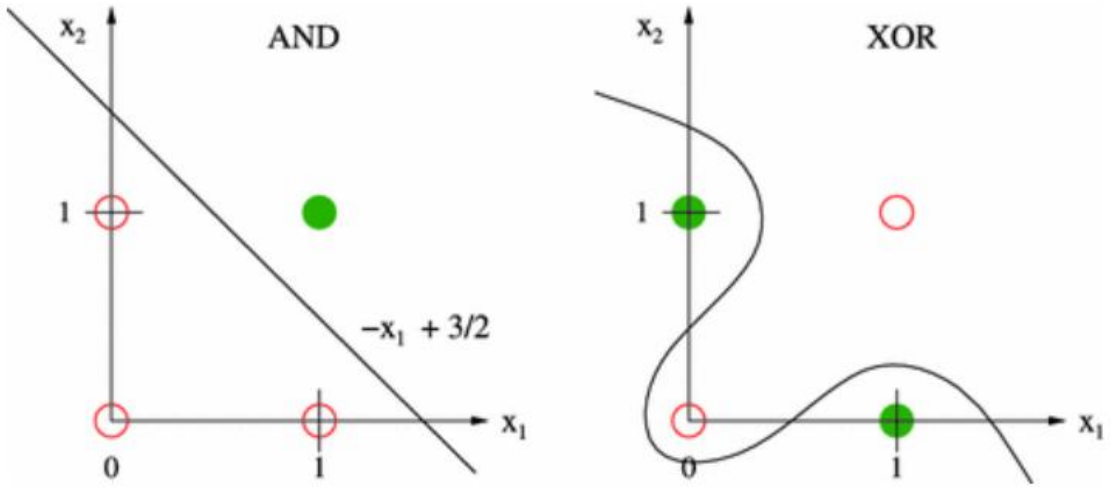


Figure 2.8: The boolean function is linearly separable, but XOR is not

Chapter 2: Machine Learning & Data Mining

5. The Perceptron, a Linear Classifier

- In contrast, the **XOR** function does *not* have a straight line of separation.
- Clearly the **XOR** function has *a more complex structure* than the AND function in this regard.

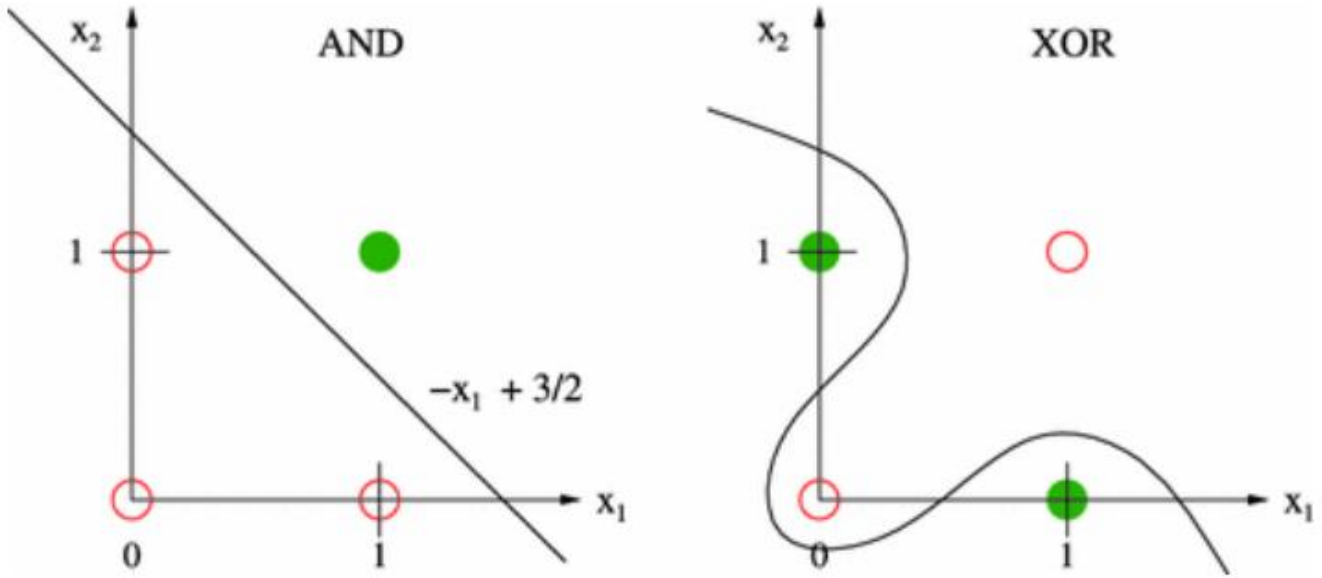


Figure 2.8: The boolean function is linearly separable, but XOR is not

Chapter 2: Machine Learning & Data Mining

5. The Perceptron, a Linear Classifier

- With the **perceptron**, we present a **very simple learning algorithm** which can separate *linearly separable* sets.

Definition 2.3

Let $w = (w_1, \ldots, w_n) \in \mathbb{R}^n$ *be a* ***weight vector*** *and* $x \in \mathbb{R}^n$ *an* ***input vector****.*

A ***perceptron*** *represents a function P:* $\mathbb{R}^n \to \{0,1\}$ *which corresponds to the following rule:*

$$P(x) = \begin{cases} 1 & if\ w.x = \sum_{i=1}^{n} w_i x_i > 0, \\ 0 & else. \end{cases}$$

Chapter 2: Machine Learning & Data Mining

5. The Perceptron, a Linear Classifier

- The perceptron is a **very simple classification algorithm**.
- It is equivalent to a *two-layer neural network* with **activation** by a *threshold function*, shown in **Figure 2.9**.
- For now, however, we will only view the perceptron as a **learning agent**, that is, as a mathematical function which maps a *feature vector* to a *function value*.
- Here the *input variables* x_i are denoted **features**.

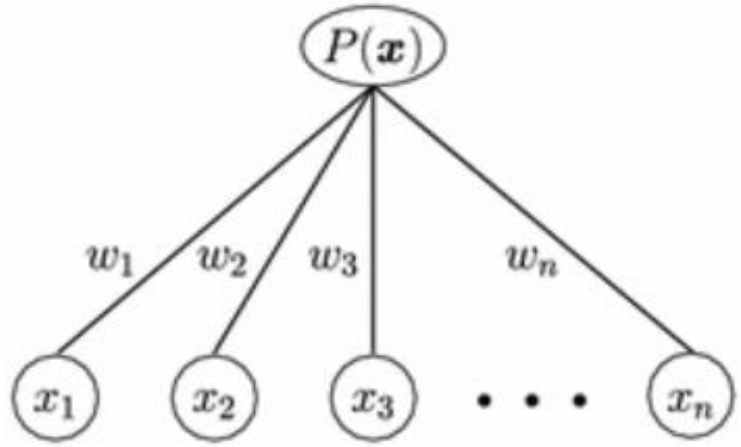


Figure 2.9: Graphical representation of a perceptron as a two-layer neural network

- We can see in the formula $\sum_{i=1}^{n} w_i x_i > 0$, all points x above the hyperplane $\sum_{i=1}^{n} w_i x_i = 0$ are classified as **positive** $(P(x) = 1)$, and all others as **negative** $(P(x) = 0)$.
- The separating hyperplane goes through the origin because $\theta = 0$.

5. The Perceptron, a Linear Classifier – The Learning Rule

- First, however, we want to introduce a **simple learning algorithm** for the **perceptron**.
- With the notation M_+ and M_- for the sets of **positive** and **negative** training patterns respectively, the **perceptron learning rule** is:

PERCEPTRONLEARNING$[M_+, M_-]$
$\boldsymbol{w}$ = arbitrary vector of real numbers
Repeat
  For all $\boldsymbol{x} \in M_+$
    If $\boldsymbol{w}\,\boldsymbol{x} \leq 0$ **Then** $\boldsymbol{w} = \boldsymbol{w} + \boldsymbol{x}$
  For all $\boldsymbol{x} \in M_-$
    If $\boldsymbol{w}\,\boldsymbol{x} > 0$ **Then** $\boldsymbol{w} = \boldsymbol{w} - \boldsymbol{x}$
Until all $\boldsymbol{x} \in M_+ \cup M_-$ are correctly classified

- The **perceptron** should output the value **1** for all $x \in M_+$. By definition this is true when $w.x > 0$.
- If this is *not* the case than x is added to the weight vector w, whereby the weight vector is changed in exactly the right direction.

Chapter 2: Machine Learning & Data Mining

5. The Perceptron, a Linear Classifier – The Learning Rule

PERCEPTRONLEARNING[M_+, M_-]
$\boldsymbol{w}$ = arbitrary vector of real numbers
Repeat
  For all $\boldsymbol{x} \in M_+$
    If $\boldsymbol{w}\,\boldsymbol{x} \leq 0$ **Then** $\boldsymbol{w} = \boldsymbol{w} + \boldsymbol{x}$
  For all $\boldsymbol{x} \in M_-$
    If $\boldsymbol{w}\,\boldsymbol{x} > 0$ **Then** $\boldsymbol{w} = \boldsymbol{w} - \boldsymbol{x}$
Until all $\boldsymbol{x} \in M_+ \cup M_-$ are correctly classified

- If this is *not* the case than x is added to the weight vector w, whereby the weight vector is changed in exactly the right direction.
- We see this when we apply the **perceptron** to the changed vector $\boldsymbol{w} + \boldsymbol{x}$ because $(\boldsymbol{w} + \boldsymbol{x}).\boldsymbol{x} = \boldsymbol{w}.\boldsymbol{x} + \boldsymbol{x}^2$
- If this step is repeated often enough, then at some point the value $w.x$ will become **positive**, as it should be.

Chapter 2: Machine Learning & Data Mining

5. The Perceptron, a Linear Classifier – The Learning Rule

PERCEPTRONLEARNING[M_+, M_-]
$\boldsymbol{w}$ = arbitrary vector of real numbers
Repeat
  For all $\boldsymbol{x} \in M_+$
    If $\boldsymbol{w}\,\boldsymbol{x} \leq 0$ **Then** $\boldsymbol{w} = \boldsymbol{w} + \boldsymbol{x}$
  For all $\boldsymbol{x} \in M_-$
    If $\boldsymbol{w}\,\boldsymbol{x} > 0$ **Then** $\boldsymbol{w} = \boldsymbol{w} - \boldsymbol{x}$
Until all $\boldsymbol{x} \in M_+ \cup M_-$ are correctly classified

- Analogously, we see that, for *negative training data*, the perceptron calculates an even smaller value $(\boldsymbol{w} - \boldsymbol{x}).\boldsymbol{x} = \boldsymbol{w}.\boldsymbol{x} - \boldsymbol{x}^2$ which at some point becomes **negative**.

Chapter 2: Machine Learning & Data Mining

5. The Perceptron, a Linear Classifier – The Learning Rule

- **Example 2**
- A perceptron is to be trained on the sets $M_+ = \{(0,1.8), (2,0.6)\}$ and $M_- = \{(-1.2,1.4), (0.4,-1)\}$.
- $w = (1,1)$ was used as an *initial weight vector*.

- The training data and the line defined by the **weight vector** $w.x = 1.x_1 + 1.x_2 = 0$ are shown in **Figure 2.10** in the first picture in the top row.

- The **solid line** shows the *current dividing line* $w.x = 0$.
- The **orthogonal dashed line** is the *weight vector* w and the **second dashed line** the *change vector* $\Delta w = x$ *or* $\Delta w = -x$ to be added, which is calculated from the currently active data point surrounded in light green.

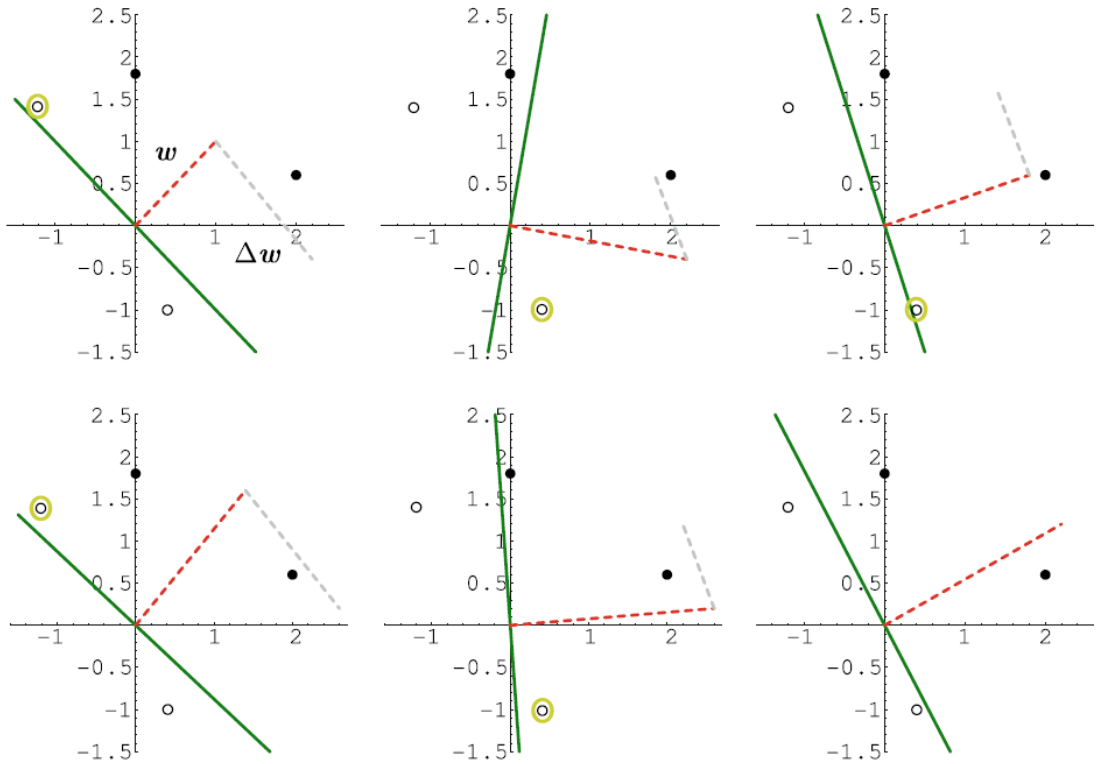


Figure 2.10: application of the perceptron learning rule to two positive (●) and two negative (○) data points.

Chapter 2: Machine Learning & Data Mining

5. The Perceptron, a Linear Classifier – The Learning Rule

- **Example 2**
- A perceptron is to be trained on the sets $M_+ = \{(0,1.8), (2,0.6)\}$ and $M_- = \{(-1.2,1.4), (0.4, -1)\}$.
- $w = (1,1)$ was used as an *initial weight vector*.
- The training data and the line defined by the **weight vector** $w.x = x_1 + x_2 = 0$ are shown in **Figure 2.10** in the first picture in the top row.
- In addition, the **weight vector** is drawn as a **dashed line**.
- Because $w.x = 0$, this is **orthogonal** to the line.

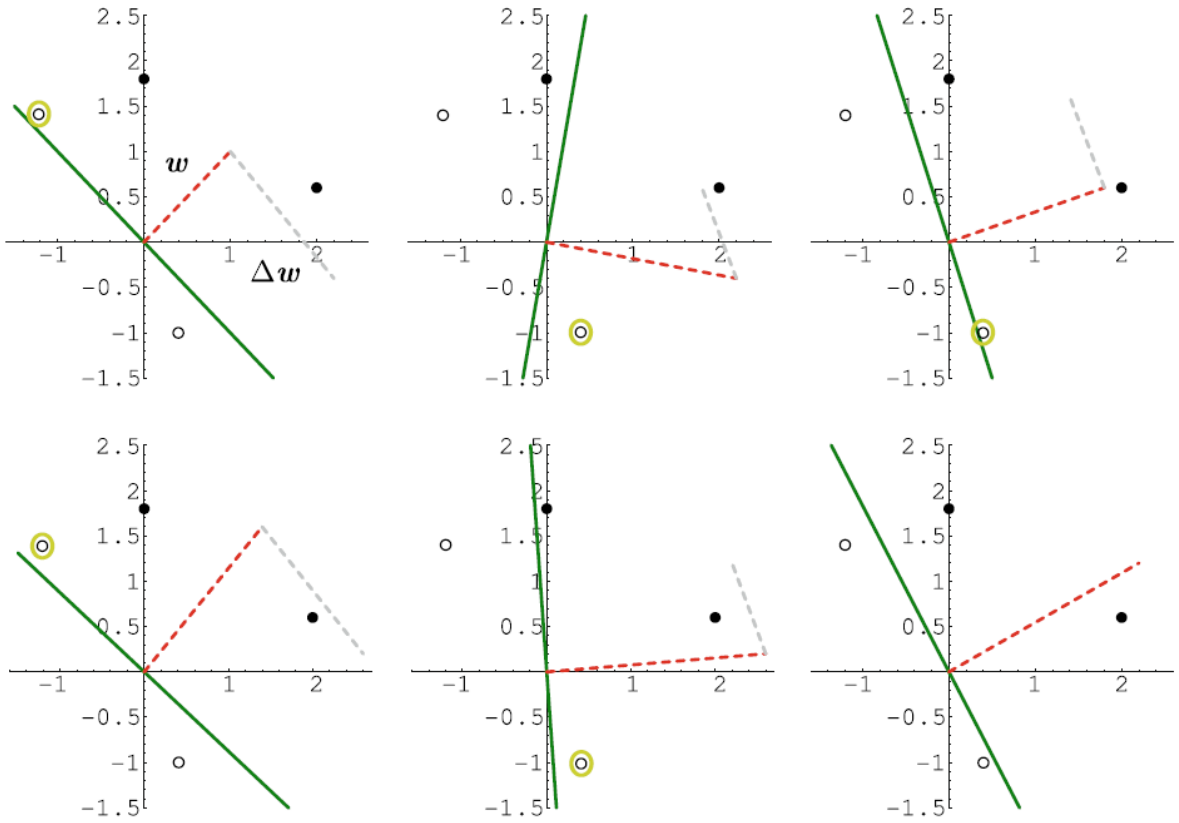


Figure 2.10: application of the perceptron learning rule to two positive (●) and two negative (○) data points.

Chapter 2: Machine Learning & Data Mining

5. The Perceptron, a Linear Classifier – The Learning Rule

- **Example 2**
- A perceptron is to be trained on the sets $M_+ = \{(0,1.8), (2,0.6)\}$ and $M_- = \{(-1.2,1.4), (0.4, -1)\}$.
- $w = (1,1)$ was used as an *initial weight vector*.

- In the **first iteration** through the loop of the learning algorithm, the only falsely classified training example is (-1.2,1.4) because $(-1.2,1.4).\begin{pmatrix}1\\1\end{pmatrix} = 0.2 > 0$.
- This results in $w = (1,1) - (-1.2,1.4) = (2.2, -0.4)$, as drawn in the **second image** in the **top row**.
- The other images show how, after a total of **five changes**, the dividing line lies between the two classes.

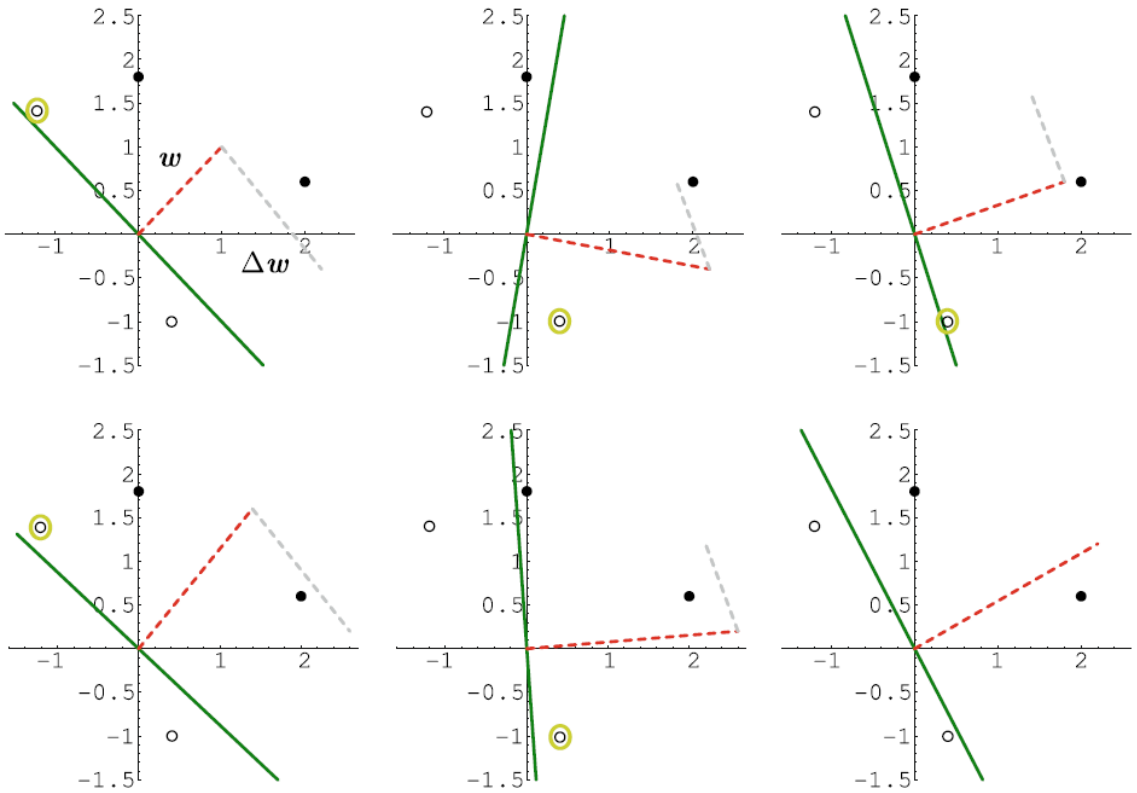


Figure 2.10: application of the perceptron learning rule to two positive (●) and two negative (○) data points.

Chapter 2: Machine Learning & Data Mining

5. The Perceptron, a Linear Classifier – The Learning Rule

- **Example 2**
- A perceptron is to be trained on the sets $M_+ = \{(0,1.8), (2,0.6)\}$ and $M_- = \{(-1.2,1.4), (0.4,-1)\}$.
- $w = (1,1)$ was used as an *initial weight vector*.

- *The **perceptron** thus classifies all data correctly*.

- We clearly see in the example that every incorrectly classified data point from M_+ "pulls" the weight vector w in its direction and every incorrectly classified point from M_- "pushes" the weight vector in the opposite direction.

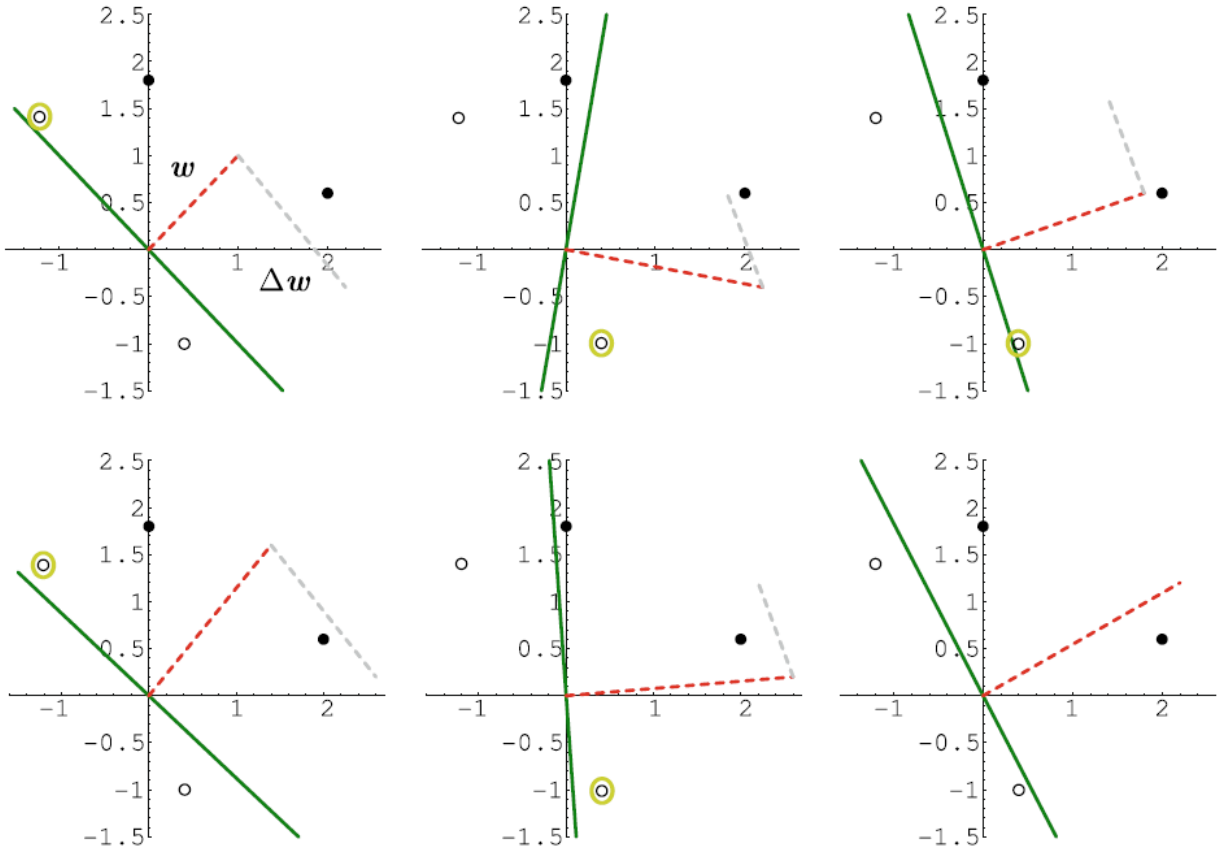


Figure 2.10: application of the perceptron learning rule to two positive (●) and two negative (○) data points.

5. The Perceptron, a Linear Classifier – The Learning Rule

- It has been shown that the **perceptron** *always* **converges** for **linearly separable data**.

> **Theorem 2.1**
>
> *Let classes M_+ and M_- be linearly separable by a hyperplane $\boldsymbol{w}.\boldsymbol{x} = \mathbf{0}$.*
> *Then **Perceptron Learning** converges for every initialization of the vector w.*
> *The perceptron P with the weight vector so calculated divides the classes M_+ and M_-, that is:*
>
> $$P(x) = 1 \quad \Leftrightarrow \quad x \in M_+$$
> *and*
> $$P(x) = 0 \quad \Leftrightarrow \quad x \in M_-$$

- As we can clearly see in this example, **perceptrons** as defined as above cannot divide arbitrary linearly separated sets, rather only those which are divisible by a line through the **origin**, or in $\mathbb{R}^n$ by a hyperplane through the **origin**, because the constant term θ is missing from the equation $\sum_{i=1}^{n} \boldsymbol{w_i}.\boldsymbol{x_i} = \mathbf{0}$.

5. The Perceptron, a Linear Classifier – The Learning Rule

- With the following trick we can generate the constant term.
- We hold the *last component* x_n of the input vector x **constant** and set it to the value **1**. Now the weight $w_n = -\theta$ works like a **threshold** because

$$\sum_{i=1}^{n} w_i . x_i = \sum_{i=1}^{n-1} w_i . x_i - \theta > 0 \quad \Leftrightarrow \quad \sum_{i=1}^{n-1} w_i . x_i > \theta$$

- Such a constant value $x_n = 1$ in the input is called a **bias unit**.
- Because the associated weight causes a **constant shift of the hyperplane**, the term "**bias**" fits well.
- In the application of the perceptron learning algorithm, a bit with the constant value **1** is appended to the training data vector.
- We observe that the weight w_n, or the threshold θ, is *learned* during the learning process!

Chapter 2: Machine Learning & Data Mining

5. The Perceptron, a Linear Classifier – The Learning Rule

- Now it has been shown that a **perceptron** $P_\theta : \mathbb{R}^n \to \{0,1\}$

$$P_\theta(x_1, \ldots, x_{n-1}) = \begin{cases} 1 & if \sum_{i=1}^{n-1} w_i . x_i > \theta \\ 0 & else \end{cases} \quad [2.1]$$

- with an **arbitrary threshold** can be simulated by a perceptron $P: \mathbb{R}^n \to \{0,1\}$ with the threshold **0**.
- If we compare **Theorem 2.1** with the definition of linearly separable, then we see that both statements are equivalent.
- In summary, we have shown that:

Theorem 2.2

*A function $f: \mathbb{R}^n \to \{0,1\}$ can be represented by a **perceptron** if and only if the two sets of positive and negative input vectors are **linearly separable**.*

5. The Perceptron, a Linear Classifier – The Learning Rule

- **Example 3**
- We now train a **perceptron** with a **threshold** on six simple, *graphical binary patterns*, represented in **Figure 2.11**, with 5x5 pixels each.
- The whole right pattern is one of the 22 test patterns for the first pattern with a sequence of *four inverted bits*.

- The training data can be learned by **Perceptron Learning** in four iterations over all patterns.

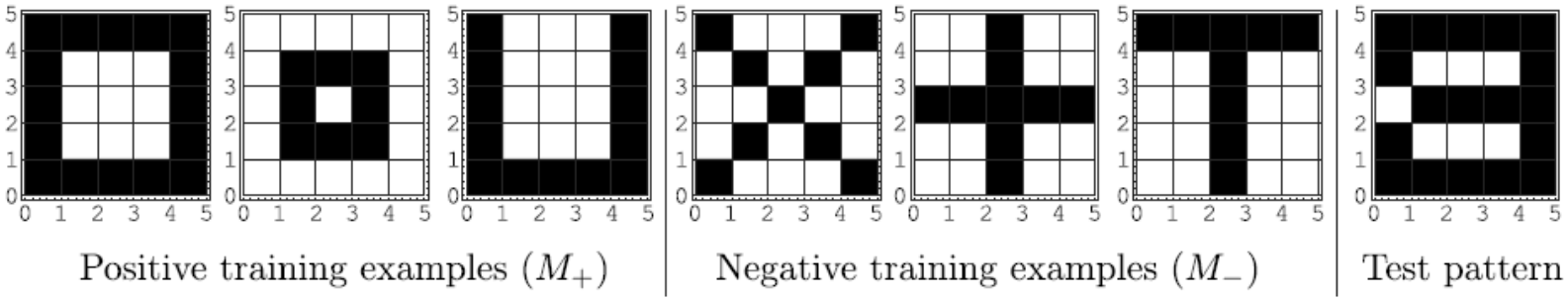


Figure 2.11: The six patterns used for training

Chapter 2: Machine Learning & Data Mining

5. The Perceptron, a Linear Classifier – The Learning Rule

- **Example 3**
- Patterns with *a variable number of inverted bits* introduced as **noise** are used to test the *system's generalization capability*.
- The inverted bits in the test pattern are in each case in sequence one after the other.
- In **Figure 2.12** the percentage of correctly patterns is plotted as a function of the number of false bits.

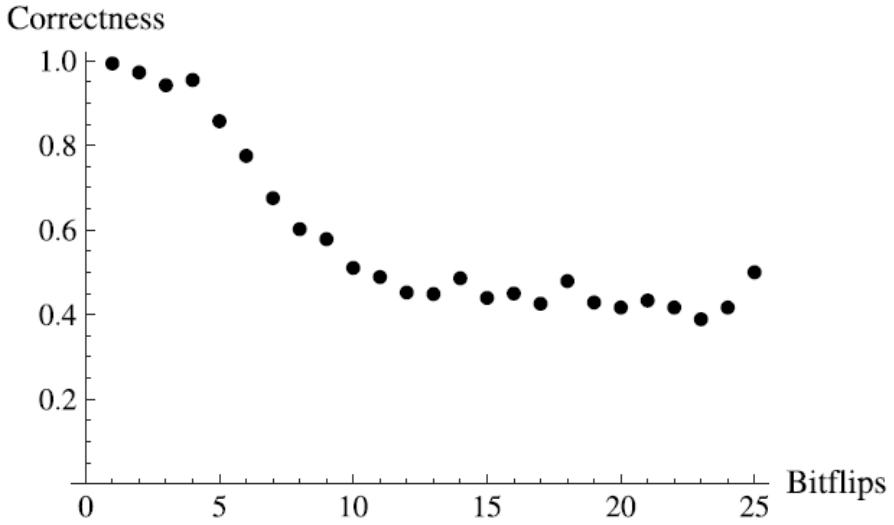


Figure 2.12: Relative correctness of the perceptron as a function of the number of inverted bits in the test data.

5. The Perceptron, a Linear Classifier – Optimization and Outlook

- As one of the simplest neural-network-based learning algorithms, the **two-layer perceptrons** can **only divide linearly separable classes**.
- Later we will see that **multi-layered networks** are significantly more powerful.
- Despite its simple structure, the perceptron in the form presented *converges very slowly*.
- It can be accelerated by **normalization** of the weight-altering vector.
- The formulas $w = w \pm x$ are replaced by $w = w \pm \frac{x}{|x|}$.
- Thereby every data point has the **same weight** during learning, independent of its value.
- The *speed of convergence* heavily depends on the **initialization** of the vector w.
- Ideally, it would not need to be changed at all and the algorithm would converge after one iteration.
- We can get closer to this goal by using the **heuristic initialization**

$$w_0 = \sum_{x \in M_+} x - \sum_{x \in M_-} x$$

AI Fundamentals

Chapter 2: Machine Learning & Data Mining

Contents

Chapter 2: Machine Learning & Data Mining

6. The Nearest Neighbor Method

- For the **perceptron**, **knowledge** available in the **training data** is extracted and saved in a compressed form in the weights w_i.
- Thereby information about the data is lost. This is exactly what is desired, if the system is supposed to *generalize* from the training data to new data.
- ***Generalization*** *in this case is a time-intensive process with the goal of finding a compact representation of data in the form of a* ***function*** *which classifies new data as good as possible.*

- ***Memorization*** *of all data by simply saving them is quite different. Here the 'learning' is extremely simple. However, as previously mentioned, the saved knowledge is not so easily applicable to new, unknow examples.*
- Such an approach is very unfit for **learning** how to ski, for example. A beginner can never become a good skier just by watching videos of good skiers.
- Evidently, when learning movements of this type are automatically carried out, something similar happens in the case of the **perceptron**. After sufficiently long practice, the **knowledge** stored in training examples is transformed into an internal representation in the brain.

6. The Nearest Neighbor Method

- However, there are successful examples of **memorization** in which *generalization* is also possible.
- During the diagnosis of a difficult case, a doctor could try to remember similar cases from the past.
- If his memory is sound, then he might hit upon this case, look it up in his files and finally come a similar diagnosis.
- For this approach the doctor must have a good feeling for **similarity**.

- *What does **similarity** mean in the formal context we are constructing?*

- We represent the training samples as usual in a **multidimensional feature space** and define:
- *The smaller their distance in the feature space, the more two examples are similar.*

Chapter 2: Machine Learning & Data Mining

6. The Nearest Neighbor Method

- We now apply this definition to the simple *two-dimensional example* from **Figure 2.13**.
- In this example with **negative** and **positive** training examples, the **nearest neighbor method** groups the new point marked in **black** into the negative class.
- Here the next neighbor to the **black** point is a **negative** example. Thus it is assigned to the **negative** class.

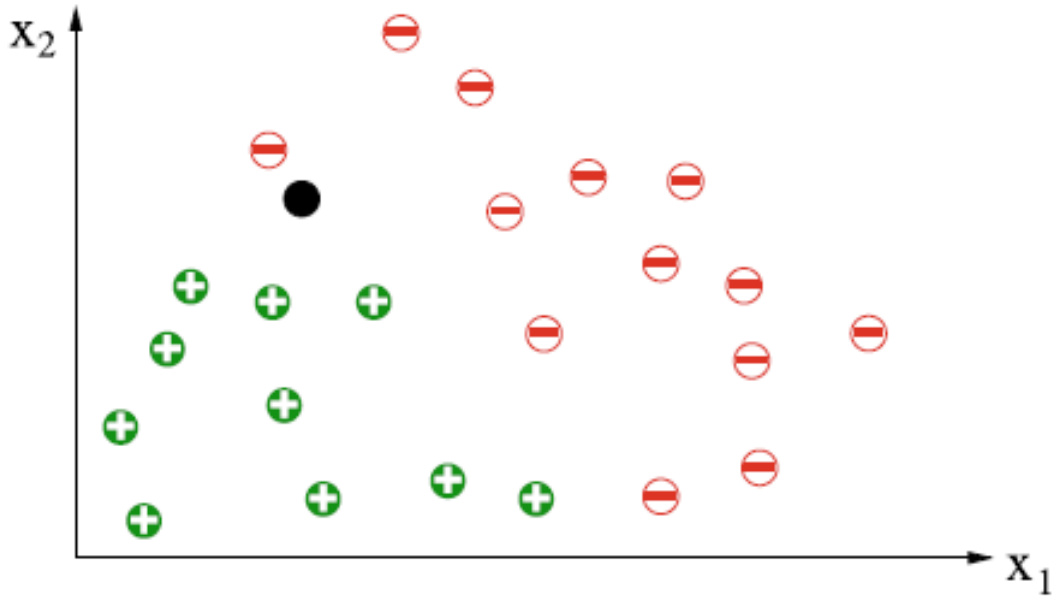


Figure 2.13: Nearest Neighbor method

Chapter 2: Machine Learning & Data Mining

6. The Nearest Neighbor Method

- The distance $d(x, y)$ between two points $x \in \mathbb{R}^n$ and $y \in \mathbb{R}^n$ can for example be measured by the **Euclidean distance**. Points x and y are defined by coordinates (x_1, x_2) and (y_1, y_2).

$$d(x, y) = |x - y| = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2} = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$$

- **URL**: Euclid's afstand
- Because there are *many other distance metrics* besides this one, it makes sense to think about alternatives for a concrete application.
- In many applications, certain **features** are *more important* than others.
- Therefore it is often sensible to **scale** the features differently by **weights** w_i.
- The formula then reads

$$d(x, y) = |x - y| = \sqrt{\sum_{i=1}^{n} w_i . (x_i - y_i)^2}$$

Chapter 2: Machine Learning & Data Mining

6. The Nearest Neighbor Method

- The following **nearest neighbor classification** program searches the training data for the nearest neighbor t to the new example s and then classifies s exactly like t.

NEARESTNEIGHBOR$[M_+, M_-, s]$
$t = \text{argmin}_{x \in M_+ \cup M_-} \{d(s, x)\}$
If $t \in M_+$ **Then Return** („+")
Else Return(„–")

Chapter 2: Machine Learning & Data Mining

6. The Nearest Neighbor Method

- In contrast to the perceptron, the **nearest neighbor method** does *not* generate a **line** that divides the training data points.
- However, an imaginary line separating the two classes certainly exist.
- We can generate this by first generating the so-called **Voronoi diagram** (see **Figure 2.14**).
- **URL**: Voronoi diagram
- In the **Voronoi diagram**, each data point is surrounded by a **convex polygon**, which thus defines a neighborhood around it.

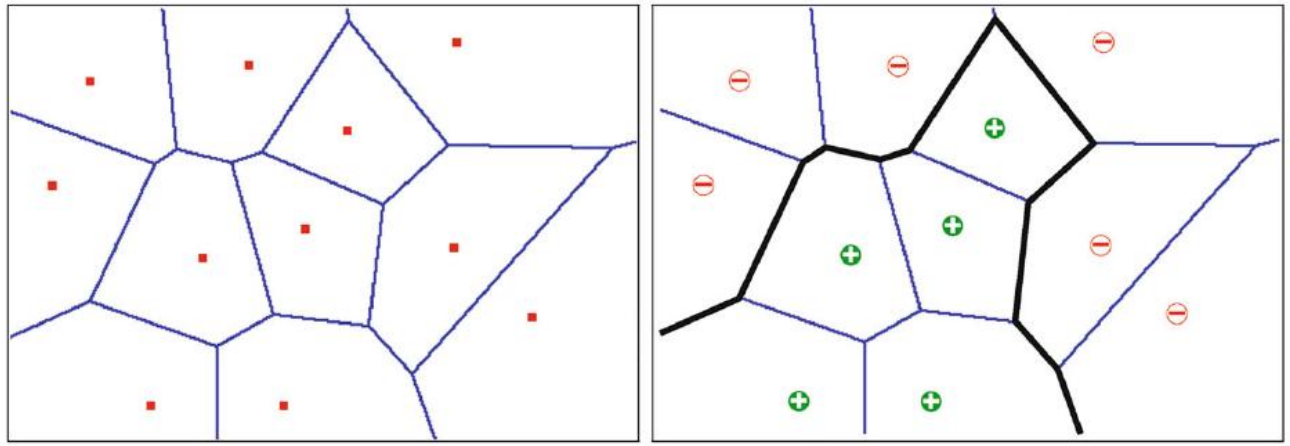


Figure 2.14: Voronoi-Diagram (left) and dividing line (right)

Chapter 2: Machine Learning & Data Mining

6. The Nearest Neighbor Method

- **Figure 2.14** illustrates a set of points with their **Voronoi-Diagram** (left) and the dividing line generated for the two classes M_+ and M_-.
- The **Voronoi diagram** has the property that for an arbitrary new point, the **nearest neighbor** among the data points is the data point, *which lies in the **same neighborhood***.
- If the Voronoi diagram for a set of training data is determined, then it is simple to find the nearest neighbor for a new point which is to be classified.
- The **class membership** will then be taken from the **nearest neighbor**.

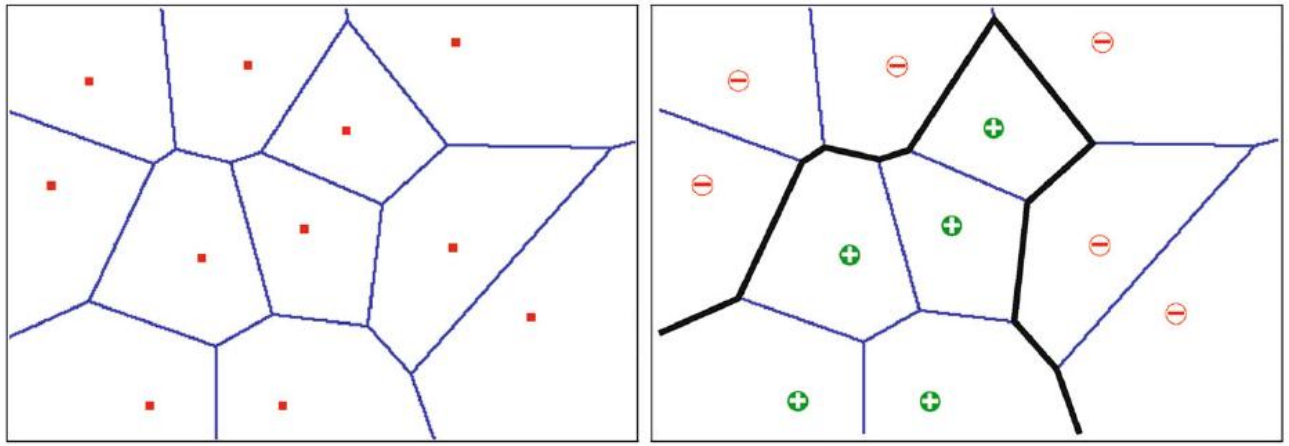


Figure 2.14: Voronoi-Diagram (left) and dividing line (right)

Chapter 2: Machine Learning & Data Mining

6. The Nearest Neighbor Method

- In **Figure 2.14** we see clearly that the **nearest neighbor method** is significantly more powerful than the **perceptron**.
- *It is capable of correctly realizing arbitrarily complex dividing lines* (in general: *hyperplanes*).

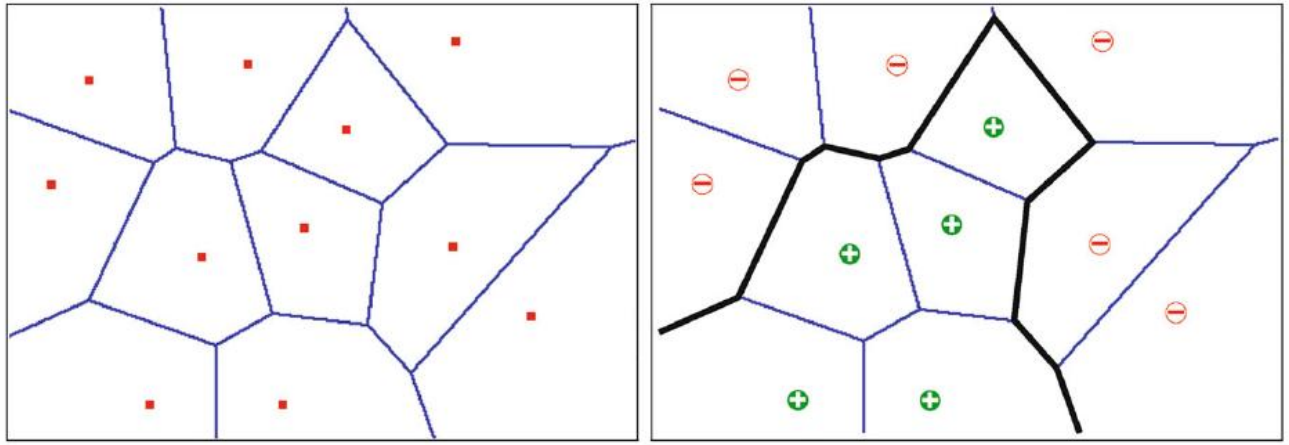


Figure 2.14: Voronoi-Diagram (left) and dividing line (right)

Chapter 2: Machine Learning & Data Mining

6. The Nearest Neighbor Method

- *However there is a danger here!*
- A single erroneous point can in certain circumstances lead to **very bad classification results**. Such a case occurs in **Figure 2.15** during the classification of the **black point**.
- The nearest neighbor method may classify this wrong.

- *If the **black** point is immediately next to a **positive point** that is an outlier of the **positive** class, then it will be classified positive rather than negative as would be intended here.*

- An erroneous fitting to random errors (noise) is called **overfitting**.

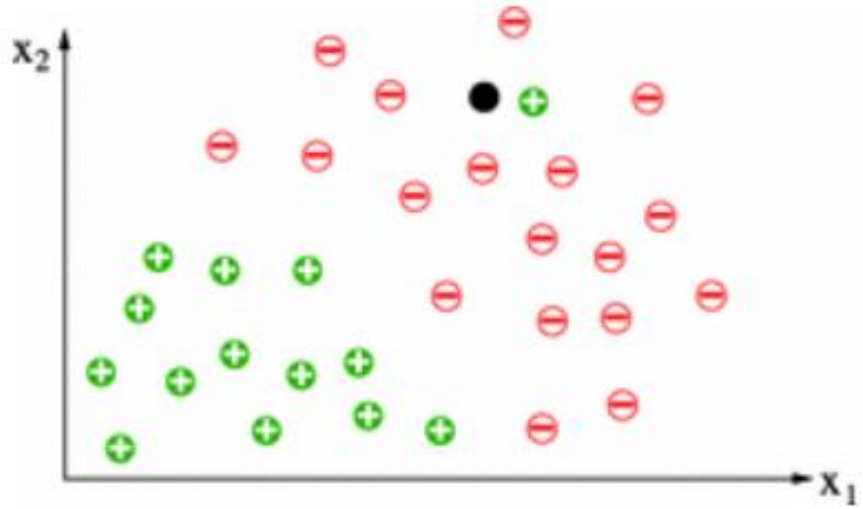


Figure 2.15: Nearest neighbor errorneous classification

Chapter 2: Machine Learning & Data Mining

6. The Nearest Neighbor Method

- To prevent false classifications due to single outliers, it is recommended *to smooth out* the division surface somewhat.
- This can be accomplished by, for example, the ***k*-Nearest Neighbor algorithm** in **Figure 2.16**, which makes a **majority decision** among the ***k*** **nearest neighbors**.

K-NEARESTNEIGHBOR(M_+, M_-, s)

$V = \{k \text{ nearest neighbors in } M_+ \cup M_-\}$

If $|M_+ \cap V| > |M_- \cap V|$ **Then** **Return**(„+”)

ElseIf $|M_+ \cap V| < |M_- \cap V|$ **Then** **Return**(„–”)

Else **Return**(Random(„+”, „–”))

Figure 2.16: The K-Nearest Neighbor algorithm

Chapter 2: Machine Learning & Data Mining

6. The Nearest Neighbor Method

Example 4

- We now apply **Nearest Neighbor** to **Example 3** (graphical binary pattern recognition).
- Because we are dealing with **binary data**, we use the **Hamming distance** as the distance metric.
- **URL**: Hammingafstand
- As a test example, we again use modified training examples with n consecutive inverted bits each.
- In **Figure 2.17** the percentage of correctly classified test examples is shown as a function of the number of inverted bits b.

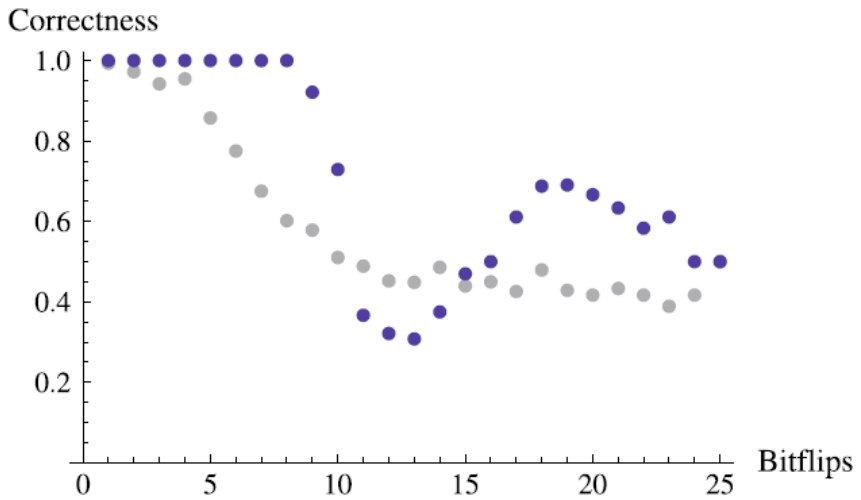


Figure 2.17: Relative correctness of nearest neighbor classification

6. The Nearest Neighbor Method

Example 4

- In **Figure 2.17** the relative correctness of the nearest neighbor classification as a **function** of the number of inverted bits is illustrated.
- The structure of the curve with its *minimum* at **13** and its *maximum* at **19** is related to the special structure of the training data.
- For comparison the values of the **perceptron** from **Example 3** are shown in *gray* (**Figure 2.12**).

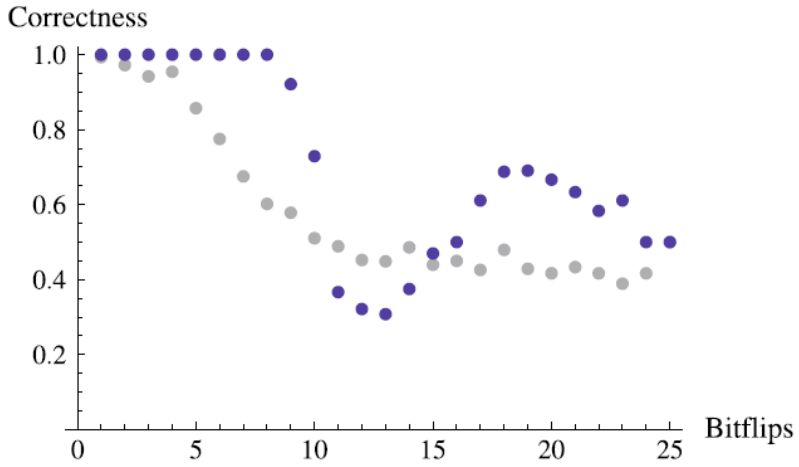


Figure 2.17: Relative correctness of nearest neighbor classification

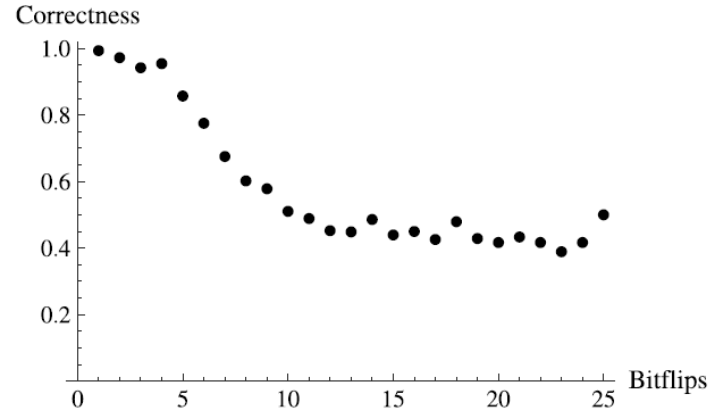


Figure 2.12: Relative correctness of the perceptron as a function of the number of inverted bits in the test data.

Chapter 2: Machine Learning & Data Mining

6. The Nearest Neighbor Method

Example 4

- For up to *eight* inverted bits, all patterns are correctly identified. Past that point, the number of errors quickly increases.
- This is unsurprising because *training pattern number 2* from class M_+ (see **Figure 2.11**) has a hamming distance of 9 to the two training examples, *numbers 4 and 5* from the other class.
- *This means that the test pattern is very likely close to the patterns of the other class.*
- Quite clearly we see that **nearest neighbor classification** is *superior* to the perceptron in this application *for up to eight false bits*.

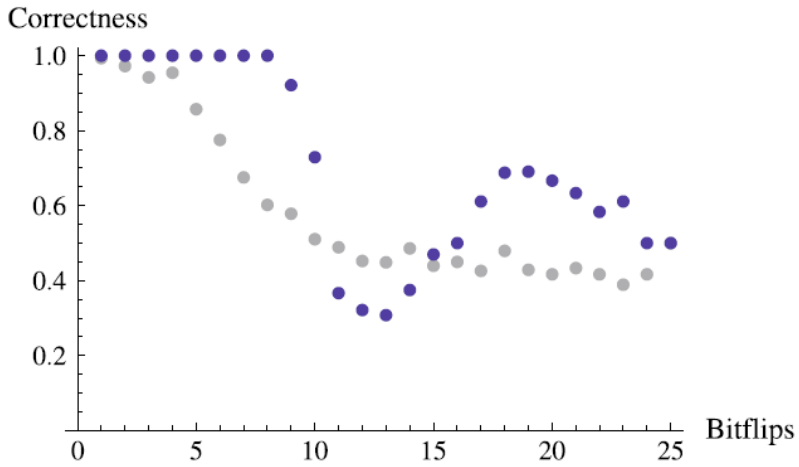


Figure 2.17: Relative correctness of nearest neighbor classification

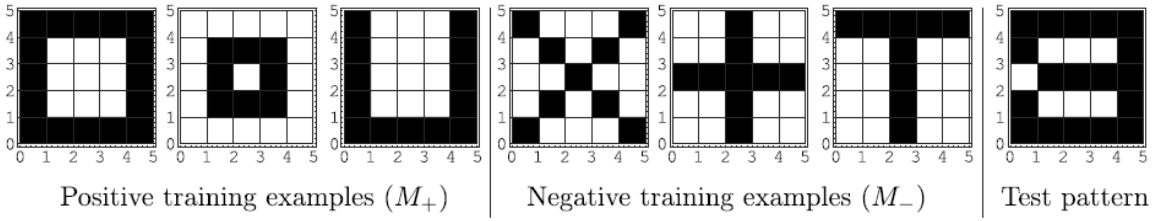


Figure 2.11: The six patterns used for training

Chapter 2: Machine Learning & Data Mining

6. The Nearest Neighbor Method – Two Classes, Many Classes, Approximation

- **Nearest neighbor classification** can also be applied to **more than two classes**.
- Just like the case of two classes, the class of the feature vector to be classified is simply set as the class of the nearest neighbor.
- *For the **k nearest neighbor method**, the class is to be determined as the class with the most members among the k nearest neighbors.*
- **URL:** k-nearest neighbors algorithm
- If the number of classes is *large*, then it usually no longer makes sense to use classification algorithms because the *size* of the necessary *training data* grows quickly with the number of classes.
- Furthermore, in certain circumstances important information is lost during classification of many classes.

Chapter 2: Machine Learning & Data Mining

6. The Nearest Neighbor Method – Two Classes, Many Classes, Approximation

Example 5

- An autonomous robot with simple sensors similar to the *Braitenberg vehicles* (see **Figure 1.1**) is supposed to learn to *move away from the light*.
- This means it should learn as optimally as possible to map its sensor values onto a steering signal which controls the driving direction.
- The robot is equipped with two simple light sensors on its front side.

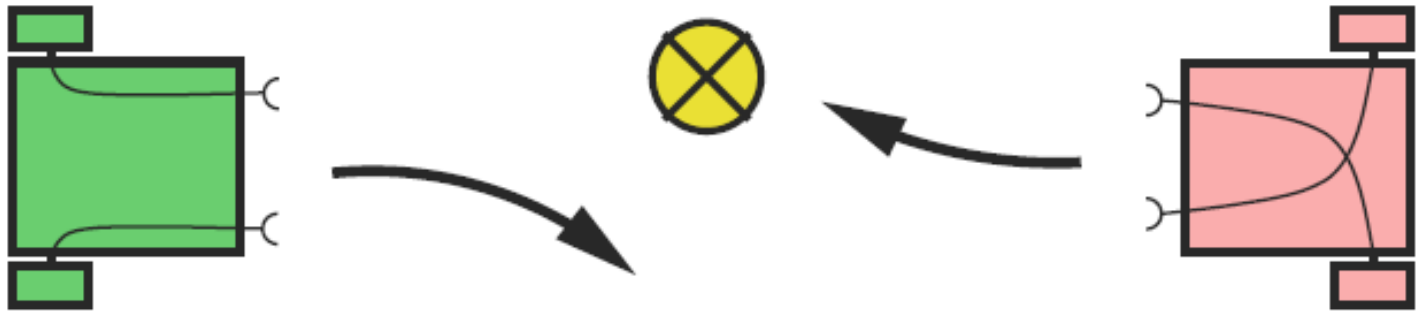


Figure 1.1: Two very simple Braitenberg vehicles and their reactions to a light source

Chapter 2: Machine Learning & Data Mining

6. The Nearest Neighbor Method – Two Classes, Many Classes, Approximation

Example 5

- From the two sensor signals (with s_l for the left and s_r for the right sensor), the relationship $x = \frac{s_r}{s_l}$ is calculated.
- To control the electric motors of the two wheels from this value x, the difference $v = U_r - U_l$ of the two voltages Ur and U_l of the left and right motors, respectively, will be determined.
- The learning agent's task is now *to avoid a light signal*. It must therefore learn a **mapping** f which calculates the "*correct*" value $v = f(x)$.

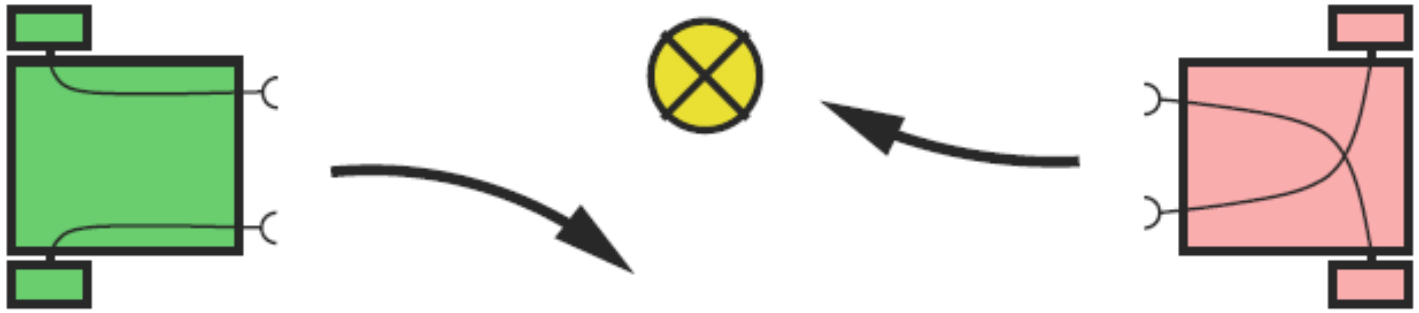


Figure 1.1: Two very simple Braitenberg vehicles and their reactions to a light source

Chapter 2: Machine Learning & Data Mining

6. The Nearest Neighbor Method – Two Classes, Many Classes, Approximation

Example 5

- For this we carry out an experiment in which, for a few measured values x, we find as optimal a value as we can. These values are plotted as data points in **Figure 2.18** and shall serve as training data for the learning agent.
- During **nearest neighbor classification** each point in the feature space (that is, on the $x-axis$), is classified exactly like its nearest neighbor among the training data.

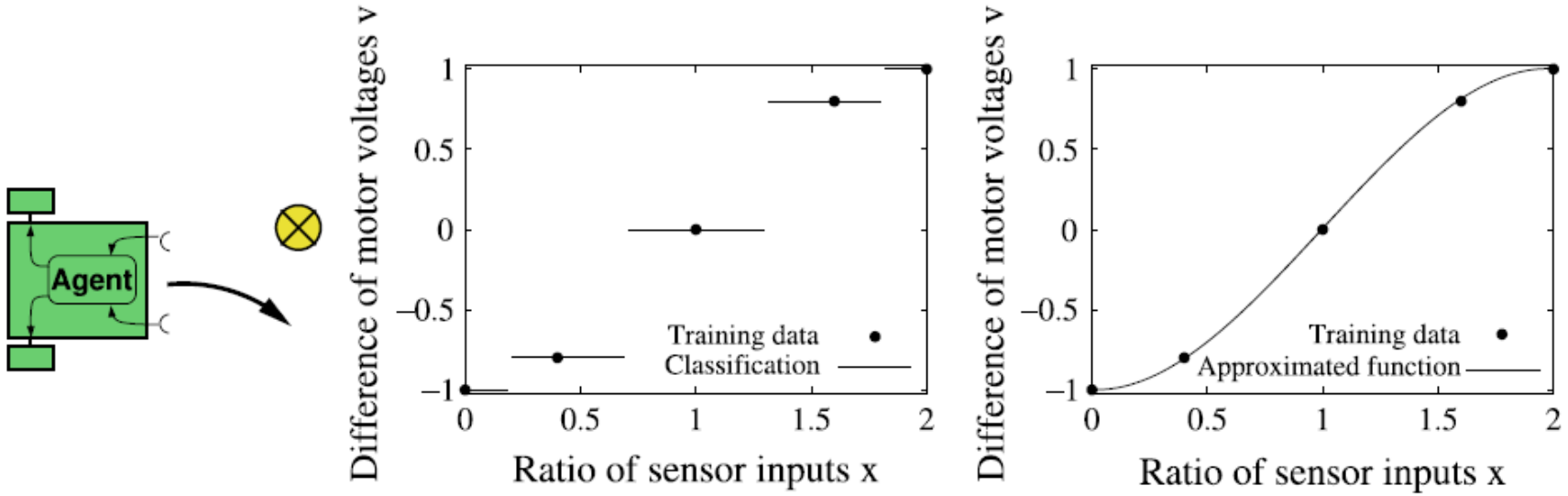


Figure 2.18: Learning agent (left), classifier (middle), approximation (right)

Chapter 2: Machine Learning & Data Mining

6. The Nearest Neighbor Method – Two Classes, Many Classes, Approximation

Example 5

- The function for steering the motors is then a **step function with large jumps** (**Figure 2.18** (*middle*)).
- If we want *finer steps*, then we must provide correspondingly more training data.
- On the other hand, we can obtain a *continuous function* if we approximate a **smooth function** to fit the five points (**Figure 2.18** (*right*)).
- Requiring the function ***f*** to be *continuous* leads to very good results, even with no additional data points.

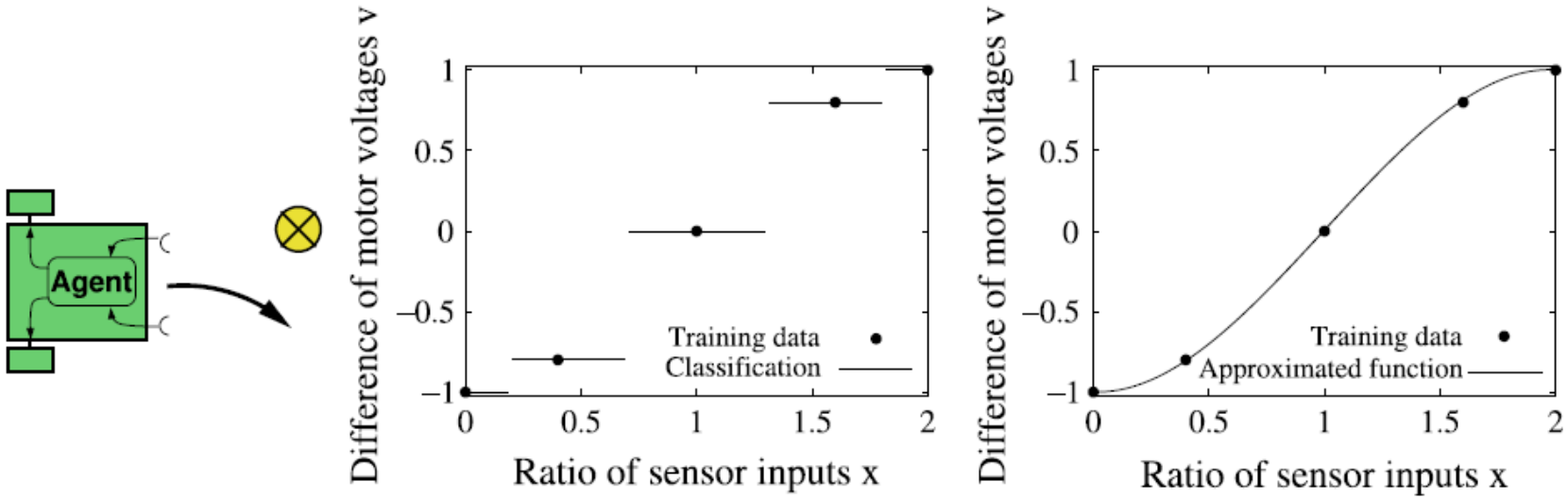


Figure 2.18: Learning agent (left), classifier (middle), approximation (right)

Chapter 2: Machine Learning & Data Mining

6. The Nearest Neighbor Method – Two Classes, Many Classes, Approximation

Example 5

- For the approximation of functions on data points there are many mathematical methods, such as *polynomial interpolation, spline interpolation*, or *the method of least squares*.
- The application of these methods becomes problematic in higher dimensions.
- The special difficulty in AI is that *model-free approximation methods* are needed. That is, a good approximation of the data must be made **without knowledge** about *special properties* of the **data** or the **application**.
- Very good results have been achieved here with *neural networks* and *other nonlinear function approximators*.
- The ***k* nearest neighbor method** can be applied in a simple way to the approximation problem.
- In the ***k* nearest neighbor algorithm**, after the set $V = \{x_1, x_2, \dots, x_k\}$ is determined, the ***k* nearest neighbor average function value**

$$\hat{f}(x) = \frac{1}{k}\sum_{i=1}^{k} f(x_i) \qquad [2.2]$$

- is calculated and taken as an **approximation** $\hat{f}$ for the query vector x. The larger k becomes, the smoother the function $\hat{f}$ is.

6. The Nearest Neighbor Method – Distance is relevant

- In practical application of *discrete* as well as *continuous* variants of the ***k*** **nearest neighbor method**, **problems** often occur.
- As k becomes large, there typically exist more neighbors with a **large** distance that those with a **small** distance. Thereby the calculation of $\hat{f}$ is dominated by neighbors that are *far away*.
- *To prevent this, the k neighbors are weighted such that the more distant neighbors have lesser influence on the result.*
- During the **majority decision** in the k nearest neighbor algorithm, the "*votes*" are **weighted** with the weight

$$w_i = \frac{1}{1 + \alpha d(x, x_i)^2} \qquad [2.3]$$

- which decreases with the square of the distance. The constant α determines the speed of decrease of the weights.

6. The Nearest Neighbor Method – Distance is relevant

- **Equation [2.2]** is now replaced by

$$\hat{f}(x) = \frac{\sum_{i=1}^{k} w_i \cdot f(x_i)}{\sum_{i=1}^{n} w_i}$$

- *For uniformly distributed concentration of points in the feature space, this ensures that the influence of points asymptotically approaches zero as distance increases.*

- Thereby it becomes possible to use **many** or even **all training data** to classify or approximate a given input vector.

6. The Nearest Neighbor Method – Distance is relevant

- To get a feeling for these methods, in **Figure 2.19** the ***k* nearest neighbor method** (in the upper row) is compared with its **distance weighted optimization** (lower row).
- **Figure 2.19** illustrates the comparison of the ***k* nearest neighbor method** (upper row) with k = 1 (left), k = 2 (middle) and k = 6 (right), to its distance weighted value (lower row), with α = 20 (left), α = 4 (middle) and α= 1 (right) on a one-dimensional dataset.

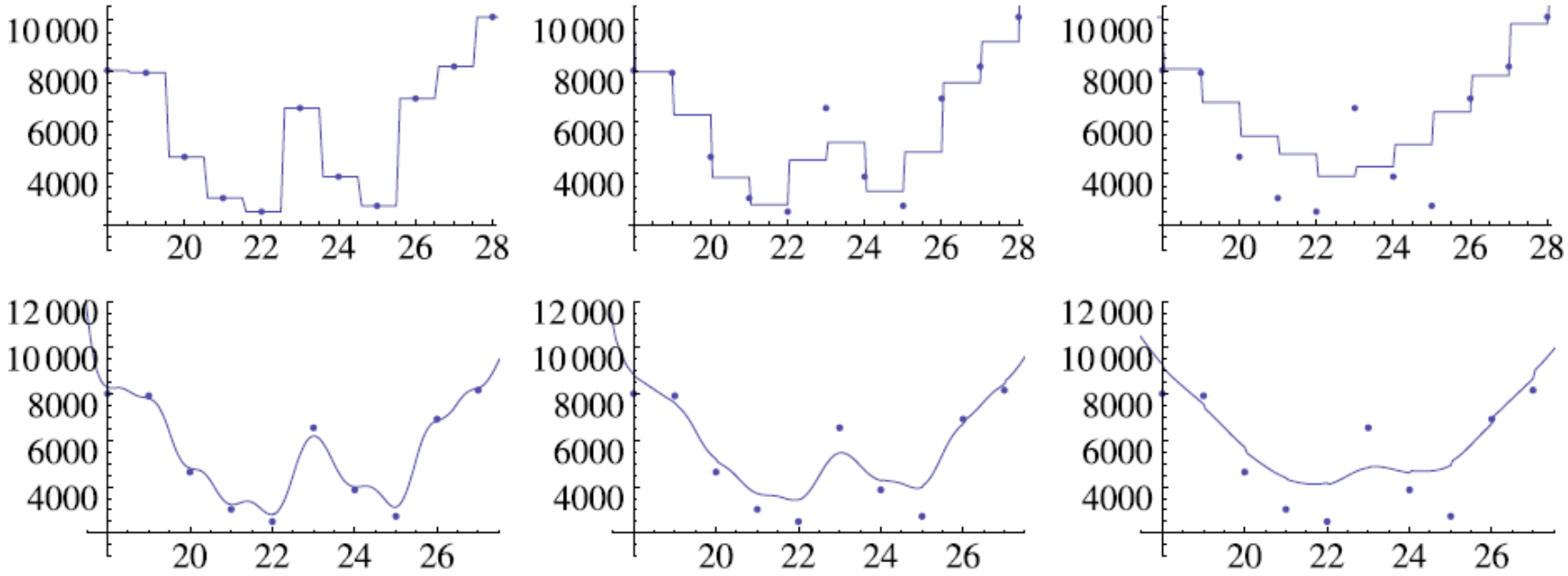


Figure 2.19: Comparison of the k nearest neighbor method

6. The Nearest Neighbor Method – Distance is relevant

- There are many alternatives to the **weight activation function** (also called **kernel**), given in **[2.3].**
- For example a *Gaussian* or *similar bell-shaped function* can be used.
- *For most applications, the results are **not** very sensible on the selection of the kernel.*
- However, the weight parameter α (speed of decrease of the weights) which for all these functions has to be set **manually** has great influence on the result, as shown in **Figure 2.19**.
- To avoid such an inconvenient manual adaption, *optimization methods* have been developed for automatically setting this parameter.

6. The Nearest Neighbor Method – Computation times

- As previously mentioned, training is accomplished in all variants of the nearest neighbor method by simply *saving* all training **vectors** together with their **labels** (class values), OR the function value $f(x)$.
- *Thus there is no learning algorithm that learns as quickly.*
- However, answering a query for classification or approximation of a vector x can become **very expensive**.
- Just finding the k nearest neighbors for n training data points requires a cost with *grows linearly with* n.
- For **classification** or **approximation**, there is additionally a cost which is *linear in* k.
- The **total computation time** thus grows as $\Theta(n+k)$. For large amounts of training data, this can lead to problems.

Chapter 2: Machine Learning & Data Mining

6. The Nearest Neighbor Method – Summary and Outlook

- Because nothing happens in the learning phase of the presented nearest neighbor method, such algorithms are also denoted **lazy learning**, in contrast to **eager learning**, in which the learning phase can be expensive, but application to new examples is very efficient.

- **URL**: Lazy learning
- **URL**: Eager learning

- The *perceptron* and *all other neural networks*, *decision tree learning*, and the *learning of Bayesian networks* are **eager learning methods**.

- Since the **lazy learning** methods need access to the **memory** with all training data for approximating a new input, they are also called **memory-based learning**.

Chapter 2: Machine Learning & Data Mining

6. The Nearest Neighbor Method – Summary and Outlook

- To compare these two classes of learning processes, we will use as an example task of determining the current **avalanche hazard** from the amount of newly fallen snow in a certain area in Switzerland.
- In **Figure 2.20** values determined by experts are entered, which we want to use as training data.
- To determine the avalanche hazard, a function is approximated from training data. Here for comparison are a **local** model (solid line), and a **global** model (dashed line).

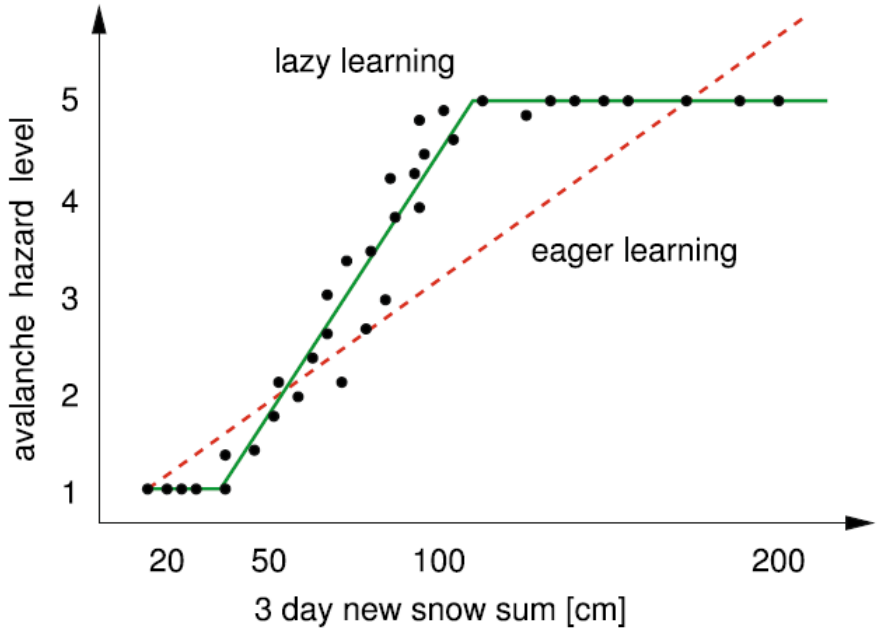


Figure 2.20: Function approximation to determine the avalanche hazard

Chapter 2: Machine Learning & Data Mining

6. The Nearest Neighbor Method – Summary and Outlook

- During the application of an **eager** learning algorithm which undertakes a linear approximation of the data, the dashed line shown in the figure is calculated.
- *Due to the restriction to a straight line, the error is relatively large with a maximum of about 1.5 hazard levels.*
- During **lazy** learning, nothing is calculated before a query for the current hazard level arrives. Then the answer is calculated from *several nearest neighbors*, that is, **locally**.
- It could result in the curve shown in the figure, which is put together from line segments and shows much smaller errors.
- *The advantage of the lazy method is its **locality**.*
- The approximation is taken **locally** from the current snow level and not globally.
- *Thus for fundamentally equal classes of functions (for example linear functions), the lazy algorithms are better.*

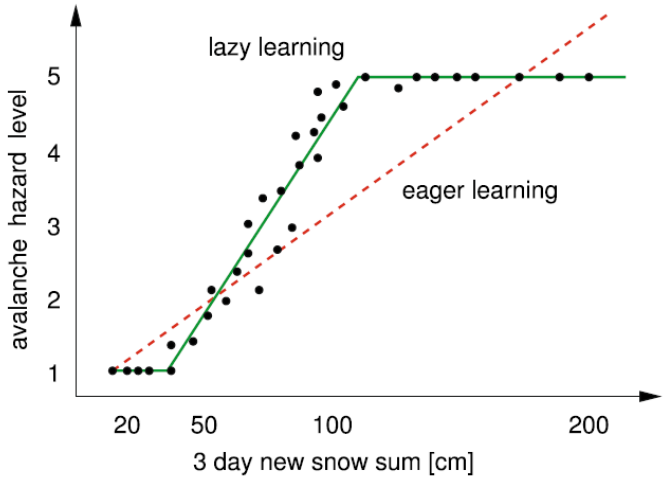


Figure 2.20: Function approximation to determine the avalanche hazard

Chapter 2: Machine Learning & Data Mining

6. The Nearest Neighbor Method – Summary and Outlook

- ***Nearest neighbor methods*** *are well suited for all problem situations in which a good* ***local approximation*** *is needed, but which do* ***not*** *place a* ***high requirement on*** *the* ***speed*** *of the system.*

- The avalanche predictor mentioned here, which runs once per day, is such an application.

- ***Nearest neighbor methods*** *are* ***not*** *suitable when a description of the knowledge extracted from the data must be understandable by humans (= symbolic), which today is the case for many* ***data mining*** *applications.*

- In recent years these memory-based learning methods are becoming popular, and various improved variants (for example *locally weighted linear regression*) have been applied.

- To be able to use the described methods, the training data must be available in the form of **vectors** of **integers** or **real numbers**. They are unsuitable for application in which the data are represented **symbolically**.

AI Fundamentals

Chapter 2: Machine Learning & Data Mining

Contents

Chapter 2: Machine Learning & Data Mining

7. Case-Based Reasoning

- In **case-based reasoning** (CBR), the nearest neighbor method is extended to **symbolic problem descriptions** and their solutions.
- CBR is used in the diagnosis of technical problems in *customer service* or for *telephone hotlines*.
- The example shown in **Figure 2.21** about the diagnosis of a bicycle light going out illustrates this type of situation.

Feature	Query	Case from case base
Defective part:	Rear light	Front light
Bicycle model:	Marin Pine Mountain	VSF T400
Year:	1993	2001
Power source:	Battery	Dynamo
Bulb condition:	ok	ok
Light cable condition:	?	ok
	Solution	
Diagnosis:	?	Front electrical contact missing
Repair:	?	Establish front electrical contact

Figure 2.21: Simple diagnosis example

Chapter 2: Machine Learning & Data Mining

7. Case-Based Reasoning

- A solution is searched for the query of a customer with a **defective rear bicycle light**.
- In the right column, a case similar to the query in the middle column is given.
- *This stems from the case base, which corresponds to training data in the **nearest neighbor method**.*

Feature	Query	Case from case base
Defective part:	Rear light	Front light
Bicycle model:	Marin Pine Mountain	VSF T400
Year:	1993	2001
Power source:	Battery	Dynamo
Bulb condition:	ok	ok
Light cable condition:	?	ok
	Solution	
Diagnosis:	?	Front electrical contact missing
Repair:	?	Establish front electrical contact

Figure 2.21: Simple diagnosis example

- If we simply took the most similar case, as we do in the *nearest neighbor method*, then we would end up trying to repair the **front** light when the **rear** light is broken.
- *We thus need a reverse transformation of the solution to the discovered similar problem back to the query.*

Chapter 2: Machine Learning & Data Mining

7. Case-Based Reasoning

- The most important steps in the solution to a CBR case are carried out in **Figure 2.22**.

- If for a case x a similar case y is found, then to obtain a solution for x, the transformation must be determined and its inverse applied to the discovered case y.

- The transformation in this example is simple: **rear light** is mapped to **front light**.

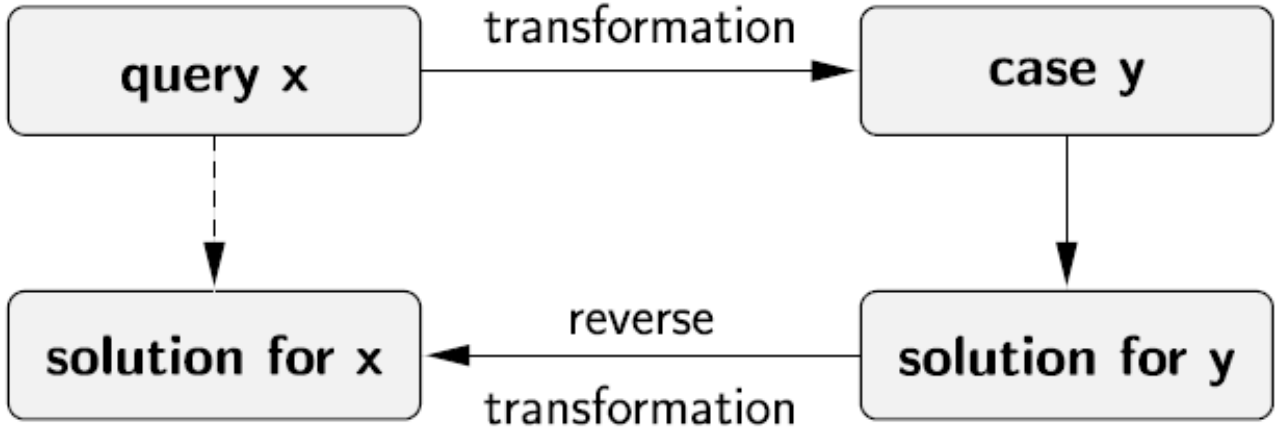


Figure 2.22: Transformation and reverse transformation in CBR

Chapter 2: Machine Learning & Data Mining

7. Case-Based Reasoning

- As beautiful and simple as this methods seems in theory, in practice the construction of CBR diagnostic systems is *a very difficult task*.
- The **three main difficulties** are:
  - **Modeling**: The domains of the application must be modeled in a **formal context**. *Can the developer predict and map all possible special cases and problem variants?*
  - **Similarity**: Finding a suitable similarity metric for **symbolic**, non-numerical features.
  - **Transformation**: Even if a similar case is found, it is not yet clear how the transformation mapping and its inverse should look.
- Indeed there are practical CBR systems for **diagnostic applications** in use today.
- However, due to the reasons mentioned, these remain *far behind human experts* in performance and flexibility.
- An interesting alternative to CBR are the **Bayesian networks** (see *chapter 7* in the book).
- Often the **symbolic** problem representation can also be mapped quite well to *discrete* or *continuous* numerical features. Then the mentioned **inductive learning methods** such as **decision trees** or **neutral networks** can be used successfully.

AI Fundamentals

Chapter 2: Machine Learning & Data Mining

Contents

8. Decision Tree Learning

- **Decision tree learning** is an extraordinary important algorithm for AI because it is *very powerful*, but also *simple* and *efficient* for extracting knowledge from data.
- Compared to the two already introduced learning algorithms, it has an important advantage.
- The extracted knowledge is not only available and usable as a black box function, but rather it can be easily understood, interpreted, and controlled by humans in the form of a readable **decision tree**.
- This also makes **decision tree learning** an important tool for **data mining**.

Chapter 2: Machine Learning & Data Mining

8. Decision Tree Learning – A simple example

- A devoted skier who lives near the high sierra, a beautiful mountain range in California, wants a decision tree to help him decide whether it is worthwhile to drive his car to a ski resort in the mountains.
- We thus have a two-class problem ski yes/no based on the variables listed in **Table 2.4**.

Variable	Value	Description
Ski (goal variable)	yes, no	Should I drive to the nearest ski resort with enough snow?
Sun (feature)	yes, no	Is there sunshine today?
Snow_Dist (feature)	≤100, >100	Distance to the nearest ski resort with good snow conditions (over/under 100 km)
Weekend (feature)	yes, no	Is it the weekend today?

Table 2.4: Variables for skiing classification problem

Chapter 2: Machine Learning & Data Mining

8. Decision Tree Learning – A simple example

- **Figure 2.23** shows a decision tree for this problem.
- A **decision tree** is a **tree** whose inner nodes represent **features** (attributes). Each *edge* stands for an **attribute** value. At each leaf node a *class value* is given.
- In the lists to the right of the nodes, the numbers of the corresponding training data are given. Notice that of the leaf nodes **sunny = yes** *only two of the three examples* are classified **correctly**.

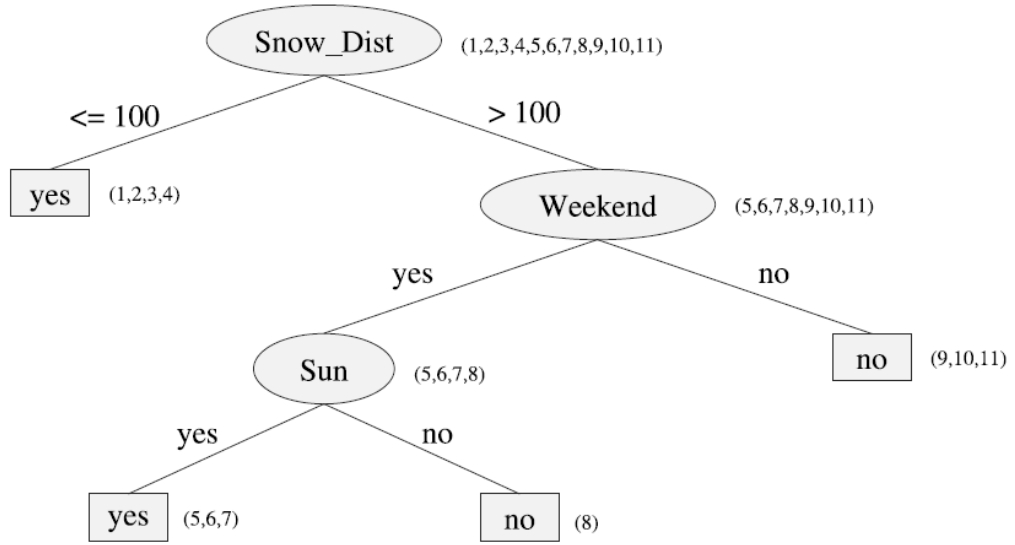


Figure 2.23: Decision tree for the skiing classification problem

Table 2.5: Data set for the skiing classification problem

Day	*Snow_Dist*	*Weekend*	*Sun*	*Skiing*
1	≤100	yes	yes	yes
2	≤100	yes	yes	yes
3	≤100	yes	no	yes
4	≤100	no	yes	yes
5	>100	yes	yes	yes
6	>100	yes	yes	yes
7	>100	yes	yes	no
8	>100	yes	no	no
9	>100	no	yes	no
10	>100	no	yes	no
11	>100	no	no	no

Chapter 2: Machine Learning & Data Mining

8. Decision Tree Learning – A simple example

- Each row in **Table 2.5** contains the data for one day and as such represents a sample.
- Upon closer examination we see that *row 6* and *row 7* contradict each other. Thus no deterministic classification algorithm can correctly classify all of the data. *The number of falsely classified data must therefore be* ≥ 1. The tree in **Figure 2.23** classifies the data **optimally**.

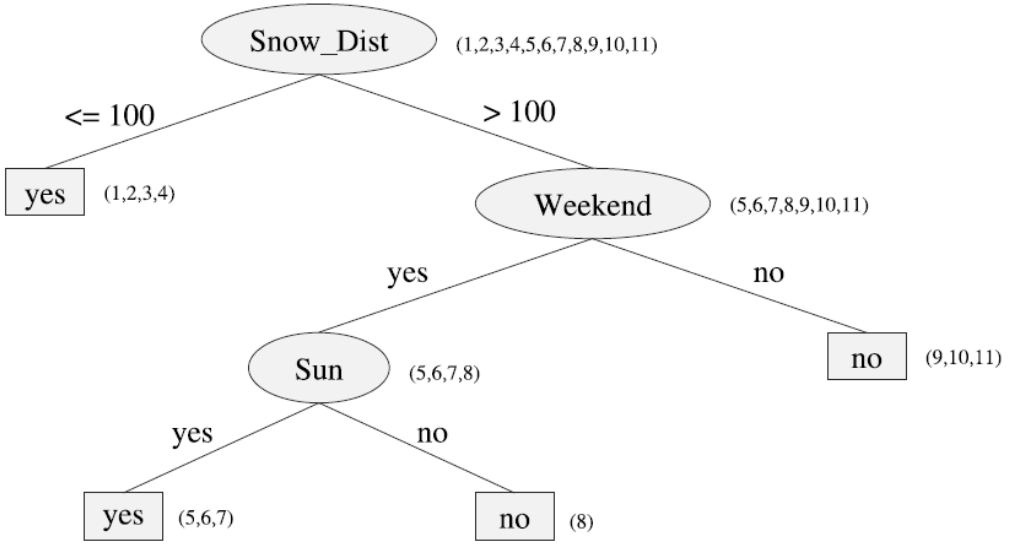


Figure 2.23: Decision tree for the skiing classification problem

Table 2.5: Data set for the skiing classification problem

Day	*Snow_Dist*	*Weekend*	*Sun*	*Skiing*
1	≤100	yes	yes	yes
2	≤100	yes	yes	yes
3	≤100	yes	no	yes
4	≤100	no	yes	yes
5	>100	yes	yes	yes
6	>100	yes	yes	yes
7	>100	yes	yes	no
8	>100	yes	no	no
9	>100	no	yes	no
10	>100	no	yes	no
11	>100	no	no	no

8. Decision Tree Learning – A simple example

- *How is such a tree created from the data?*
- To answer this question we will at first restrict ourselves to discrete **attributes** with finitely many values.
- Because the **number of attributes** is also **finite** and each **attribute** can occur **at most once per path**, there are *finitely* many different decision trees.

- *A simple, obvious algorithm for the construction of a tree would simple generate all trees, then for each tree calculate the number of erroneous classifications of the data, and at the end choose the tree with the minimum number of errors*.
- Thus we would even have an **optimal algorithm** (in the sense of errors for the training data) for **decision tree learning**.

8. Decision Tree Learning – A simple example

- The obvious disadvantage of this algorithm is its *unacceptably high computation time*, as soon as the number of attributes becomes somewhat larger.
- We will now develop a **heuristic algorithm** which, starting from the root, **recursively** builds a decision tree.
  - **URL**: Heuristic (computer science)
  - **URL**: Recursie
  - **URL**: Decision tree learning
- First the attribute with the **highest information gain** (*Snow_Dist*) is chosen for the **root node** from the set of all attributes.
- For each attribute value ($\leq 100, > 100$) there is a branch in the tree.
- Now for every branch this process is repeated *recursively*.
- During generation of the nodes, the attribute value with the **highest information gain** among the attributes which have not yet been used is always chosen, in the spirit of a *greedy strategy*.

8. Decision Tree Learning

Entropy as a Metric for Information Content

- The described top-down algorithm for the construction of a decision tree, at each step selects the attribute with the **highest information gain**.
- We now introduce the **entropy** as *the* metric for the information content of a set of training data D.
- If we look at the binary variable **skiing** in the above example, then D can be described as

$$D = (yes, yes, yes, yes, yes, no, no, no, no, no)$$

- with estimate probabilities $p_1 = P(yes) = \frac{6}{11}$ en $p_2 = P(no) = \frac{5}{11}$
- Here we evidently have a probability distribution $p = \left(\frac{6}{11}, \frac{5}{11}\right)$.
- In general, for an n class problem this reads $p = (p_1, \ldots, p_n)$ with

$$\sum_{i=1}^{n} p_i = 1$$

Table 2.5: Data set for the skiing classification problem

Day	*Snow_Dist*	*Weekend*	*Sun*	*Skiing*
1	≤100	yes	yes	yes
2	≤100	yes	yes	yes
3	≤100	yes	no	yes
4	≤100	no	yes	yes
5	>100	yes	yes	yes
6	>100	yes	yes	yes
7	>100	yes	yes	no
8	>100	yes	no	no
9	>100	no	yes	no
10	>100	no	yes	no
11	>100	no	no	no

Chapter 2: Machine Learning & Data Mining

8. Decision Tree Learning – Entropy as a Metric for Information Content

- To introduce the **information content** of a **distribution** we observe two extreme cases.
- First let

$$p = (1,0,0,\dots,0) \qquad \textbf{[2.4]}$$

- That is, the **first one** of the n events will **certainly** occur and all others will not.
- In contrast, for the **uniform distribution**

$$p = \left(\frac{1}{n}, \frac{1}{n}, \dots, \frac{1}{n}\right) \qquad \textbf{[2.5]}$$

- the **uncertainty** is **maximal** because no event can be distinguished from the others.
- Here **Claude Shannon** asked himself how many bits would be needed to encode such an event.
- In the certain case of **[2.4]** zero bits are needed because we know that the case $i = 1$ will always occur.
- In the uniformly case of **[2.5]** there are n equally probable possibilities.
- For **binary** encodings, $\boldsymbol{log_2(n)}$ **bits** are needed here. Because all individual probabilities are $p_i = \frac{1}{n}$, $log_2\left(\frac{1}{p_i}\right)$ bits are needed for this encoding.

8. Decision Tree Learning – Entropy as a Metric for Information Content

- In the general case $p = (p_1, \ldots, p_n)$, if the probabilities of the elementary events deviate from the uniform distribution, then the **expectation value H** for the number of bits is calculated.

- To this end we will weight all values $log_2\left(\frac{1}{p_i}\right) = -log_2(p_i)$ with their probabilities and obtain

$$H = \sum_{i=1}^{n} p_i \,.\left(-log_2(p_i)\right) = -\sum_{i=1}^{n} p_i . log_2\,(p_i)$$

- *The more bits we need to encode an event, clearly the higher the uncertainty about the outcome.*

Chapter 2: Machine Learning & Data Mining

8. Decision Tree Learning – Entropy as a Metric for Information Content

- Therefore we define:

Definition 2.4

*The **entropy H** as a metric for the **uncertainty** of a **probability distribution** is defined by*

$$H(p) = H(p_1, \ldots, p_n) = -\sum_{i=1}^{n} p_i \, log_2(p_i)$$

Chapter 2: Machine Learning & Data Mining

8. Decision Tree Learning

Entropy as a Metric for Information Content

- Now we can calculate $H(1,0,\dots,0) = 0$.
- We will show that the **entropy** in the **hypercube** $[0,1]^n$ under the constraint $\sum_{i=1}^{n} p_i = 1$ takes on its maximum value with the uniform distribution $\left(\frac{1}{n},\dots,\frac{1}{n}\right)$.
- In the case of an event with **two possible outcomes**, which correspond to two classes, the result is

$$H(p) = H(p_1, p_2) = H(p_i, 1-p_i)$$
$$= -\left(p_1 log_2(p_1) + (1-p_1) log_2(1-p_1)\right)$$

- The expression is shown as a function of p_1 in **Figure 2.24** with its maximum at $p_1 = \frac{1}{2}$.
- We see the **maximum** at $p = \frac{1}{2}$ and the **symmetry** with respect to swapping p and $(1-p)$.

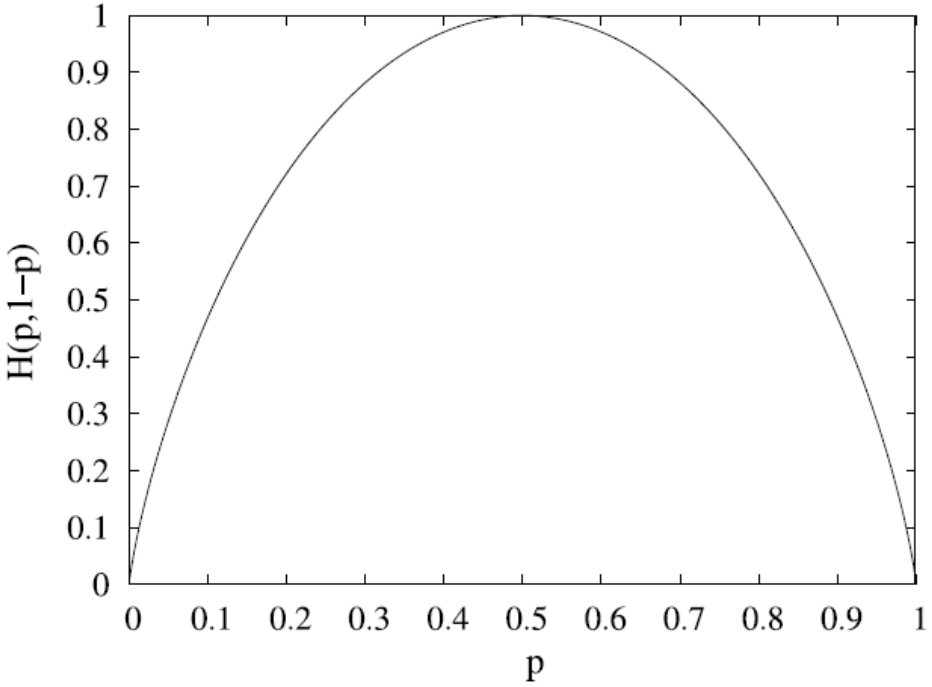


Figure 2.24: The entropy function for the case of two classes

8. Decision Tree Learning – Entropy as a Metric for Information Content

- Because each classified dataset D is assigned a probability distribution p by estimating the class probabilities, we can extend the concept of **entropy** to data by the definition

$$H(D) = H(p)$$

- Now, since the **information content** $I(D)$ of the dataset D is meant to be the **opposite of uncertainty**.

Definition 2.5

The information content of a dataset is defined as

$$I(D) = 1 - H(D) \qquad \textbf{[2.6]}$$

8. Decision Tree Learning – Information Gain

- If we apply the entropy formula to the example, the result is $H\left(\frac{6}{11}, \frac{5}{11}\right) = \mathbf{0.994}$
- During construction of a **decision tree**, the dataset is further subdivided by each new attribute.
- *The more an attribute raises the information content of the distribution by dividing the data, the better the attribute is.*
- We define accordingly:

Definition 2.6

*The **information gain** $G(D, A)$ through the use of the attribute A is determined by the difference of the average information content of the dataset $D = D_1 \cup D_2 \cup \cdots \cup D_n$ divided by the n-value attribute A and the information content $I(D)$ of the undivided dataset, which yields*

$$G(D, A) = \sum_{i=1}^{n} \frac{|D_i|}{|D|} I(D_i) - I(D)$$

8. Decision Tree Learning – Information Gain

- With **Definition 2.6** we obtain from this

$$G(D,A) = \sum_{i=1}^{n} \frac{|D_i|}{|D|} I(D_i) - I(D) = \sum_{i=1}^{n} \frac{|D_i|}{|D|} \big(1 - H(D_i)\big) - \big(1 - H(D)\big)$$

$$= 1 - \sum_{i=1}^{n} \frac{|D_i|}{|D|} H(D_i) - 1 + H(D)$$

$$= H(D) - \textstyle\sum_{i=1}^{n} \frac{|D_i|}{|D|} H(D_i) \qquad \textbf{[2.7]}$$

- Applied to our example for the attribute *Snow_Dist*, this yields

$$G(D, Snow\_Dist) = H(D) - \left(\frac{4}{11} H(D \leq 100) + \frac{7}{11} H(D > 100)\right)$$

$$= 0.994 - \left(\frac{4}{11}.0 + \frac{7}{11}.0.863\right) = \mathbf{0.445}$$

8. Decision Tree Learning – Information Gain

- Analogously we obtain $G(D, Weekend) = \mathbf{0.150}$ and $G(D, Sun) = \mathbf{0.049}$
- The attribute *Snow_Dist* now becomes the **root node** of the decision tree.
- The situation of the selection of this attribute is once again clarified in **Figure 2.25**.
- The calculated gain for the various attributes reflects whether the division of the data by the respective attribute results in a better class division. *The more the distributions generated by an attribute deviate from the uniform distribution, the higher the information gain*.

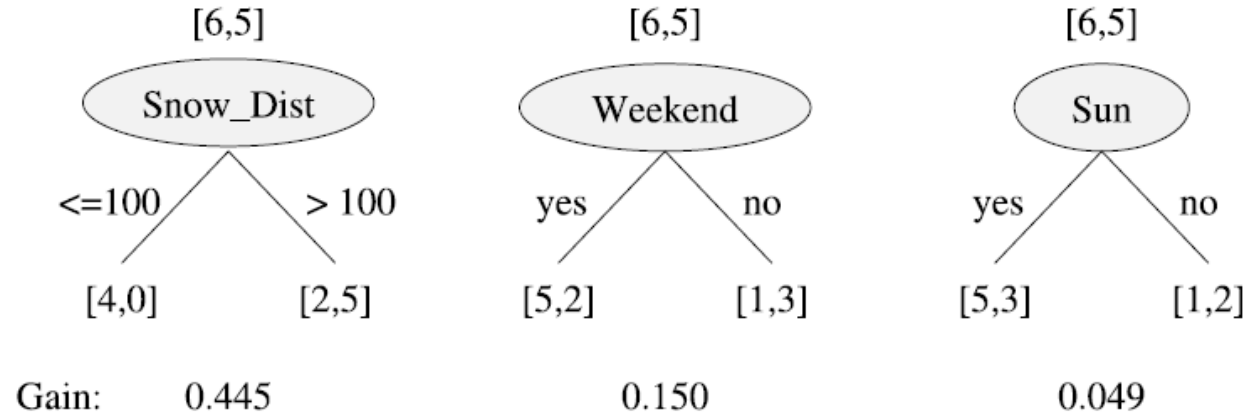


Figure 2.25: The calculated gain for the various attributes

8. Decision Tree Learning – Information Gain

- The two attribute values ≤ 100 and > 100 generate two edges in the tree, which correspond to the subsets $D_{\leq 100}$ and $D_{>100}$.
- For the subset $D_{\leq 100}$ the classification is clearly *yes,* thus the tree terminates here.
- In the other branch $D_{>100}$ there is no clear result.
- Thus the algorithm repeats **recursively**.

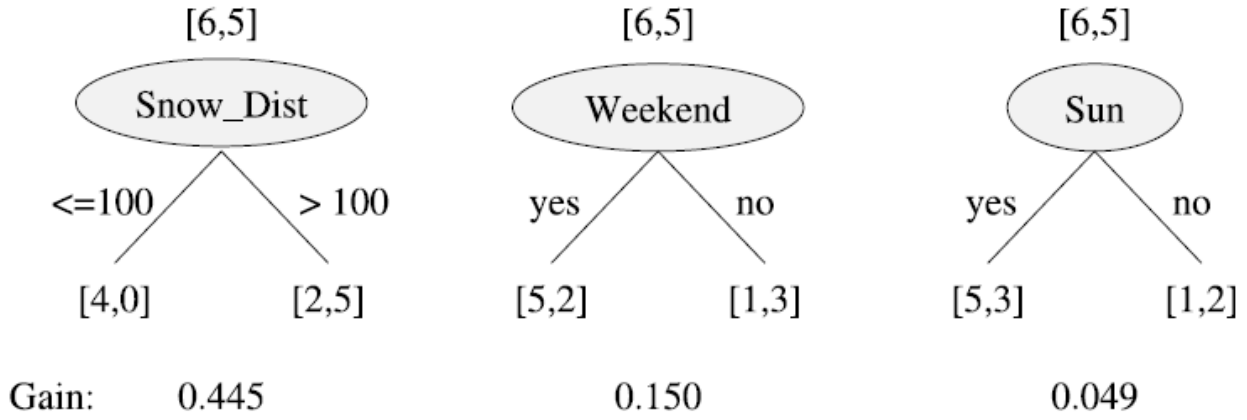


Figure 2.25: The calculated gain for the various attributes

8. Decision Tree Learning – Information Gain

- From the two attributes still available, *Sun* and *Weekend*, the better one must be chosen.
- We calculate $G(D_{>100}, Weekend) = \mathbf{0.292}$ and $G(D_{>100}, Sun) = \mathbf{0.170}$
- The node thus gets the attribute *Weekend* assigned. For *Weekend = no* the tree terminates with the decision *Ski = no*.
- A calculation of the gain here returns the value **0**. For *Weekend=yes*, Sun results in a gain of **0.171**.
- Then the construction of the tree terminates because no further attributes are available, although example *number 7* is *falsely* classified.

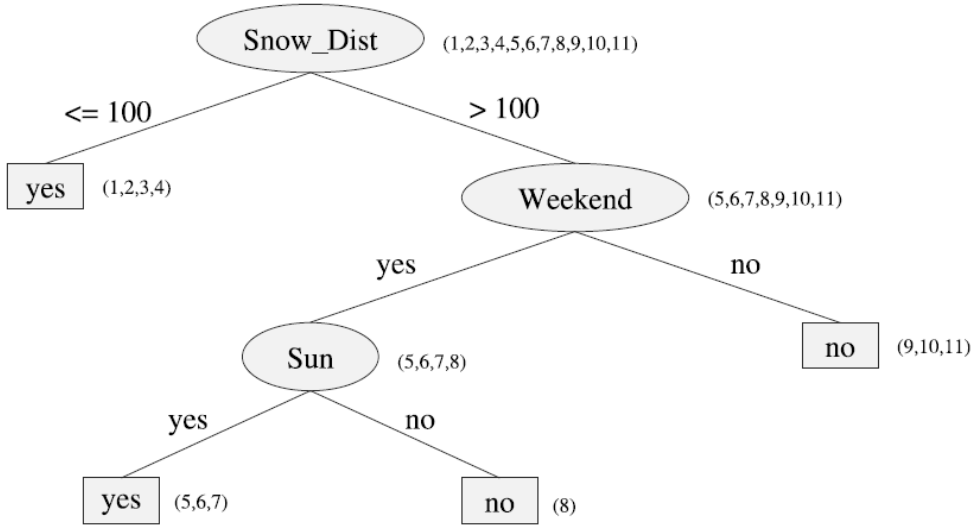


Figure 2.23: Decision tree for the skiing classification problem

Chapter 2: Machine Learning & Data Mining

8. Decision Tree Learning – Decision Tree Algorithm

- The algorithm for the construction of a decision tree is given in **Figure 2.26**.

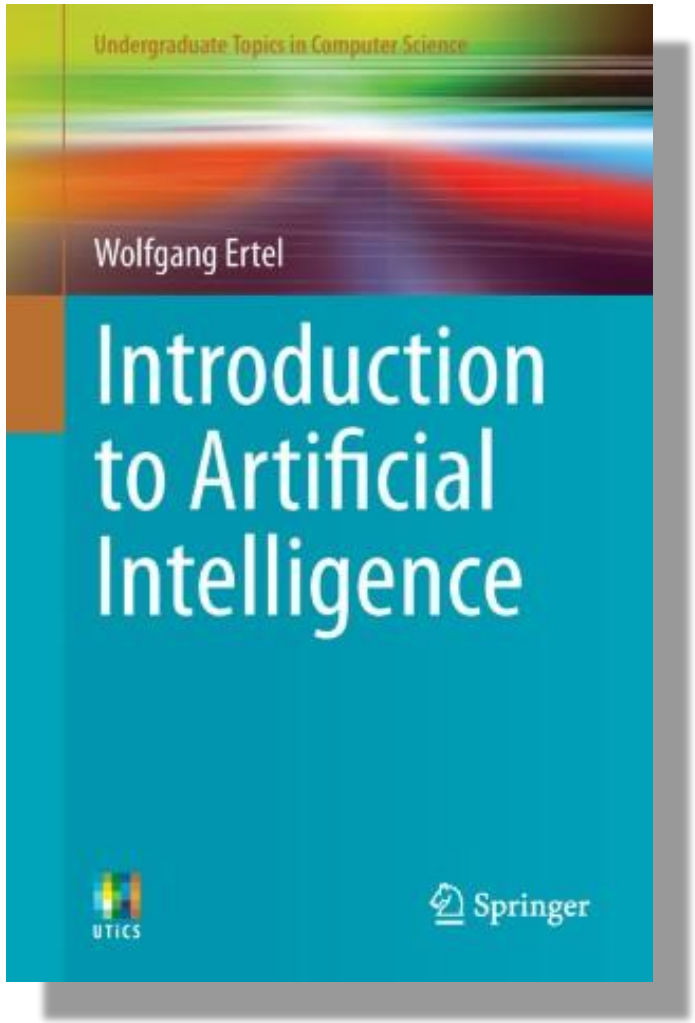
GENERATEDECISIONTREE(*Data*,*Node*)

A_{max} = Attribute with maximum information gain
If $G(A_{max}) = 0$
  Then *Node* becomes leaf node with most frequent class in *Data*
  Else assign the attribute A_{max} to *Node*
    For each value $a_1, \ldots, a_n$ of A_{max}, generate
      a successor node: $K_1, \ldots, K_n$
    Divide *Data* into $D_1, \ldots, D_n$ with $D_i = \{x \in Data | A_{max}(x) = a_i\}$
    For all $i \in \{1, \ldots, n\}$
      If all $x \in D_i$ belong to the same class C_i
        Then generate leaf node K_i of class C_i
        Else GENERATEDECISIONTREE(D_i, K_i)

Figure 2.26: Algorithm for the construction of a decision tree

AI Fundamentals

End of Chapter 2- Machine Learning & Data Mining



hogeschool VIVES

VIVES University of Applied Sciences
Bachelor in electronics-ICT